VERIQTA

CKA HANDBOOK

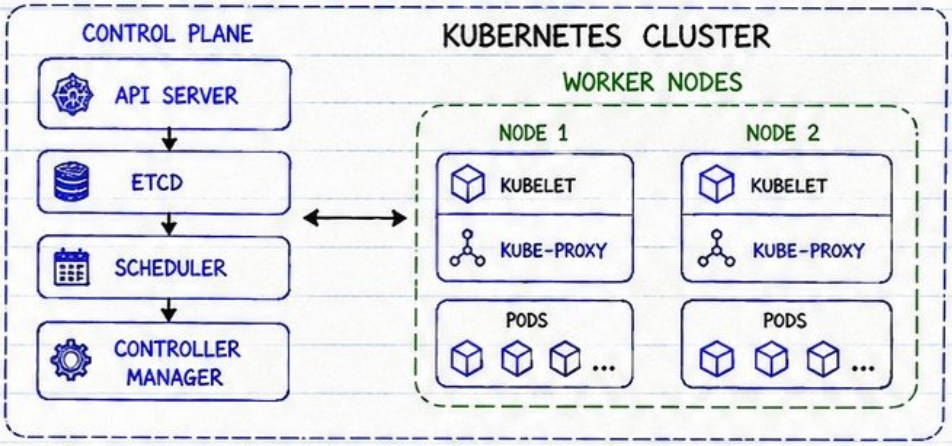
CERTIFIED KUBERNETES ADMINISTRATOR (CKA) HANDBOOK

THE ULTIMATE EXAM PREPARATION GUIDE

- ☑ EXAM FOCUSED
- ☑ PRACTICAL
- ☑ STRUCTURED
- ☑ FAST RECALL
- ☑ TROUBLESHOOTING
- ☑ PROVEN STRATEGY

60-PAGE FLAGSHIP HANDBOOK

MASTER THE CKA EXAM. BECOME A KUBERNETES ADMINISTRATOR.



EXAM DOMAINS COVERED

- Cluster Architecture, Installation & Configuration
- Workloads & Scheduling
- Services & Networking
- Storage
- Security
- Troubleshooting
- Cluster Maintenance

EXAM FOCUSED
Covers all CKA exam tasks and objectives.

PRACTICAL
Real-world commands, scenarios & workflows.

FAST RECALL
Cheat sheets, YAML patterns & shortcuts.

TROUBLESHOOTING
Step-by-step debugging for exam scenarios.

PROVEN STRATEGY
Structured plan to pass the CKA exam.

PURPOSE

- ☑ This handbook is not a Kubernetes tutorial.
- ☑ It assumes you already understand the fundamentals of Kubernetes.
- ☑ Its goal is to help candidates pass the CKA examination through structured revision, fast recall, practical administration, troubleshooting, and exam-focused workflows.

VERIQTA | veriqta youtube | veriqta github | veriqta linkedin | veriqta x | veriqta instagram | veriqta telegram

LEARN. PRACTICE. ANALYZE. TROUBLESHOOT. SUCCEED.

1. CKA EXAM OVERVIEW

The CKA exam validates your ability to perform the day-to-day operational tasks of a Kubernetes Administrator.

CKA EXAM DOMAINS

- Cluster Architecture, Installation & Configuration
- Workloads & Scheduling
- Services & Networking
- Storage
- Troubleshooting
- Security

☆ Focus more on hands-on skills than theory.

EXAM SCORING

- ☑ Pass / Fail: 66% or higher
- ☑ Total Questions: 15+ tasks
- ☑ Performance-based (Hands-on)
- ☑ Each task has hidden sub-tasks
- ☑ Partial credit available
- ☑ Results available immediately after exam

TIME MANAGEMENT STRATEGY

Total Time: 120 Minutes
Read & Plan: 10–15 min
Easy Wins First: 40–50 min
Medium Tasks: 40–50 min
Review & Verify: 10–15 min
Buffer Time: Always keep 5–10 min

Don't get stuck! Skip and return later. Make progress, then perfect.

REMOTE EXAM ENVIRONMENT

- Online proctored exam from home or quiet private space.
- Webcam, microphone, and screen recording are required.
- Room scan required before starting the exam.
- Valid government-issued ID required.
- Stable internet connection (min. 5 Mbps recommended).

ALLOWED RESOURCES

- ☑ Official Kubernetes Documentation https://kubernetes.io/docs
- ☑ kubectl Command Line (Pre-configured)
- ☑ Built-in Kubernetes Tools (e.g., vim, grep, awk, etc.)
- ☑ No additional websites, notes, or help from others

Read docs fast. Practice commands faster.

STUDY ROADMAP

1. **Foundation Refresh** — Review key concepts, architecture, and core components.
2. **Domain Deep Dive** — Study each domain with hands-on practice.
3. **Hands-on Labs** — Perform real tasks until you are fast and confident.
4. **Practice Exams** — Attempt mock exams under time pressure.
5. **Review & Weakness Fix** — Analyze mistakes and strengthen weak areas.
6. **Final Revision** — Rapid recall, key commands, and exam mindset.

EXAM DAY CHECKLIST

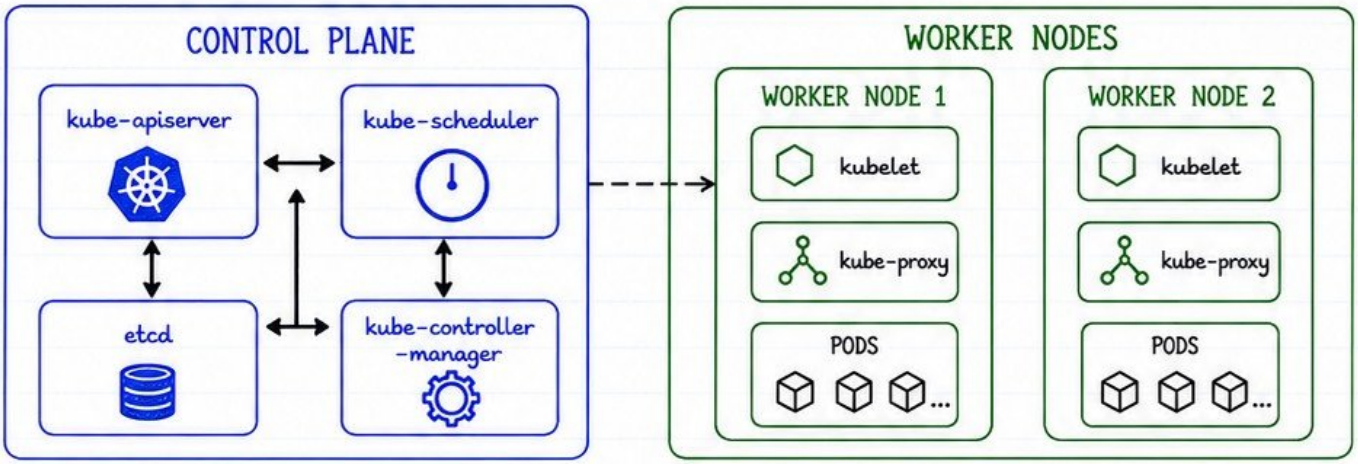
- ☑ Get good sleep
- ☑ Have your ID ready
- ☑ Ensure stable internet
- ☑ Quiet, well-lit room
- ☑ Relax, stay confident, execute!

☆ KEY TO SUCCESS

Understand the task. | Plan your approach. | Execute with precision. | Verify your results. | Stay calm. Stay focused. Pass the CKA!

2. KUBERNETES ARCHITECTURE REVIEW

Kubernetes follows a master-worker architecture. The control plane manages the cluster and worker nodes run your applications.

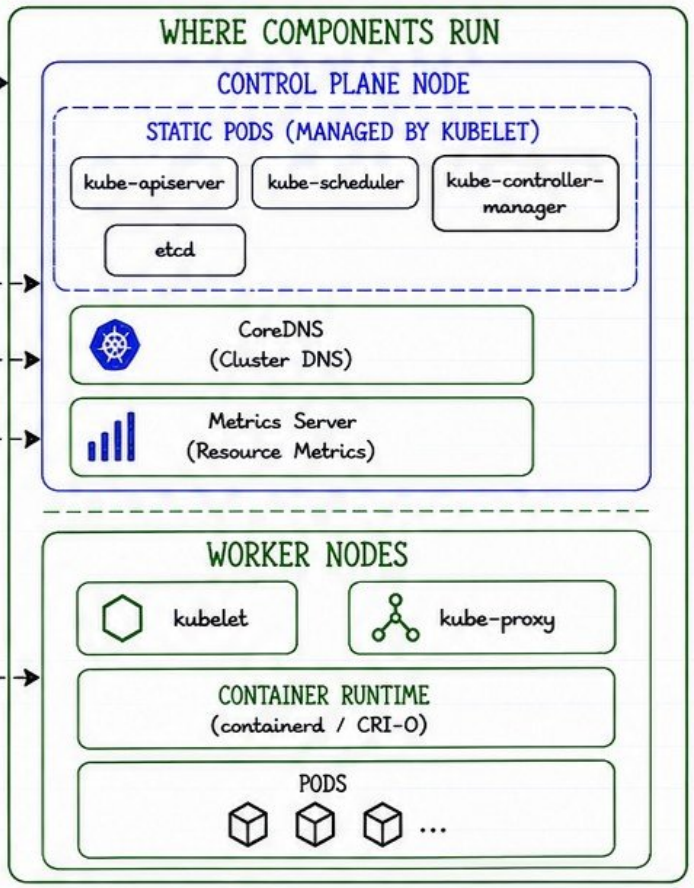


CONTROL PLANE	• The brain of the cluster. • Makes global decisions about the cluster. • Manages workloads, networking, storage, and cluster state.
WORKER NODES	• Machines that run your applications. • Hosted containers as Pods. • Communicate with the control plane.
ETCD	• Consistent and highly-available key-value store. • Stores all cluster data and state. • Critical for cluster reliability.
API SERVER	• Frontend for the Kubernetes control plane. • Exposes the Kubernetes API. • Validates and processes all requests.
SCHEDULER	• Watches for newly created Pods without a node. • Selects the best node for the Pod to run on. • Considers resources, policies, affinity, and taints.
CONTROLLER MANAGER	• Runs controller processes. • Ensures the actual state matches the desired state. • Examples: Node Controller, ReplicaSet Controller, etc.
KUBELET	• Runs on every worker node. • Ensures containers are running in a Pod. • Reports node and Pod status to the control plane.
KUBE-PROXY	• Runs on every worker node. • Maintains network rules on the node. • Enables Service networking and load balancing.

3. CLUSTER COMPONENTS

Kubernetes is built from modular components. Each component has a specific role and together they keep the cluster running.

STATIC PODS	Pods managed directly by the kubelet on the control plane node.
COREDNS	Cluster DNS service that provides name resolution for services.
METRICS SERVER	Collects resource metrics from kubelet and exposes them to the API server.
CONTAINER RUNTIME	Runs containers as defined by Kubernetes. (e.g., containerd, CRI-O)
CRI (CONTAINER RUNTIME INTERFACE)	Standard interface between kubelet and the container runtime.
ADD-ONS	Additional components that extend the cluster functionality.
COMPONENT TROUBLESHOOTING	Identify and resolve issues with cluster components quickly.



COMPONENT TROUBLESHOOTING QUICK GUIDE

COMPONENT	COMMON SYMPTOMS	CHECK COMMANDS	FIX APPROACH
kube-apiserver	API not reachable, kubectl timeout	systemctl status kube-apiserver journalctl -u kube-apiserver	Check logs, certificates, network, etcd connectivity
etcd	Cluster down, data inconsistent	systemctl status etcd etcdctl endpoint health	Check disk, logs, restore from backup
kube-scheduler	Pods pending, not scheduled	systemctl status kube-scheduler journalctl -u kube-scheduler	Check logs, node resources, scheduling rules
kube-controller-manager	Controllers not working, resources not created	systemctl status kube-controller-manager journalctl -u kube-controller-manager	Check logs, RBAC, API connectivity
CoreDNS	DNS resolution failing	kubectl -n kube-system get pods -l k8s-app=kube-dns kubectl -n kube-system logs -l k8s-app=kube-dns	Check CoreDNS pods, configmap, network
kubelet	Node NotReady, Pods not running	systemctl status kubelet journalctl -u kubelet	Check logs, runtime, disk, network
Container Runtime	Pods stuck in ContainerCreating, runtime errors	systemctl status containerd crictl info	Restart runtime, check config, disk, logs

TIP: Always check logs first, then events, then configuration and resource state.
Use: `kubectl describe <resource>` and `kubectl get events --sort-by=.metadata.creationTimestamp`

4. KUBECTL MASTERY

kubectl is your primary tool in the CKA exam. Master these commands for speed, accuracy, and confidence.

1. ESSENTIAL COMMANDS

COMMAND	DESCRIPTION
kubectl get pods	List all Pods in the current namespace
kubectl get pods -A	List Pods in all namespaces
kubectl get nodes	List all nodes
kubectl describe <resource> <name>	Show detailed information
kubectl logs <pod>	View logs from a Pod
kubectl exec -it <pod> -- <cmd>	Execute a command in a container
kubectl delete <resource> <name>	Delete a resource
kubectl apply -f <file.yaml>	Create or update from a file
kubectl edit <resource> <name>	Edit resource in-place
kubectl scale deployment <name> --replicas=3	Scale a deployment
kubectl rollout status deployment/<name>	Check rollout status
kubectl rollout undo deployment/<name>	Rollback a deployment
kubectl get all	List common resources
kubectl get all -A	List all resources in all namespaces

2. OUTPUT FORMATTING

FORMAT	EXAMPLE
Wide output	kubectl get pods -o wide
Custom columns	kubectl get pods -o custom-columns= NAME:.metadata.name,IP:.status.podIP,NODE:.spec.nodeName
No headers	kubectl get pods -o custom-columns= NAME:.metadata.name --no-headers
YAML output	kubectl get pod <name> -o yaml
JSON output	kubectl get pod <name> -o json

3. JSONPATH

Extract specific fields using JSONPath.

EXAMPLES	
kubectl get pods -o jsonpath='{.items[*].metadata.name}'	List pod names
kubectl get pods -o jsonpath='{.items[*].status.podIP}'	List pod IPs
kubectl get nodes -o jsonpath='{.items[*].status.capacity.cpu}'	List node CPU capacity
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}'	Pretty print pod names

4. DRY-RUN

Test your changes without applying them.

EXAMPLES	
kubectl apply -f pod.yaml --dry-run=client	Validato YAML locally
kubectl create deployment web --image=nginx --dry-run=client -o yaml	Preview generated YAML
kubectl apply -f deploy.yaml --dry-run=server	Validate with API server

5. EXPLAIN

Get documentation for Kubernetes resources.

EXAMPLES	
kubectl explain pods	Show Pod documentation
kubectl explain pod.spec.containers	Show containers field docs
kubectl explain deployment.spec.template.spec	Show nested field docs

6. ALIASES (EXAMPLE SET)

Create aliases to save time.

EXAMPLES

alias k='kubectl'	Short alias for kubectl
alias kg='kubectl get'	Quick get
alias kga='kubectl get all'	Get all resources
alias kl='kubectl logs -f'	Follow logs
alias kd='kubectl describe'	Describe resource
alias ka='kubectl apply -f'	Apply from file
alias kdel='kubectl delete'	Delete resource
alias kx='kubectl exec -it'	Exec into pod

7. EXAM SHORTCUTS

- ☑ Always set the correct context and namespace first.
  - kubectl config use-context <context>
  - kubectl config set-context --current --namespace=<ns>
- ☑ Use TAB completion to save time.
- ☑ Use history and ↑↓ arrows to repeat commands.
- ☑ Validate YAML before applying.
  - kubectl apply -f file.yaml --dry-run=client
- ☑ Use --record flag when creating resources (legacy but useful).
  - kubectl run nginx --image=nginx --record
- ☑ Master resource types: pods, deployments, services, configmaps, secrets, nodes, namespaces, roles, rolebindings, pv, pvc, daemonsets.

KEY REMINDER

Speed comes from repetition.
Accuracy comes from understanding.
Confidence comes from practice.

MASTER COMMANDS.
PASS THE CKA.

5. YAML MASTERY

YAML is the language of Kubernetes. Master its structure, fields, and editing for fast, accurate, and exam-ready performance.

1. API VERSIONS

Kubernetes uses API groups and versions to manage resources.

TYPE	EXAMPLE
Core API Group	v1
Named API Group	apps/v1
Beta API Group	extensions/v1beta1
Custom Resource	apiextensions.k8s.io/v1

Check the correct API version for each resource in the exam.

2. METADATA

Metadata identifies and organizes Kubernetes objects.

```yaml
metadata:
  name: my-app
  namespace: default
  labels:
    app: my-app
    tier: frontend
  annotations:
    description: frontend component
```

Labels and annotations are critical for selection and management.

3. SPEC

Spec defines the desired state of the resource.

```yaml
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
```

This is what you declare. Kubernetes works to match this state.

4. STATUS

Status shows the current state of the resource (managed by Kubernetes).

```yaml
status:
  observedGeneration: 1
  replicas: 3
  updatedReplicas: 3
  readyReplicas: 3
  availableReplicas: 3
  conditions:
  - type: Available
    status: "True"
    lastTransitionTime: "2024-05-01T10:00:002"
```

You rarely edit status. It is set by the system. Use it to verify and troubleshoot.

5. COMMON MANIFESTS

Know the structure of these core resources.

RESOURCE	PURPOSE
Pod	Run one or more containers
Deployment	Manage stateless applications
Service	Expose and connect applications
ConfigMap	Store non-sensitive configuration
Secret	Store sensitive information
PersistentVolumeClaim	Request storage
Namespace	Isolate resources
Job	Run batch tasks
Ingress	Expose HTTP/HTTPS routes

6. FAST EDITING TECHNIQUES

Speed in editing YAML saves time in the exam.

- ☑ Use kubectl create with --dry-run=client -o yaml
  Example:
  ```
  kubectl create deployment my-app --image=nginx \
  --dry-run=client -o yaml > deploy.yaml
  ```
- ☑ Edit efficiently in vim
  i → insert
  Esc → normal mode
  :wq → save and exit
  yy → copy line, dd → delete line
- ☑ Indentation matters
  Use 2 spaces. Wrong indentation = invalid YAML.
- ☑ Validate before applying
  ```
  kubectl apply -f file.yaml --dry-run=client
  ```

7. COMMON MANIFEST STRUCTURES (QUICK REFERENCE)

POD

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports:
    - containerPort: 80
```

DEPLOYMENT

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
```

SERVICE

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 80
```

CONFIGMAP

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  app.properties: |
    key1=value1
    key2=value2
```

SECRET

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  username: YWRtaW4=
  password: cGFzc3dvcmQ=
```

These are the most tested manifest types in the CKA exam.

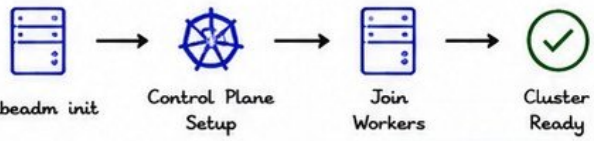
YAML MASTER TIPS

- ☑ Always use the correct apiVersion.
- ☑ Check field names carefully.
- ☑ Follow the resource schema.
- ☑ Use dry-run to generate YAML.
- ☑ Validate before applying.
- ☑ Read error messages and adjust quickly.
- ☑ Practice editing under time pressure.

6. CLUSTER INITIALIZATION REVIEW

The kubeadm workflow is the recommended way to install, configure, and manage Kubernetes clusters.

1. KUBEADM WORKFLOW



1. Initialize the control plane node.
2. Configure kubectl for the admin user.
3. Join worker nodes to the cluster.
4. Verify the cluster is healthy.

2. CERTIFICATES

Certificates secure all communications in the cluster.

CERTIFICATE	PURPOSE
apiserver.crt	API Server HTTPS certificate
apiserver-kubelet-client.crt	API Server to Kubelet auth
front-proxy-client.crt	Front proxy client certificate
etcd/server.crt	etcd server certificate
etcd/peer.crt	etcd peer communication
ca. crt	Cluster Certificate Authority

Certificates are stored in `/etc/kubernetes/pki`

3. BOOTSTRAP TOKENS

Bootstrap tokens allow new nodes to join the cluster securely.

COMMAND	PURPOSE
`kubeadm token create`	Create a new bootstrap token
`kubeadm token list`	List all tokens
`kubeadm token delete <token>`	Delete a token
`kubeadm token describe <token>`	Show token details

Tokens have TTL. Use strong tokens for production.

4. CONTROL PLANE SETUP

1. Initialize the control plane:
   ```
   kubeadm init --pod-network-cidr=10.244.0.0/16
   ```
2. Configure kubectl for the admin user:
   ```
   mkdir -p $HOME/.kube
   sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
   sudo chown $(id -u):$(id -g) $HOME/.kube/config
   ```
3. Verify the control plane components:
   ```
   kubectl get pods -n kube-system
   ```
4. Install Pod network (example: Calico):
   ```
   kubectl apply -f calico.yaml
   ```
5. Verify node status:
   ```
   kubectl get nodes
   ```
6. Control plane is ready when all pods are Running.

5. NODE JOIN

Join worker nodes to the cluster using the token generated during kubeadm init.

On the control plane node, get the join command:

```
kubeadm token create --print-join-command
```

Example output:

```
kubeadm join 192.168.1.10:6443 --token abcdef.1234567890abcdef \
--discovery-token-ca-cert-hash sha256:xxxxxxxxxxxxxxxxxxxx
```

Run this command on each worker node.

Verify:

```
kubectl get nodes
```

QUICK VERIFICATION CHECKLIST

- [x] Control plane node status is Ready
- [x] All control plane pods are Running
- [x] Pod network is installed and functional
- [x] Worker nodes are joined and Ready
- [x] DNS (CoreDNS) pods are Running
- [x] Services and system components are healthy

COMMON KUBEADM COMMANDS CHEAT SHEET

```
kubeadm init \
  --pod-network-cidr=10.244.0.0/16
kubeadm init phase list
kubeadm init phase <phase-name>
kubeadm reset -f
```

```
kubeadm token create
kubeadm token list
kubeadm token delete <token>
kubeadm token describe <token>
```

```
kubeadm join <ip>:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
kubeadm upgrade plan
kubeadm upgrade apply v1.x.x
```

KEY TAKEAWAY kubeadm simplifies Kubernetes cluster setup. Understand each step, the components involved, and how nodes securely join the cluster.

MASTER THE WORKFLOW.
PASS THE CKA.

7. MANAGING CLUSTER NODES

Managing nodes is a critical skill on the CKA exam. You must know how to label, taint, cordon, drain, and maintain nodes safely.

1. LABELS

Labels are key/value pairs attached to nodes to organize and select resources.

COMMON LABELS	PURPOSE
kubernetes.io/hostname	Node hostname
kubernetes.io/os	Operating system
kubernetes.io/arch	Architecture
node-role.kubernetes.io/control-plane	Control plane node
node-role.kubernetes.io/worker	Worker node

COMMANDS

```
kubectl label node <node-name> key=value
kubectl label node <node-name> key=value --overwrite
kubectl label node <node-name> key-
kubectl get nodes --show-labels
```

☆ Labels are used in nodeSelector, nodeAffinity, and for organization.

2. TAINTS

Taints allow you to repel Pods from nodes unless they tolerate the taint.

TAINT EFFECTS

EFFECT	DESCRIPTION
NoSchedule	Do not schedule Pods without toleration
PreferNoSchedule	Try not to schedule Pods without toleration
NoExecute	Do not schedule and evict existing Pods

COMMANDS

```
kubectl taint node <node> key=value:NoSchedule
kubectl taint node <node> key=value:PreferNoSchedule
kubectl taint node <node> key=value:NoExecute
kubectl taint node <node> key-
```

Pods must have matching tolerations to run on tainted nodes.

3. CORDONING

Cordoning marks a node as unschedulable. No new Pods will be scheduled.

COMMANDS

```
kubectl cordon <node-name>
kubectl get nodes
```

EXAMPLE OUTPUT

```
NAME    STATUS                     ROLES          AGE    VERSION
node1   Ready                      control-plane  10d    v1.28.2
node2   Ready                      <none>         10d    v1.28.2
node3   Ready,SchedulingDisabled   <none>         10d    v1.28.2
```

☆ Cordon before maintenance to prevent new Pods from being scheduled.

4. DRAINING

Draining evicts Pods from a node safely so it can be maintained.

COMMAND

```
kubectl drain <node-name> \
  --ignore-daemonsets \
  --delete-emptydir-data \
  --force
```

WHAT DRAIN DOES

- Evicts all non-DaemonSet Pods
- Respects PodDisruptionBudgets
- Deletes local data (emptyDir)
- Prepares node for maintenance

⚠ Always cordon before drain.

5. UNCORDONING

After maintenance, uncordon the node to allow scheduling again.

COMMAND

```
kubectl uncordon <node-name>
```

VERIFY

```
kubectl get nodes
```

EXAMPLE OUTPUT

```
NAME    STATUS   ROLES          AGE   VERSION
node1   Ready    control-plane  10d   v1.28.2
node2   Ready    <none>         10d   v1.28.2
node3   Ready    <none>         10d   v1.28.2
```

Node is schedulable again.

6. NODE MAINTENANCE WORKFLOW

Standard workflow for safe node maintenance.

STEPS

1. Check node status
   `kubectl get nodes`
2. Cordon the node
   `kubectl cordon <node>`
3. Drain the node
   `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --force`
4. Perform maintenance (OS updates, patches, etc.)
5. Verify node health
6. Uncordon the node
   `kubectl uncordon <node>`
7. Verify Pods are running on the node

QUICK REFERENCE TABLE

ACTION	PURPOSE	COMMAND	NOTES
Label node	Organize and select nodes	kubectl label node <node> key=value	Used by nodeSelector, affinity
Taint node	Prevent Pods from scheduling	kubectl taint node <node> key=value:NoSchedule	Requires toleration
Cordon node	Stop new Pods from scheduling	kubectl cordon <node>	Does not evict existing Pods
Drain node	Evict Pods and prepare for maintenance	kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --force	Evicts non-DaemonSet Pods
Uncordon node	Allow scheduling again	kubectl uncordon <node>	Node becomes schedulable

KEY TAKEAWAY

Mastering node management ensures high availability, smooth maintenance, and exam success.

PRACTICE. REPEAT. MASTER.
YOU WILL PASS THE CKA.

8. NAMESPACES & CONTEXTS

Namespaces and contexts help you isolate resources and work across multiple clusters with speed and clarity.

1. NAMESPACE ISOLATION

Namespaces divide resources within a cluster. They providle scope, isolation, and access control.

DEFAULT NAMESPACES	PURPOSE
default	Default namespace for resources
kube-system	System components namespace
kube-public	Publicly readable resources
kube-node-lease	Node heartbeat leases

COMMANDS

kubectl get namespaces	List all namespaces
kubectl create namespace <ns>	Create a namespace
kubectl delete namespace <ns>	Delete a namespace
kubectl get all -n <ns>	List all resources in namespace
kubectl describe namespace <ns>	Show namespace details

Use namespaces to separate environments (dev, test, prod) or teams within the same cluster.

2. CONTEXT SWITCHING

Contexts define which cluster and user you are working with.

COMMANDS

kubectl config get-contexts	List all contexts
kubectl config current-context	Show current context
kubectl config use-context <ctx>	Switch to context
kubectl config set-context <ctx> \ --namespace=<ns>	Set default namespace for context

EXAMPLE OUTPUT

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	dev-cluster	dev-cluster	dev-user	dev
	test-cluster	test-cluster	test-user	test
	prod-cluster	prod-cluster	prod-user	prod

Switching context instantly changes your target cluster and user.

3. KUBECONFIG

kubeconfig stores clusters, users, and contexts.

STRUCTURE

klusters	→	Cluster connection details
users	→	Authentication credentials
contexts	→	Mapping of user + cluster
current-context	→	Active context
preferences	→	User preferences

COMMANDS

echo $KUBECONFIG	Show kubeconfig path
kubectl config view	View full kubeconfig
kubectl config view --minify	View current context only
kubectl config set-context <ctx> \ --cluster=<c> --user=<u>	Create a context
kubectl config unset contexts.<ctx>	Remove a context
kubectl config delete-context <ctx>	Delete a context

Default path: =/.kube/config on Linux/macOS
C:\Users\<user>\.kube\config on Windows

4. MULTIPLE CLUSTERS

Manage many clusters from one kubeconfig file.

COMMANDS

kubectl config get-clusters	List all clusters
kubectl config use-context <ctx>	Switch to cluster via context
kubectl cluster-info	Show current cluster info
kubectl get nodes	List nodes in current cluster
kubectl config rename-context <old> <new>	Rename a context
kubectl config delete-cluster <cluster>	Remove cluster from config

BEST PRACTICES

- Keep context names meaningful (e.g., prod-us-east-1).
- Use separate users for each cluster.
- Do not mix admin and read-only users.
- Regularly clean unused clusters and contexts.

5. FAST NAVIGATION

Speed is everything in the CKA exam.

TIPS

- [x] Use short context names
- [x] Set default namespace per context
- [x] Use aliases for frequent commands
- [x] Minimize typing
- [x] Verify your context before executing destructive commands

QUICK ACTIONS

```
kubectl config get-contexts
kubectl config use-context <ctx>
kubectl get nodes
kubectl get all -A
kubectl config current-context
```

6. EXAM SHORTCUTS

Commands and tricks that save time.

SHORTCUTS

- [x] Always verify context first:
  `kubectl config current-context`
- [x] List resources in all namespaces:
  `kubectl get all -A`
- [x] Set namespace for current context:
  `kubectl config set-context $(kubectl config current-context) --namespace=<ns>`
- [x] Switch context and namespace in one line:
  `kubectl config use-context <ctx> && kubectl config set-context <ctx> \ --namespace=<ns>`
- [x] Use aliases to save time (see below).

QUICK REFERENCE TABLE

TOPIC	COMMAND	PURPOSE	EXAMPLE
List namespaces	kubectl get namespaces	Show all namespaces	kubectl get namespaces
Create namespace	kubectl create namespace <ns>	Create a new namespace	kubectl create namespace dev
Switch context	kubectl config use-context <ctx>	Switch to another context	kubectl config use-context prod-cluster
Set namespace for context	kubectl config set-context <ctx> --namespace=<ns>	Set default namespace	kubectl config set-context prod-cluster --namespace=prod
Check current context	kubectl config current-context	Show active context	kubectl config current-context
Resources in all namespaces	kubectl get all -A	List all resources cluster-wide	kubectl get all -A

KEY TAKEAWAY

Control your scope and speed.
Master namespaces and contexts to work across any environment with confidence.

FOCUS. PRACTICE. MASTER.
YOU WILL PASS THE CKA.

9. RESOURCE QUOTAS & LIMITS

Resource quotas and limits ensure fair usage, prevent noisy neighbors, and keep your cluster stable and predictable.

1. RESOURCEQUOTA

ResourceQuota sets aggregate limits on total resources in a namespace.

KEY FEATURES

- Limits total CPU, memory, storage, objects.
- Prevents quota exceed errors.
- Enforces governance at the namespace level.

EXAMPLE MANIFEST

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
    persistentvolumeclaims: "10"
    services: "10"
    configmaps: "20"
```

If a new object creation exceeds the quota, it will be rejected.

2. LIMITRANGE

LimitRange sets default or min/max limits for individual resources in a namespace.

KEY FEATURES

- Defines default requests and limits.
- Sets min/max constraints.
- Ensures Pods have proper resource settings.

EXAMPLE MANIFEST

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: dev
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: 512Mi
    defaultRequest:
      cpu: "250m"
      memory: 256Mi
    min:
      cpu: "100m"
      memory: 128Mi
    max:
      cpu: "2000m"
      memory: 2Gi
```

Applies to all Pods in the namespace unless explicitly overridden.

3. NAMESPACE GOVERNANCE

Use quotas and limits to enforce governance and protect cluster stability.

BEST PRACTICES

- Apply ResourceQuota to every team namespace.
- Use LimitRange to set sensible defaults.
- Monitor usage and adjust quotas.
- Prevent a single team from consuming all resources.

TYPICAL GOVERNANCE MODEL

POLICY	PURPOSE
ResourceQuota	Control total resource consumption per namespace
LimitRange	Enforce per-container resource standards
RBAC	Restrict who can create or modify quota objects
Monitoring	Track usage and alert on thresholds

Governance = Stability + Fairness + Security

4. COMMON QUOTA SCENARIOS

SCENARIO	DESCRIPTION
Team isolation	Each team has its own namespace with strict quotas.
Dev/Test separation	Lower quotas for dev/test namespaces to save resources.
Production protection	High quotas for prod with strict object limits.
Storage control	Limit total PVCs and storage consumption.
Object explosion	Limit number of Pods, Services, ConfigMaps, Secrets, etc.

⚠ Always test quotas in dev before applying to prod.

5. CHECK QUOTA & LIMITS STATUS

USEFUL COMMANDS

`kubectl get resourcequota -n dev`	List quotas in namespace
`kubectl describe resourcequota compute-quota -n dev`	View quota details
`kubectl get limitrange -n dev`	List limit ranges
`kubectl describe limitrange default-limits -n dev`	View limit range details
`kubectl get pods -n dev`	Check pods and their usage
`kubectl top pods -n dev`	See current resource usage (requires metrics-server)

Always verify quotas before deploying large workloads.

6. EXAM SCENARIOS

PRACTICE QUESTIONS

1. Create a ResourceQuota that limits namespace to 2 CPU cores and 4Gi memory.
2. Set a LimitRange with default CPU 500m and memory 512Mi.
3. Prevent a namespace from creating more than 10 Pods and 5 PVCs.
4. A Pod creation fails with "exceeded quota". How do you troubleshoot?
5. Explain the difference between ResourceQuota and LimitRange.
6. How do you view current resource usage in a namespace?

RESOURCE QUOTA OBJECTS REFERENCE

OBJECT	PURPOSE	EXAMPLE KEY	EXAMPLE VALUE	DESCRIPTION
pods	Limit number of Pods	pods	"20"	Maximum Pods in namespace
requests.cpu	Limit total CPU requests	requests.cpu	"4"	Total requested CPU
requests.memory	Limit total memory requests	requests.memory	8Gi	Total requested memory
limits.cpu	Limit total CPU limits	limits.cpu	"8"	Total CPU limits
limits.memory	Limit total memory limits	limits.memory	16Gi	Total memory limits
persistentvolumeclaims	Limit PVC count	persistentvolumeclaims	"10"	Maximum PVCs
services	Limit Services count	services	"10"	Maximum Services
configmaps	Limit ConfigMaps count	configmaps	"20"	Maximum ConfigMaps
secrets	Limit Secrets count	secrets	"20"	Maximum Secrets

KEY TAKEAWAY

Use ResourceQuota to control overall usage.
Use LimitRange to set per-container standards.
Together, they keep your cluster stable and fair.

MASTER GOVERNANCE.
PASS THE CKA.

10. CLUSTER UPGRADE

Upgrading a Kubernetes cluster safely is critical in the CKA exam and in production.
Know the workflow, order, and validation steps.

1. KUBEADM UPGRADE WORKFLOW

1. Review current version and available upgrades.
2. Upgrade the control plane components.
3. Upgrade worker nodes one by one.
4. Validate cluster and application health.
5. Clean up deprecated components.

CORE PRINCIPLES

- Upgrade control plane before worker nodes.
- Upgrade one node at a time.
- Validate at every stage.
- Always have a rollback plan.

2. NODE UPGRADE PROCESS

Upgrade worker nodes one at a time.

1. Cordon the node
   `kubectl cordon <node>`
2. Drain the node
   `kubectl drain <node> --ignore-daemonsets \`
   `--delete-emptydir-data`
3. Upgrade kubeadm, kubelet, kubectl
   `apt-get update && apt-get install -y \`
   `kubeadm=<version> kubelet=<version> kubectl=<version>`
4. Run kubeadm upgrade node
   `kubeadm upgrade node`
5. Restart kubelet
   `systemctl restart kubelet`
6. Uncordon the node
   `kubectl uncordon <node>`
7. Validate node is Ready
   `kubectl get nodes`

3. VERSION SKEW RULES

Kubernetes supports only a limited version skew between components.

COMPONENT	VERSION SKEW POLICY
kube-apiserver	Can be newer than kubelet by 1 minor version
kubelet	Can be older than kube-apiserver by 1 minor version
kube-proxy	Should match kubelet version
kubectl	Can be +/- 1 minor version from cluster version

Always check the official documentation for the exact skew policy.

4. KUBEADM UPGRADE COMMANDS

CHECK CURRENT VERSION

```
kubectl version --short
kubeadm version
kubectl get nodes
```

CHECK AVAILABLE UPGRADES

```
kubeadm upgrade plan
```

UPGRADE CONTROL PLANE

```
kubeadm upgrade apply v1.29.0
kubectl get nodes
```

CLEAN UP OLD COMPONENTS

```
kubeadm upgrade cleanup
```

5. VALIDATION STEPS

After each major step, validate the cluster.

CHECK	COMMAND
Cluster status	`kubectl get nodes`
System pods	`kubectl get pods -A`
Control plane pods	`kubectl get pods -n kube-system`
Component health	`kubectl get componentstatuses`
API server	`kubectl get --raw='/healthz'`
DNS resolution	`kubectl run -it --rm test --image=busybox \ -- nslookup kubernetes.default`
Application check	`kubectl get deployments -A`

Do not proceed to the next step until everything is healthy.

6. ROLLBACK PLANNING

Always plan before you upgrade.

ROLLBACK STRATEGY

- Upgrade one node at a time.
- If an issue occurs, drain the upgraded node and roll back to the previous version.
- For control plane rollback, re-run:
  `kubeadm upgrade apply v<previous-version>`
- Restore from snapshot if etcd is affected.
- Keep backups of /etc/kubernetes/manifests and etcd data.

KEY ROLLBACK COMMANDS

```
kubeadm upgrade apply v<previous-version>
kubeadm reset
systemctl restart kubelet
```

UPGRADE CHECKLIST

PHASE	TASK	COMMAND / ACTION	STATUS (✓)
Pre-Upgrade	Check current cluster version	`kubectl version --short`	
	Check upgrade plan	`kubeadm upgrade plan`	
	Review release notes	Read Kubernetes release notes	
Control Plane Upgrade	Upgrade control plane	`kubeadm upgrade apply vX.Y.Z`	
	Verify control plane pods	`kubectl get pods -n kube-system`	
	Validate cluster health	`kubectl get nodes`	
Worker Node Upgrade	Cordon node	`kubectl cordon <node>`	
	Drain node	`kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`	
	Upgrade node	`kubeadm upgrade node`	
	Restart kubelet	`systemctl restart kubelet`	
	Uncordon node	`kubectl uncordon <node>`	
	Verify node Ready	`kubectl get nodes`	
Post-Upgrade	Validate all system pods	`kubectl get pods -A`	
	Validate components	`kubectl get componentstatuses`	
	Test applications	Run end-to-end tests	

KEY TAKEAWAY

Upgrades test your operational maturity.
Follow the order. Validate at every step.
Stay calm, follow the process, and you will pass the CKA.

UPGRADE WITH CONFIDENCE.
PLAN. EXECUTE. VALIDATE.
YOU WILL PASS THE CKA.

11. PODS

Pods are the smallest deployable units in Kubernetes. Understanding pod behavior deeply is essential for the CKA exam and production operations.

1. POD LIFECYCLE

Pending
↓
ContainerCreating
↓
Running
↓
Succeeded
↓
Failed
↓
Unknown

LIFECYCLE EXPLANATION

- **Pending:** Pod accepted by the cluster but not yet scheduled.
- **ContainerCreating:** Images pulled, containers created.
- **Running:** Pod is bound to a node and containers are running.
- **Succeeded:** All containers exited successfully.
- **Failed:** At least one container failed.
- **Unknown:** Pod state cannot be determined.

A Pod IP is assigned when the Pod is Running.

2. INIT CONTAINERS

Init containers run before app containers start. They must complete successfully.

KEY FEATURES

- Run sequentially.
- Each must exit with status 0.
- Used for setup, migrations, or checks.
- Do not provide services to app containers.
- Failures in init containers prevent the Pod from starting.

EXAMPLE USE CASES

- Waiting for a database to be ready.
- Downloading configuration files.
- Performing schema migrations.
- Checking external service availability.

EXAMPLE YAML SNIPPET

```
initContainers:
  - name: wait-for-db
    image: busybox
    command: ['sh', '-c', 'until nc -z db 3306; do sleep 2; done']
```

3. MULTI-CONTAINER PODS

A Pod can run multiple containers that share the same network and storage.

KEY CHARACTERISTICS

- All containers share the same IP and port space.
- Share volumes for data exchange.
- Containers are started and stopped together.
- Used for sidecar, adapter, or helper patterns.

COMMON PATTERNS

PATTERN	DESCRIPTION
Sidecar	Adds/extends main container functionality (e.g., logging).
Adapter	Transforms data from main container.
Helper	Supports main container (e.g., backup, sync).
Ambassador	Handles incoming/outgoing network connections.

Use multi-container Pods only when containers are tightly coupled.

4. RESTART POLICIES

Defines how containers in a Pod should behave when they exit.

POLICY	BEHAVIOR
Always	Restart container always (default).
OnFailure	Restart only if exit code is non-zero.
Never	Do not restart the container.

USAGE GUIDE

- Use **Always** for stateless apps.
- Use **OnFailure** for batch jobs.
- Use **Never** for one-time tasks or debugging.

5. TROUBLESHOOTING PODS

Steps to identify and fix pod issues quickly.

TROUBLESHOOT WORKFLOW

1. Identify the Pod issue
   `kubectl get pods -A`
2. Check Pod description
   `kubectl describe pod <pod-name> -n <ns>`
3. Check Pod logs
   `kubectl logs <pod-name> -n <ns> [-c <container>]`
4. Check events
   `kubectl get events --sort-by=.lastTimestamp`
5. Check container status
   `kubectl get pods -n <ns> -o wide`
6. Check resource usage
   `kubectl top pod <pod-name> -n <ns>`
7. Execute into container
   `kubectl exec -it <pod-name> -n <ns> -- /bin/sh`

6. COMMON POD ISSUES & FIXES

ISSUE	POSSIBLE CAUSE & FIX
ImagePullBackOff	Wrong image or no access. Check image name and pull secret.
CrashLoopBackOff	App crash or wrong config. Check logs and fix configuration.
Pending	Scheduling issue. Check nodes, resources, taints.
Evicted	Not enough resources. Check node pressure/events.
CreateContainerError	Invalid config or missing file. Check volume mounts and env.
OOMKilled	Memory limit too low. Increase memory request/limit.

7. ESSENTIAL POD COMMANDS CHEAT SHEET

BASIC COMMANDS

```
kubectl get pods -A
kubectl get pods -n <ns> -o wide
kubectl describe pod <pod-name> -n <ns>
kubectl delete pod <pod-name> -n <ns>
kubectl get pods --show-labels
kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase
```

LOGS & DEBUGGING

```
kubectl logs <pod-name> -n <ns>
kubectl logs <pod-name> -c <container> -n <ns>
kubectl logs -f <pod-name> -n <ns>
kubectl exec -it <pod-name> -n <ns> -- /bin/sh
kubectl exec <pod-name> -n <ns> -- ls /app
```

ADVANCED COMMANDS

```
kubectl get pods -n <ns> -o yaml
kubectl get pod <pod-name> -n <ns> -o jsonpath='{.status.podIP}'
kubectl top pod <pod-name> -n <ns>
kubectl port-forward <pod-name> <local-port>:<pod-port> -n <ns>
```

KEY TAKEAWAY

Pods are the building blocks of Kubernetes.
Master pod lifecycle, init containers, multi-container patterns, restart policies, and troubleshooting to pass the CKA.

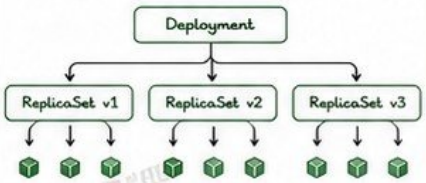
FOCUS. PRACTICE. MASTER.
YOU WILL PASS THE CKA.

12. DEPLOYMENTS

Deployments provide declarative updates for Pods and ReplicaSets.
They enable rolling updates, rollbacks, and scaling with zero downtime.

1. WHAT IS A DEPLOYMENT?

A Deployment manages ReplicaSets and ensures the desired number of Pods are running at all times.



KEY BENEFITS

- Declarative updates and rollbacks
- Self-healing Pods
- Rolling updates with zero downtime
- Scaling and version control
- Easy history tracking

You define the desired state.
Kubernetes maintains it for you.

2. ROLLOUTS (UPDATES)

Deployments update Pods gradually using a rolling update strategy.

HOW ROLLOUT WORKS

1. Create new ReplicaSet with new version
2. Scale up new ReplicaSet
3. Scale down old ReplicaSet
4. Repeat until update is complete

EXAMPLE ROLLOUT COMMANDS

```
kubectl apply -f deployment.yaml
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout pause deployment/<name>
kubectl rollout resume deployment/<name>
```

ROLLING UPDATE STRATEGY DEFAULTS

```
maxUnavailable: 25%  → max Pods unavailable
maxSurge: 25%        → extra Pods during update
```

3. ROLLBACKS

If something goes wrong, you can rollback to a previous version instantly.

ROLLBACK COMMANDS

```
kubectl rollout undo deployment/<name>
kubectl rollout undo deployment/<name> --to-revision=2
kubectl rollout history deployment/<name>
```

ROLLBACK EXAMPLE

1. View rollout history
   `kubectl rollout history deployment/app`
2. Identify revision number

REVISION	CHANGE-CAUSE
1	initial
2	image updated

3. Rollback to revision 1
   `kubectl rollout undo deployment/app --to-revision=1`
4. Verify rollback
   `kubectl rollout status deployment/app`

Rollbacks are fast because old ReplicaSets are kept until garbage collected.

4. DEPLOYMENT STRATEGY

Control how updates are rolled out.

STRATEGY	DESCRIPTION
Recreate	Terminates all old Pods before creating new ones. Causes downtime.
RollingUpdate	Default strategy. Updates Pods gradually with zero downtime.

STRATEGY CONFIGURATION (RollingUpdate)

- maxUnavailable: Max Pods unavailable at a time (e.g., 25% or 1)
- maxSurge: Max extra Pods created during update (e.g., 25% or 1)

RollingUpdate is recommended for production workloads.

5. DEPLOYMENT HISTORY

Every rollout creates a new revision.
You can track and review changes.

HISTORY COMMANDS

```
kubectl rollout history deployment/<name>
kubectl rollout history deployment/<name> --revision=2
kubectl rollout history deployment/<name> --record
```

HISTORY EXAMPLE OUTPUT

REVISION	CHANGE-CAUSE
1	kubectl apply --record
2	kubectl set image nginx=1.21
3	kubectl set image nginx=1.22
4	kubectl set resources nginx

Use history to audit changes and identify what caused an issue.

6. SCALING

Scale your application up or down easily.

SCALING COMMANDS

```
kubectl scale deployment/<name> --replicas=10
kubectl scale deployment/<name> --replicas=2
kubectl get deployment <name>
kubectl get pods -l app=<name>
```

SCALING EXAMPLE

ACTION	COMMAND
Scale up to 10	`kubectl scale deployment/app --replicas=10`
Scale down to 2	`kubectl scale deployment/app --replicas=2`
Auto scale (HPA)	`kubectl autoscale deployment/app --cpu-percent=70 --min=2 --max=10`

HPA (Horizontal Pod Autoscaler) adjusts replicas based on CPU or custom metrics.

7. DEPLOYMENT YAML EXAMPLE (NGINX)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 25%
      maxSurge: 25%
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.22
        ports:
          - containerPort: 80
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
```

DEPLOYMENT BEST PRACTICES

- [x] Use RollingUpdate strategy for production.
- [x] Always set resource requests and limits.
- [x] Use meaningful labels for selectors.
- [x] Monitor rollout status after every update.
- [x] Keep history for audit and quick rollback.
- [x] Test rollbacks periodically.
- [x] Use HPA for dynamic scaling.

KEY TAKEAWAY

Deployments give you control, safety, and speed.
Roll forward with confidence.
Roll back without panic.
Master deployments. Master the CKA.

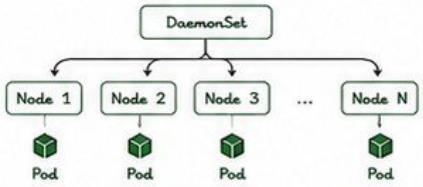
PRACTICE. ROLLOUT. RECOVER.
YOU WILL PASS THE CKA.

13. DAEMONSETS

DaemonSets ensure that a copy of a Pod runs on all (or some) nodes in the cluster. They are ideal for node-level services, monitoring, logging, and system tasks.

1. WHAT IS A DAEMONSET?

A DaemonSet creates Pods that run on every node that matches the selector.



KEY CHARACTERISTICS

- One Pod per node (by default)
- Runs as long as the node is available
- Automatically adds Pods on new nodes
- Removes Pods from nodes that no longer match
- Ideal for cluster-wide node services

If a node goes down, the Pod is removed. When the node returns, the Pod is recreated.

2. COMMON USE CASES

DaemonSets are used for node-level services that must run everywhere.

USE CASE	EXAMPLE
Logging	Fluentd, Fluent Bit
Monitoring	Node Exporter, Telegraf
Networking	CNI plugins, Calico Node
Security	Falco, OS security agents
Storage	Longhorn, OpenEBS
System Management	kube-proxy, node-problem-detector
GPU / Device Management	NVIDIA device plugin
Backup / Snapshot	Velero node agent

WHY DAEMONSETS?

- Ensure cluster-wide coverage
- Reduce manual node configuration
- Self-healing across the cluster

3. UPDATES & ROLLING UPDATE

DaemonSets support rolling updates by default.

UPDATE BEHAVIOR

- When an update is triggered, DaemonSet replaces Pods on each node one at a time.
- New Pod is scheduled before the old Pod is terminated.
- Ensures minimal impact and no downtime.

UPDATE COMMANDS

```
kubectl rollout status daemonset/<name>
kubectl rollout history daemonset/<name>
kubectl rollout undo daemonset/<name>
kubectl rollout restart daemonset/<name>
```

UPDATE STRATEGY (DEFAULT)

```
updateStrategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
```

You can also set maxUnavailable as a percentage.

4. SCHEDULING & TARGETING

Control which nodes run the DaemonSet Pods.

NODE SELECTION METHODS

- nodeSelector: Match labels
- affinity / nodeAffinity: Advanced rules
- taints & tolerations: Allow Pod on tainted nodes
- nodeName: Schedule on a specific node

EXAMPLE: RUN ON LINUX NODES ONLY

```
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      nodeSelector:
        kubernetes.io/os: linux
      tolerations:
        - operator: "Exists"
          effect: NoSchedule
```

Use tolerations carefully in production.

5. PRODUCTION EXAMPLES

Real-world DaemonSet implementations.

EXAMPLE	PURPOSE
Fluent Bit	Collect and forward container logs from every node
Node Exporter	Expose node metrics for Prometheus
Calico Node	Provide networking on each node
Falco	Runtime security monitoring
kube-proxy	Manage network rules on each node
EBS CSI Node	Enable EBS volume operations
NVIDIA Plugin	Expose GPU devices to Pods
Node Problem Detector	Detect hardware and OS issues
Velero Node Agent	Backup and snapshot node data

BEST PRACTICES

- [x] Keep DaemonSet Pods lightweight
- [x] Use resource requests and limits
- [x] Monitor DaemonSet rollout status
- [x] Test updates in staging before production
- [x] Use labels to control node targeting
- [x] Avoid running unnecessary DaemonSets

6. DAEMONSET TROUBLESHOOTING

Common problems and how to fix them.

ISSUE	POSSIBLE CAUSE & FIX
Pod not running on all nodes	Check nodeSelector, taints, tolerations and node readiness.
Pod in CrashLoopBackOff	Check Pod logs: `kubectl logs <pod> -n <ns>`
Update not rolling	Check DaemonSet updateStrategy and maxUnavailable.
Old Pods still running	Wait for rollout or check resource constraints on nodes.
Pod stuck in ContainerCreating	Check CNI, image pull, storage or node resources.
High resource usage	Set proper resource requests/limits and optimize configuration.

DEBUG COMMANDS

```
kubectl get daemonset -A
kubectl get pods -A -o wide | grep <daemonset-name>
kubectl describe daemonset <name> -n <ns>
kubectl logs <pod> -n <ns>
kubectl top node
kubectl top pod -n <ns>
```

7. DAEMONSET YAML EXAMPLE

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    app: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter:v1.8.2
          ports:
            - containerPort: 9100
              name: metrics
      tolerations:
        - operator: "Exists"
          effect: NoSchedule
```

EXPLANATION

- [x] Runs node-exporter on every Linux node
- [x] Uses toleration to allow on tainted nodes
- [x] Exposes metrics on port 9100
- [x] Automatically manages Pods on new nodes
- [x] Ensures monitoring coverage across cluster

KEY TAKEAWAY

DaemonSets are for node-level services. They keep your cluster observable, secure, and reliable across all nodes.

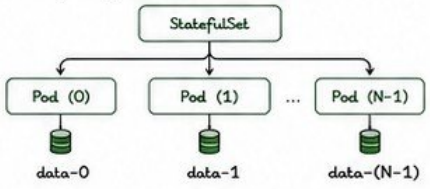
RUN EVERYWHERE. MONITOR EVERYTHING.
MASTER DAEMONSETS. MASTER THE CKA.

14. STATEFULSETS

StatefulSets manage stateful applications that require stable, unique network identity and persistent storage across restarts and rescheduling.

1. WHAT IS A STATEFULSET?

StatefulSets are used to deploy and manage stateful applications in Kubernetes.



KEY FEATURES

- Stable, unique network identity
- Persistent storage per Pod
- Ordered deployment and scaling
- Graceful deletion and recreation
- Ideal for databases and stateful apps

Use StatefulSets when data and identity must be preserved.

2. IDENTITY & NETWORK

Each Pod gets a stable hostname and DNS record that never changes.

IDENTITY COMPONENTS	
Stable Pod Name	<statefulset-name>-<ordinal> Example: web-0, web-1
Stable Hostname	<pod-name>.<headless-service> Example: web-0.web
Stable DNS	<pod-name>.<headless-service> .<namespace>.svc.cluster.local
Ordinal Index	Starts at 0 Used for ordering and identity

EXAMPLE DNS RESOLUTION

```
nslookup web-0.web.default.svc.cluster.local
nslookup web-1.web.default.svc.cluster.local
```

Stable identity is critical for clustering and replication.

3. PERSISTENT STORAGE

Each Pod gets its own PersistentVolumeClaim that is not shared with others.

STORAGE BEHAVIOR	
One PVC per Pod	Created from volumeClaimTemplates
Stable Mapping	PVC stays with the same Pod even after restart
Data Preservation	Data survives Pod reschedule, deletion, and upgrade
Scaling Down	PVCs are retained by default
Scaling Up	New Pods get new PVCs

EXAMPLE volumeClaimTemplates

```
volumeClaimTemplates:
- metadata:
    name: data
  spec:
    accessModes: ["ReadWriteOnce"]
    resources:
      requests:
        storage: 10Gi
```

Stateful data is safe only with proper persistent storage.

4. SCALING

Scaling is ordered and controlled to protect stateful data and application consistency.

SCALING RULES

1. Scale up: Pods are created in ascending order (0 → N-1).
2. Scale down: Pods are terminated in descending order (N-1 → 0).
3. New Pods get new PVCs with next ordinal.
4. Existing PVCs are never reused for other ordinals.
5. Ensure your app can handle scaling events.

EXAMPLE: SCALE TO 3 REPLICAS

```
kubectl scale statefulset web --replicas=3
```
Creates: web-0, web-1, web-2 (in order)

```
kubectl scale statefulset web --replicas=1
```
Deletes: web-2, web-1 (reverse order)

⚠ Improper scaling can lead to data inconsistency.

5. UPDATES & ROLLING UPDATE

StatefulSets perform ordered rolling updates to maintain stability.

UPDATE BEHAVIOR	
Ordered Update	Updates Pods one by one.
Wait for Ready	Next Pod updates only after previous is Ready.
Partition Support	Update only Pods with ordinal ≥ partition.
Rollback Support	Rollback to previous revision if issues occur.

EXAMPLE UPDATE COMMANDS

```
# Rolling update to new image
kubectl set image statefulset/web app=nginx:1.25

# Rollback to previous revision
kubectl rollout undo statefulset/web

# Update only Pods with ordinal >= 2
kubectl rollout status statefulset/web --partition=2
```

Always validate after each Pod update.

6. PRODUCTION EXAMPLES

StatefulSets are used for critical stateful systems.

APPLICATION	WHY STATEFULSET?
Databases	Stable identity, persistent storage, ordered updates.
Messaging Systems	Reliable node identity and persistent queues.
Distributed Caches	Consistent hashing and replication.
Search Engines	Stable data nodes and state persistence.
Time Series DB	Data retention and consistent replication.

POPULAR PATTERNS

- Headless Service + StatefulSet
- volumeClaimTemplates for storage
- Pod Disruption Budgets for availability

Use StatefulSets for systems where data and identity are critical.

7. STATEFULSET YAML EXAMPLE

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: web
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: app
        image: nginx:1.25
        ports:
        - containerPort: 80
          name: http
        volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 10Gi
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  clusterIP: None    # Headless Service
  selector:
    app: web
  ports:
  - port: 80
    targetPort: http
    name: http
```

BEST PRACTICES

- ☑ Use Headless Service for stable DNS.
- ☑ Use volumeClaimTemplates for storage.
- ☑ Monitor rollout status during updates.
- ☑ Use PodDisruptionBudgets.
- ☑ Backup stateful data regularly.
- ☑ Test scaling and failover scenarios.

KEY TAKEAWAY

StatefulSets give you stable identity, persistent storage, and ordered operations.
Perfect for databases, messaging systems, and any stateful workload.
Understand identity, storage, and update flow to pass the CKA.

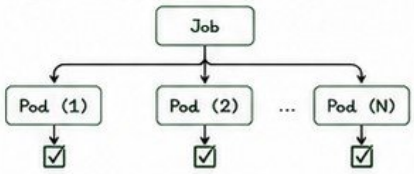
STABILITY. CONSISTENCY. RELIABILITY.
MASTER STATEFULSETS. MASTER THE CKA.

15. JOBS & CRONJOBS

Jobs and CronJobs run batch tasks in Kubernetes. They are ideal for one-time jobs, recurring tasks, data processing, and maintenance operations.

1. WHAT ARE JOBS?

A Job creates one or more Pods to run a task to completion. It ensures the task succeeds before marking complete.



KEY FEATURES

- Runs Pods until the task completes successfully.
- Automatically retries on failure.
- Ideal for batch processing and one-time tasks.
- Ensures completion before finishing.

Use Jobs for tasks that must run to completion, like database migrations or data processing.

2. CRONJOBS (SCHEDULING)

A CronJob creates Jobs on a schedule using cron syntax.

CRON SYNTAX (5 FIELDS)

*	*	*	*	*
Minute	Hour	Day	Month	Day of Week

EXAMPLE SCHEDULES

`0 2 * * *`	Every day at 2:00 AM
`*/15 * * * *`	Every 15 minutes
`0 */6 * * *`	Every 6 hours
`0 9 * * MON-FRI`	Weekdays at 9:00 AM
`0 0 1 * *`	First day of every month

CRONJOB WORKFLOW

Schedule triggers CronJob
↓
CronJob creates a Job
↓
Job creates Pods to run the task
↓
Job completes and cleans up Pods

3. JOB SPECIFICATION

Key fields used in a Job definition.

IMPORTANT FIELDS

template	Pod template that defines the task
backoffLimit	Number of retries before marking failed
completions	Number of successful completions required
parallelism	Number of Pods to run in parallel
activeDeadlineSeconds	Maximum time the Job can run
ttlSecondsAfterFinished	Clean up Job after completion

EXAMPLE JOB YAML (SIMPLE)

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: example-job
spec:
  completions: 1
  backoffLimit: 3
  template:
    spec:
      containers:
      - name: worker
        image: busybox
        command: ["sh", "-c", "echo Hello && sleep 10"]
      restartPolicy: OnFailure
```

4. BATCH PROCESSING USE CASES

Common real-world batch processing tasks.

USE CASES

- Data import/export
- Report generation
- Database backups
- Log aggregation
- Data cleanup and archival
- Machine learning model training
- ETL (Extract, Transform, Load) jobs

EXAMPLE SCENARIO

A company runs daily sales reports at 1 AM. CronJob creates a Job, which generates the report and uploads it to S3.

5. RETRIES & RELIABILITY

Jobs automatically retry on failure.

RETRY BEHAVIOR

backoffLimit: N	Retry N times before failing
OnFailure	Restart Pod on container failure
Never	Do not restart failed Pods
Active Deadline	Stop Job if it exceeds time limit

BEST PRACTICES

- ☑ Set backoffLimit to handle transient failures.
- ☑ Use meaningful Pod logs for debugging.
- ☑ Set activeDeadlineSeconds for long jobs.
- ☑ Make tasks idempotent when possible.
- ☑ Monitor Job status and failures.

6. CLEANUP & RESOURCE MANAGEMENT

Keep your cluster clean and efficient.

CLEANUP STRATEGIES

ttlSecondsAfterFinished	Auto-delete completed Jobs
Successful Jobs History Limit	Limit CronJob successful history
Failed Jobs History Limit	Limit CronJob failed history
Manual Deletion	Delete old Jobs manually

CLEANUP EXAMPLE

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-example
spec:
  schedule: "0 1 * * *"
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      ttlSecondsAfterFinished: 86400
```

7. JOBS & CRONJOBS CHEAT SHEET

USEFUL COMMANDS

```
# Create a Job
kubectl apply -f job.yaml
# List Jobs
kubectl get jobs
# Describe a Job
kubectl describe job <job-name>
# View Job Pods
kubectl get pods --selector=job-name=<job-name>
# Delete a Job
kubectl delete job <job-name>
```

CRONJOB COMMANDS

```
# Create a CronJob
kubectl apply -f cronjob.yaml
# List CronJobs
kubectl get cronjobs
# Describe a CronJob
kubectl describe cronjob <cronjob-name>
# View Jobs created by CronJob
kubectl get jobs --selector=cronjob-name=<cronjob-name>
```

JOB STATUS

Active	Job is running
Complete	Job completed successfully
Failed	Job failed after retries
BackoffLimitExceeded	Job failed after max retries
DeadlineExceeded	Job exceeded active deadline

KEY TAKEAWAY

Jobs run tasks to completion.
CronJobs automate recurring tasks.
Use retries, cleanup, and monitoring for reliable batch processing.

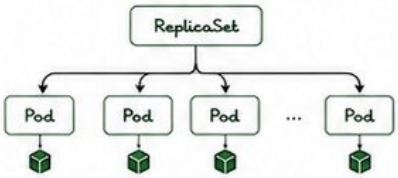
AUTOMATE. RELIABILITY. CLEANUP.
BATCH OPERATIONS MADE SIMPLE.
MASTER JOBS. MASTER THE CKA.

16. REPLICASETS

ReplicaSets ensure that a specified number of Pod replicas are running at any given time.
They are the foundation for Deployments and other higher-level controllers.

1. WHAT IS A REPLICASET?

A ReplicaSet maintains a stable set of replica Pods running at all times.



KEY FEATURES

- Maintains the desired number of replicas.
- Replaces Pods that fail or are deleted.
- Ensures Pods match a label selector.
- Does not perform updates or rollbacks.
- Used directly only in simple or custom scenarios.

ReplicaSets match a label selector and keep that number of Pods running.

2. CONTROLLER BEHAVIOR

ReplicaSets constantly watch the cluster and take action to match the desired state.

CONTROLLER LOOP

1. Watch for changes in Pods
2. Count current matching Pods
3. Compare with desired replicas
4. Create or delete Pods to match
5. Repeat continuously

WHAT TRIGGERS RECONCILIATION?

- Pod deletion or failure
- Scaling changes
- Label selector changes
- Node failures or evictions

IMPORTANT

ReplicaSets do not update Pod templates.
Changing the template creates a new ReplicaSet.

3. SCALING REPLICASETS

Scale up or down by changing the replica count.

SCALE METHODS

- `kubectl scale replicaset <name> --replicas=<n>`
- Edit the YAML file and apply
- Used by Deployments automatically

SCALE EXAMPLE

```
# Scale a ReplicaSet to 5 replicas
kubectl scale replicaset web-rs --replicas=5

# Check current replicas
kubectl get rs web-rs

# Output
NAME      DESIRED   CURRENT   READY   AGE
web-rs    5         5         5       2m
```

Scaling changes the desired count. The controller ensures the cluster matches it.

4. OWNERSHIP & RELATIONSHIPS

ReplicaSets can be owned by higher-level controllers.

OWNER	PURPOSE
Deployment	Manages updates and rollbacks
StatefulSet	Manages stateful Pod replicas
Custom Controller	Custom automation
None	Independent ReplicaSet

OWNER BENEFITS

- Automatic cleanup when owner is deleted
- Controlled updates and rollbacks
- Clear ownership hierarchy

Avoid orphaned ReplicaSets without owners. They will not be cleaned up automatically.

5. REPLICASET YAML EXAMPLE

BASIC REPLICASET

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
  labels:
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80
```

Selector labels must match template labels. This is how the ReplicaSet knows which Pods to manage.

6. OBSERVABILITY & STATUS

CHECK STATUS

- `kubectl get rs`
- `kubectl describe rs <name>`
- `kubectl get pods -l <label-selector>`
- `kubectl logs <pod-name>`

STATUS FIELDS

- DESIRED - Desired number of replicas
- CURRENT - Current number of Pods
- READY - Ready Pods
- AGE - Age of the ReplicaSet

COMMON EVENTS

- SuccessfulCreate - Pod created
- SuccessfulDelete - Pod deleted
- FailedCreate - Failed to create Pod
- FailedDelete - Failed to delete Pod

7. REPLICASET CHEAT SHEET

USEFUL COMMANDS

```
# Create ReplicaSet
kubectl apply -f replicaset.yaml
# Get all ReplicaSets
kubectl get rs
# Describe ReplicaSet
kubectl describe rs <name>
# Scale ReplicaSet
kubectl scale rs <name> --replicas=<n>
# Delete ReplicaSet
kubectl delete rs <name>
```

EXAMPLE OUTPUT

```
kubectl get rs
NAME      DESIRED   CURRENT   READY   AGE
web-rs    3         3         3       2m

kubectl get pods -l app=web
NAME                 READY   STATUS    RESTARTS   AGE
web-rs-5f6d7c9b6-abc12   1/1     Running   0          2m
web-rs-5f6d7c9b6-def34   1/1     Running   0          2m
web-rs-5f6d7c9b6-ghi56   1/1     Running   0          2m
```

BEST PRACTICES

- [x] Use label selectors carefully.
- [x] Avoid modifying Pods directly.
- [x] Use Deployments for updates and rollbacks.
- [x] Monitor ReplicaSets and events.
- [x] Keep configurations in version control.
- [x] Clean up unused ReplicaSets.

KEY TAKEAWAY

ReplicaSets keep your Pods running.
They replace failures and maintain stability.
Deployments build on ReplicaSets for advanced features.

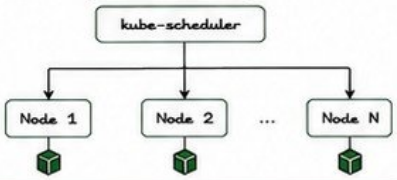
STABLE REPLICAS. RELIABLE APPS.
MASTER REPLICASETS.
MASTER THE CKA.

17. SCHEDULING

Scheduling determines which Node a Pod runs on. You control placement using node selectors, node affinity, pod affinity/anti-affinity, taints, and tolerations.

1. SCHEDULING OVERVIEW

The kube-scheduler watches for unscheduled Pods and assigns them to the best Node based on filters and scoring.



KEY POINTS

- Pods are scheduled, not containers.
- Scheduler considers requests, limits, policies, constraints, and cluster state.
- You can influence placement with rules.
- If no Node fits, Pod remains Pending.

Good scheduling improves performance, reliability, and cost efficiency.

2. NODE SELECTOR

Assign Pods to Nodes with specific labels. Simple equality-based constraint.

HOW IT WORKS

- You label Nodes.
- You specify key: value in the Pod.
- Pod is scheduled only on matching Nodes.

NODE LABEL EXAMPLE

```
kubectl label node node-1 disktype=ssd
```

POD YAML EXAMPLE

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  nodeSelector:
    disktype: ssd
```

Use nodeSelector for simple, hard requirements.

3. NODE AFFINITY

Advanced way to constrain Pods to Nodes using rules and expressions.

TYPES

requiredDuringSchedulingIgnoredDuringExecution
Hard requirement (must match).

preferredDuringSchedulingIgnoredDuringExecution
Soft preference (best effort).

POD YAML EXAMPLE (REQUIRED)

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
                operator: In
                values: ["ssd", "nvme"]
```

Use node affinity for flexible and advanced Node placement.

4. POD AFFINITY

Place Pods close to other Pods that match specific labels.

USE CASES

- Co-locate frontend with backend
- Reduce network latency
- Improve performance

POD YAML EXAMPLE

```
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        labelSelector:
          matchLabels:
            app: backend
        topologyKey: kubernetes.io/hostname
```

Pod affinity ensures Pods run on the same Node (or same topology domain).

5. POD ANTI-AFFINITY

Spread Pods away from other Pods that match specific labels.

USE CASES

- High availability
- Avoid single Node failure
- Distribute workload

POD YAML EXAMPLE

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        labelSelector:
          matchLabels:
            app: backend
        topologyKey: kubernetes.io/hostname
```

Anti-affinity protects against Pod failures on the same Node.

6. TAINTS

Taints repel Pods that do not have matching tolerations.

TAINT FORMAT

key=value:effect

- effect= NoSchedule | PreferNoSchedule | NoExecute

TAINT EXAMPLE

```
kubectl taint nodes node-1 dedicated=gpu:NoSchedule
```

EFFECTS

NoSchedule	Pods without toleration will not schedule.
PreferNoSchedule	Scheduler tries to avoid placing Pods.
NoExecute	Evicts existing Pods without toleration.

Use taints to reserve Nodes for specific workloads (e.g., GPU, critical, system).

7. TOLERATIONS

Tolerations allow Pods to be scheduled on tainted Nodes.

TOLERATION YAML EXAMPLE

```
spec:
  tolerations:
    - key: dedicated
      operator: Equal
      value: gpu
      effect: NoSchedule
```

Only Pods with matching tolerations can run on tainted Nodes.

8. COMBINING RULES

You can combine multiple rules for precise Pod placement.

COMMON COMBINATIONS

- ☑ nodeSelector + tolerations
- ☑ nodeAffinity + podAntiAffinity
- ☑ taints + tolerations + affinity
- ☑ preferred + required rules

Combine rules carefully to avoid Pods stuck in Pending state.

9. TROUBLESHOOTING SCHEDULING

Tools and steps to debug scheduling issues.

USEFUL COMMANDS

```
kubectl get pods -o wide
kubectl describe pod <pod-name>
kubectl describe node <node-name>
kubectl get events --sort-by=.metadata.creationTimestamp
```

CHECK FOR

- Taints without tolerations
- Node selectors not matching
- Insufficient resources
- Affinity rules too strict

Events section in Pod describe gives the real reason why it is Pending.

10. SCHEDULING SUMMARY CHEAT SHEET

FEATURE	PURPOSE	TYPE	EXAMPLE
Node Selector	Place Pod on Nodes with specific labels	Hard	nodeSelector: disktype: ssd
Node Affinity	Constrain Pod to Nodes using rules	Hard / Soft	requiredDuringSchedulingIgnoredDuringExecution
Pod Affinity	Place Pod near matching Pods	Hard / Soft	podAffinity with labelSelector
Pod Anti-Affinity	Keep Pod away from matching Pods	Hard / Soft	podAntiAffinity with labelSelector
Taints	Repel Pods without tolerations	NoSchedule / PreferNoSchedule / NoExecute	dedicated=gpu:NoSchedule
Tolerations	Allow Pod to run on tainted Nodes	Matching rule	tolerations for key=value:effect

BEST PRACTICES

- ☑ Label your Nodes consistently.
- ☑ Use Selector for simple needs.
- ☑ Use affinity/anti-affinity for topology control.
- ☑ Use taints to protect critical Nodes.
- ☑ Always test rules before production.
- ☑ Monitor Pending Pods and events.

KEY TAKEAWAY

Scheduling places Pods on the right Nodes. Use selectors, affinity, taints, and tolerations to control placement, improve reliability, and optimize resource usage.

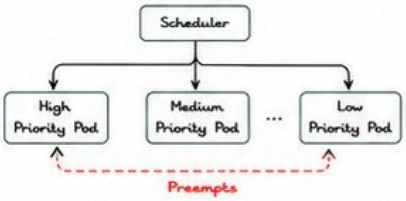
RIGHT POD. RIGHT NODE. RIGHT RULES.
MASTER SCHEDULING.
MASTER THE CKA.

18. PRIORITY & PREEMPTION

Priority and preemption allow important workloads to take resources from lower priority Pods when the cluster is under pressure.

1. CONCEPT OVERVIEW

Priority tells the scheduler which Pods are more important. Preemption allows high priority Pods to take resources from lower priority Pods.



KEY POINTS

- Higher priority Pods are scheduled first.
- If no resources, scheduler can preempt lower priority Pods.
- Preempted Pods are evicted to free resources.
- Priority is defined using PriorityClass.
- System critical Pods usually have high priority.

Use priority and preemption carefully to avoid impacting user workloads.

2. PRIORITYCLASS

PriorityClass defines different priority levels for Pods.

PRIORITY CLASS FIELDS

metadata.name	Name of the PriorityClass
value	Integer value (higher = higher priority)
globalDefault	Default for Pods without priorityClass
description	Human readable description
preemptionPolicy	PreemptLowerPriority (default) or Never

EXAMPLE PRIORITY CLASSES

NAME	VALUE	DESCRIPTION
system-cluster-critical	2000000000	Critical system Pods
system-node-critical	1900000000	Node level critical Pods
high-priority	100000	High priority apps
medium-priority	1000	Medium priority apps
low-priority	100	Low priority workloads

EXAMPLE PRIORITYCLASS YAML

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 100000
preemptionPolicy: PreemptLowerPriority
description: "High priority workload"
```

Higher value means higher priority.

3. EVICTIONS

Evictions happen when a high priority Pod preempts a lower priority Pod.

EVICTION PROCESS

1. High priority Pod is created.
2. No enough resources available.
3. Scheduler finds lower priority Pods.
4. Lower priority Pods are marked for eviction.
5. Resources are freed.
6. High priority Pod is scheduled.

WHAT HAPPENS TO EVICTED PODS?

- Pod status becomes: Failed
- Reason: Preempted
- They do not restart automatically.
- Controllers (Deployment, StatefulSet) will create new Pods.

CHECK EVICTIONS

```
kubectl describe pod <evicted-pod-name>
```

Look for events like:

Warning Preempted The node was low on resources.

Evictions help critical workloads run during resource pressure.

4. SCHEDULING DECISIONS

Scheduler uses priority during scheduling.

DECISION FLOW

1. Filter Nodes based on resources, constraints.
2. Score Nodes to find the best fit.
3. If no Node fits, check preemption.
4. Select lower priority Pods to evict.
5. Preempt and schedule the high priority Pod.

PREEMPTION CHECKS

- Only lower priority Pods can be evicted.
- PodDisruptionBudget (PDB) is respected.
- System critical Pods are not preempted.
- Scheduler chooses minimal Pods to evict.

Scheduler tries to minimize impact while making room for important workloads.

5. PREEMPTION POLICIES

Control whether a Pod can preempt other Pods.

POLICY	DESCRIPTION
PreemptLowerPriority (Default)	Pod can preempt lower priority Pods to schedule.
Never	Pod will never preempt other Pods. It will remain Pending if no resources.

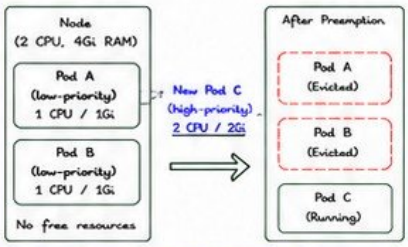
EXAMPLE USAGE IN POD

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: important-app
spec:
  priorityClassName: high-priority
  containers:
    - name: app
      image: nginx
```

Use 'Never' for workloads that should wait instead of evicting other workloads.

6. PRACTICAL EXAMPLE

High priority Pod preempts lower priority.



CHECK PENDING & PREEMPTION

```
# See pending Pods
kubectl get pods
# See events on a Node
kubectl get events --sort-by=.lastTimestamp
```

Events will show which Pods were preempted and why.

7. PRIORITY & PREEMPTION CHEAT SHEET

USEFUL COMMANDS

```
# List priority classes
kubectl get priorityclass
# Describe a priority class
kubectl describe priorityclass <name>
# Get Pods with priority
kubectl get pods -o custom-columns=
  NAME:.metadata.name,PRIORITY:.spec.priorityClassName
# Describe an evicted Pod
kubectl describe pod <evicted-pod-name>
# Get events on a Node
kubectl get events --field-selector involvedObject.kind=Node
```

EXAMPLE PRIORITYCLASSES

NAME	VALUE	USE CASE
system-cluster-critical	2000000000	Control plane components
system-node-critical	1900000000	Node critical components
high-priority	100000	Important business apps
medium-priority	1000	Normal workloads
low-priority	100	Batch / background jobs

BEST PRACTICES

- [x] Reserve high priorities for critical workloads.
- [x] Test preemption in a staging environment.
- [x] Use PDBs to protect important Pods.
- [x] Avoid giving high priority to everything.
- [x] Monitor evictions and pending Pods.

KEY TAKEAWAY

Priority decides importance. Preemption makes space.
Use PriorityClass to protect what matters most.
Design priorities carefully for a stable and reliable cluster.

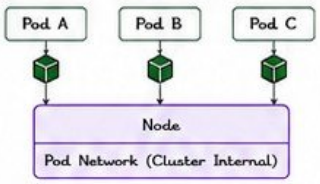
RIGHT PRIORITY. RIGHT PREEMPTION.
PROTECT CRITICAL WORKLOADS.
MASTER THE CLUSTER.

19. CLUSTER NETWORKING REVIEW

Kubernetes networking enables communication between Pods, Services, and external systems through a flat, IP-based network using CNI plugins and DNS.

1. POD NETWORKING

Each Pod gets its own IP address on the Pod network. Pods on the same node share the node network.

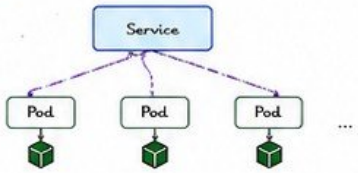


Key Points

- Pods have unique IP addresses.
- Pods can communicate directly without NAT.
- Network is flat across the cluster.

2. SERVICE NETWORKING

Services provide a stable virtual IP and DNS name to access Pods.



Key Points

- Load balances traffic to matching Pods.
- Provides stable endpoints for clients.
- Utilizes ClusterIP, NodePort, LoadBalancer, etc.

3. CNI (CONTAINER NETWORK INTERFACE)

CNI plugins configure networking for Pods according to Kubernetes network model.

Common CNI Plugins

- Flannel – Overlay network using VXLAN
- Calico – Network policies and routing
- Cilium – eBPF-based networking and security
- Weave Net – Overlay networking

Key Responsibilities

- Assign IP addresses to Pods
- Set up routing and network policies
- Provide connectivity across nodes

4. DNS IN KUBERNETES

Kubernetes uses CoreDNS as the cluster DNS. It provides DNS for Services and Pods.

Key Features

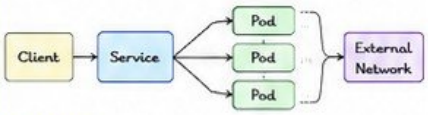
- Service DNS: <service-name>.<namespace>.svc.cluster.local
- Pod DNS: <pod-ip>.pod.cluster.local
- Automatic service discovery
- Configurable via ConfigMaps

Example

```
my-service.my-namespace.svc.cluster.local
```

5. NETWORK FLOW

Understanding how traffic flows in Kubernetes helps with troubleshooting and design.



Flow Steps

1. Client sends request to Service (ClusterIP or External IP)
2. Service routes to one of the healthy Pods
3. Pod processes request and sends response
4. Response returns through Service to Client

KEY NETWORKING CONCEPTS

- **Pod-to-Pod**: Direct communication within cluster
- **Pod-to-Service**: Access via Service IP or DNS
- **Service-to-External**: NodePort or LoadBalancer
- **Ingress**: HTTP(S) routing to Services
- **Network Policies**: Control traffic between Pods

POD NETWORKING DETAILS

Feature	Description
IP Address	Unique per Pod
Network	Flat, no NAT between Pods
Isolation	Enforced by network policies (optional)
Communication	Direct between Pods on the same or different nodes

Common Pod IP Range
10.244.0.0/16 (example, depends on CNI)

SERVICE TYPES (NETWORKING)

Type	Description
ClusterIP	Internal cluster access (default)
NodePort	Access via each Node's IP:port
LoadBalancer	Cloud provider load balancer
ExternalName	Maps to external DNS name

Service IP Range (Example)
10.96.0.0/12

DNS EXAMPLE RECORDS

Type	Example
cluster.local	Cluster domain
Service	my-svc.default.svc.cluster.local
Pod	10-244-1-23.default.pod.cluster.local
SRV Record	_tcp.my-svc.default.svc.cluster.local

CNI PLUGIN COMPARISON (EXAMPLES)

CNI	Overlay	Network Policy	Routing	Notes
Flannel	Yes	No	Simple	Easy to use
Calico	No	Yes	BGP	Advanced security
Cilium	Yes	Yes	eBPF	High performance
Weave Net	Yes	Yes	Overlay	Simple setup

KEY NETWORKING COMMANDS

- kubectl get pods -o wide
- kubectl get svc
- kubectl describe svc <name>
- kubectl get endpoints <svc>
- kubectl exec <pod> -- nslookup <svc>
- kubectl get networkpolicy

TROUBLESHOOTING TIPS

- Check Pod IP, Service, and Endpoints
- Verify DNS resolution
- Check CNI plugin status
- Review Network Policies
- Use kubectl describe for events

KEY TAKEAWAY

Networking in Kubernetes connects Pods and Services, enables discovery with DNS, and is powered by CNI plugins for reachability and security.

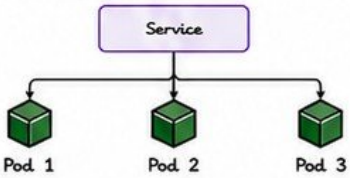
MASTER NETWORKING. MASTER CLUSTER.
RIGHT CONNECTIONS. STRONGER CLUSTERS.

20. SERVICES

Kubernetes Services provide stable, reliable network access to a set of Pods. They enable service discovery, load balancing, and controlled access to applications.

1. WHAT IS A SERVICE?

A Service is a stable endpoint that exposes a group of Pods and provides consistent network access to them.



KEY POINTS

- Provides stable IP and DNS name
- Load balances traffic across healthy Pods
- Decouples clients from Pod lifecycle
- Enables controlled access to applications

Services are the foundation for networking in Kubernetes.

2. SERVICE TYPES OVERVIEW

Kubernetes provides different Service types for different access requirements.

TYPE	PURPOSE
ClusterIP	Internal access within the cluster
NodePort	Access via Node IP and static port
LoadBalancer	External access via cloud load balancer
ExternalName	Map to external DNS name
Headless	No cluster IP, direct Pod discovery

Choose the right Service type based on your access and architecture requirements.

3. CLUSTERIP

Default Service type. Accessible only within the cluster.

SERVICE YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
```

Features:
- ✓ Internal access
- ✓ Stable virtual IP
- ✓ DNS name
- ✓ Load balancing

4. NODEPORT

Exposes the Service on a static port on every Node IP address.

SERVICE YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080
```

Features:
- ✓ External access
- ✓ Static port
- ✓ Accessible via Node IP
- ✓ Simple setup

5. LOADBALANCER

Provision a cloud load balancer for external access to your application.

SERVICE YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-lb
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
```

Features:
- ✓ External access
- ✓ Cloud load balancer
- ✓ Public IP
- ✓ Production ready
- ✓ Auto provisioning

6. EXTERNALNAME

Maps the Service to an external DNS name.

SERVICE YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: external-db
spec:
  type: ExternalName
  externalName:
    db.example.com
```

Features:
- ✓ Points to DNS
- ✓ No proxying
- ✓ External service
- ✓ Simple alias

7. HEADLESS SERVICE

Service without a cluster IP. Returns actual Pod IPs for direct communication.

SERVICE YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: wb-headless
spec:
  clusterIP: None
  selector:
    app: db
  ports:
  - port: 5432
```

Features:
- ✓ No cluster IP
- ✓ Direct Pod access
- ✓ Stateful applications
- ✓ Custom discovery
- ✓ DNS returns Pod IPs

8. SERVICE DNS

Kubernetes automatically creates DNS records for every Service.

DNS FORMATS

- <service-name>
- <service-name>.<namespace>
- <service-name>.<namespace>.svc
- <service-name>.<namespace>.svc.cluster.local

EXAMPLE

- web-service
- web-service.default
- web-service.default.svc
- web-service.default.svc.cluster.local

Use Service DNS for stable application communication instead of Pod IPs.

9. BEST PRACTICES

- ☑ Use ClusterIP for internal services
- ☑ Use LoadBalancer for production web apps
- ☑ Avoid NodePort in production when possible
- ☑ Use Headless for Stateful applications
- ☑ Use labels and selectors consistently
- ☑ Monitor Service endpoints
- ☑ Use NetworkPolicies for additional security

KEY TAKEAWAY

Services provide stable, reliable, and scalable network access to applications in Kubernetes. Choose the right type for your use case.

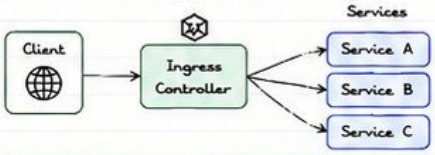
STABLE ACCESS. RELIABLE NETWORKING.
MASTER KUBERNETES.
MASTER THE CKA.

21. INGRESS

Ingress exposes HTTP and HTTPS routes from outside the cluster to Services.
It provides path-based and host-based routing, TLS termination, and traffic management.

1. INGRESS OVERVIEW

Ingress sits at the edge of the cluster and routes external traffic to Services.



KEY POINTS

- Layer 7 (HTTP/HTTPS) routing
- Host and path based rules
- TLS termination support
- Uses Ingress Controller
- Routes traffic to Services, not Pods

YAML EXAMPLE (SIMPLE)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-ingress
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

Ingress manages external access to Services using rules.

2. INGRESS RULES

Rules define how requests are matched and which Service receives the traffic.

RULE STRUCTURE

- Host (optional): domain name
- Path: URL path
- Backend: Service and port

EXAMPLE RULES

Host	Path	Service	Port
app.example.com	/	web	80
app.example.com	/api	api	8080
admin.example.com	/	admin	80

MULTIPLE HOSTS EXAMPLE

```yaml
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web
            port:
              number: 80
  - host: admin.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: admin
            port:
              number: 80
```

Rules match requests and send them to the correct Service.

3. TLS (HTTPS)

Ingress can terminate TLS using certificates stored in Kubernetes Secrets.

TLS FLOW

1. Client connects via HTTPS (443)
2. Ingress Controller terminates TLS
3. Traffic is routed to Service (HTTP)
4. Service communicates internally

YAML EXAMPLE (TLS)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls-secret
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web
            port:
              number: 80
```

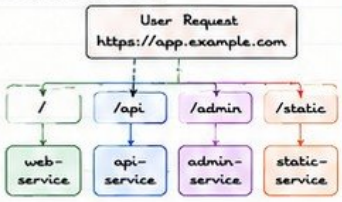
Use TLS to secure traffic with HTTPS.

4. PATH ROUTING

Path routing allows different parts of a URL to be routed to different Services.

EXAMPLE: app.example.com

Path	Service
/	web-service
/api	api-service
/admin	admin-service
/static	static-service



KEY POINTS

- Requests are matched by longest path prefix
- More specific paths take precedence
- pathType: Prefix (default)
- Exact match available with pathType: Exact

Path routing enables multiple applications on the same domain.

5. INGRESS CONTROLLERS

Ingress Controllers watch Ingress resources and implement the routing.

POPULAR CONTROLLERS

- NGINX Ingress Controller
- Traefik
- HAProxy Ingress
- Istio Ingress Gateway
- AWS ALB Ingress Controller
- GCP Ingress (GCE)
- Azure Application Gateway Ingress Controller

INGRESS CONTROLLER ROLE

- ☑ Watches Ingress resources
- ☑ Validates configuration
- ☑ Programs the underlying proxy/load balancer
- ☑ Handles TLS termination
- ☑ Performs routing and load balancing
- ☑ Provides metrics and logs

DEPLOYMENT EXAMPLE (NGINX)

```
# Install NGINX Ingress Controller
kubectl apply -f https://raw.githuburastectont.com/kubernetes/ingress-nginx/
controller-v1.10.0/deploy/static/provider/cloud/deploy.yaml
# Verify
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

Ingress Controller is required for Ingress to work.

6. COMMON MISTAKES

- ✗ Forgetting to install an Ingress Controller
- ✗ Using wrong pathType (Prefix vs Exact)
- ✗ Missing backend Service or wrong port
- ✗ TLS secret not created or wrong secretName
- ✗ Overlapping host rules causing conflicts
- ✗ Using NodePort when Ingress is required
- ✗ Not considering redirect from HTTP to HTTPS
- ✗ Ignoring default backend for unmatched routes

BEST PRACTICES

- ☑ Plan host and path structure
- ☑ Use TLS for all external traffic
- ☑ Keep rules simple and specific
- ☑ Use meaningful names
- ☑ Monitor Ingress Controller logs
- ☑ Test routing after changes
- ☑ Use annotations wisely

Avoiding common mistakes ensures reliable access and security.

7. INGRESS ANNOTATIONS (EXAMPLES)

Annotations customize controller behavior.

ANNOTATION	DESCRIPTION	EXAMPLE VALUE
nginx.ingress.kubernetes.io/rewrite-target	Rewrite URL	/
nginx.ingress.kubernetes.io/ssl-redirect	Redirect HTTP to HTTPS	"true"
nginx.ingress.kubernetes.io/force-ssl-redirect	Force HTTPS	"true"
nginx.ingress.kubernetes.io/proxy-body-size	Max upload size	"10m"
nginx.ingress.kubernetes.io/timeout-read	Read timeout	"60"
nginx.ingress.kubernetes.io/limit-rps	Rate limiting	"10"

KEY TAKEAWAY

Ingress provides smart, secure, and scalable access to your applications. Good rules, TLS, and a reliable controller are the key to success.

FINAL PRINCIPLE

Route traffic at Layer 7 with precision.
Secure it with TLS. Monitor it always.

22. NETWORK POLICIES

Network Policies control traffic flow at Layer 3/4 between Pods.
They provide micro-segmentation and improve security inside the cluster.

1. OVERVIEW

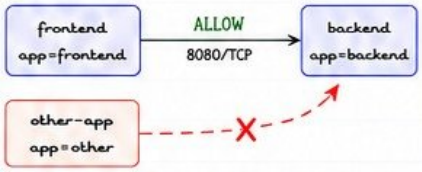
Network Policies are namespace-scoped.
They select Pods using labels and define allowed traffic.

KEY POINTS

- Applied to Pods, not Nodes.
- Additive: multiple policies = union of allows.
- If no policy selects a Pod → all traffic allowed.
- If at least one policy selects a Pod → default deny.

2. INGRESS (INCOMING TRAFFIC)

Controls who can send traffic to selected Pods.
Example: Allow only frontend to talk to backend on port 8080.



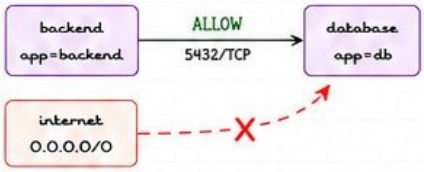
YAML EXAMPLE (INGRESS)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

Only Pods with app=frontend can access backend on port 8080.

3. EGRESS (OUTGOING TRAFFIC)

Controls where selected Pods can send traffic.
Example: Allow backend to access database only.



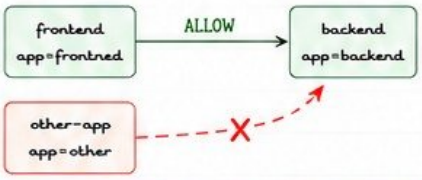
YAML EXAMPLE (EGRESS)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-egress-db-only
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Egress
  egress:
   - to:
     - podSelector:
         matchLabels:
           app: db
     ports:
       - protocol: TCP
         port: 5432
```

Backend Pods can only reach the database on port 5432.

4. ISOLATION

Policies provide isolation between applications.
Example: Frontend cannot talk to other-app.



ISOLATION BENEFITS

- Prevent lateral movement inside the cluster.
- Enforce least-privilege communication.
- Protect sensitive workloads.
- Reduce blast radius during compromise.

5. DEFAULT DENY

If any Network Policy selects a Pod, traffic is denied by default unless explicitly allowed.
Example: Deny all ingress to backend.

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-backend-ingress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress: []  # No rules = deny all
```

DEFAULT DENY RULE

At least one policy selects the Pod
+ No matching rule
= Traffic denied

6. TROUBLESHOOTING

Common steps to debug Network Policies.

1. Verify if a policy selects the Pod.
2. Check policyTypes (Ingress / Egress / Both).
3. Review podSelector and namespace.
4. Validate labels on Pods.
5. Ensure rules match traffic (ports, protocols, IPs).
6. Check CNI plugin and policy support.
7. Test with temporary allow rule.

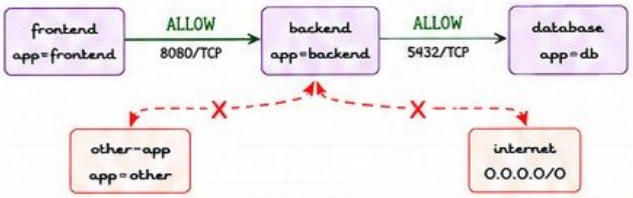
USEFUL COMMANDS

```
# List network policies
kubectl get networkpolicies -A
# Describe a policy
kubectl describe networkpolicy <name> -n <ns>
# Check labels on Pod
kubectl get pod <pod> -n <ns> --show-labels
# Test connectivity from a Pod
kubectl exec -it <pod> -- curl -v <ip:port>
```

Use logs from CNI plugin for deeper debugging.

7. EXAMPLE: COMPLETE MICRO-SEGMENTATION

Frontend → Backend → Database only. Everything else blocked.



8. BEST PRACTICES

- ☑ Start with default deny.
- ☑ Allow only required traffic.
- ☑ Use labels consistently.
- ☑ Keep policies simple and specific.
- ☑ Review policies regularly.
- ☑ Document rules and intent.

REMEMBER

Network Policies protect your Pods, not the Nodes. They work with your CNI plugin to enforce rules.

KEY TAKEAWAY

Network Policies enforce secure, controlled communication between Pods. Use default deny and allow only what is needed for a secure and resilient cluster.

VERIQTA

FINAL PRINCIPLE

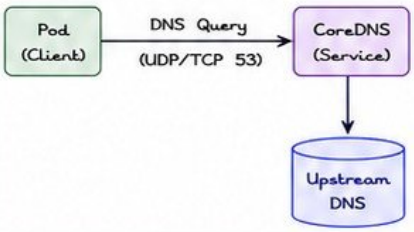
Secure traffic. Limit access.
Isolate workloads. Protect your cluster.

23. DNS TROUBLESHOOTING

DNS is the foundation of Service discovery in Kubernetes. When DNS fails, applications cannot find Services, leading to timeouts and outages.

1. COREDNS OVERVIEW

CoreDNS is the default DNS server in Kubernetes. It runs as Pods in the kube-system namespace.



KEY POINTS

- CoreDNS resolves names for Services, Pods, and External domains.
- It uses the Kubernetes API as its data source.
- It can forward queries to upstream DNS servers.

2. DNS RESOLUTION FLOW

1. A Pod makes a DNS query for service-name.namespace.svc.cluster.local
2. Query is sent to CoreDNS (ClusterIP Service: kube-dns 10.96.0.10)
3. CoreDNS checks cache
4. If not found, CoreDNS queries Kubernetes API for the Record
5. CoreDNS returns the IP to the Pod
6. Pod connects to the Service IP

DNS NAME FORMAT

<service>.<namespace>.svc.cluster.local

Examples:

- web.default.svc.cluster.local
- api.prod.svc.cluster.local
- my-service.ns.svc.cluster.local

Short names only work within the same namespace.

3. SERVICE DISCOVERY

Kubernetes creates DNS records automatically for each Service.

RECORD TYPE	EXAMPLE
A Record	web.default.svc.cluster.local -> 10.96.45.23
SRV Record	_http._tcp.web.default.svc.cluster.local -> 10 96 45 23:80
Pod A Record	10-244-1-5.default.pod.cluster.local -> 10.244.1.5
NXDOMAIN	Non-existent names return NXDOMAIN

KEY POINTS

- Services get A and SRV records.
- Headless Services return Pod IPs.
- DNS is namespace-aware.
- External names (e.g., google.com) are forwarded upstream.

Service discovery relies entirely on CoreDNS.

4. COMMON DNS ISSUES

ISSUE	SYMPTOMS
CoreDNS Pod Down	Pods cannot resolve any names. nslookup fails for all domains.
DNS Service Unreachable	Timeouts when querying CoreDNS. check kube-dns Service IP.
Incorrect Config	Some domains fail to resolve. Errors in Corefile.
Upstream DNS Failure	External domains fail. Internal Service resolution works.
Network Policies Block DNS	Pods cannot reach CoreDNS. UDP/TCP 53 blocked.
High Latency	Slow DNS responses. Applications experience delays.

5. DEBUGGING WORKFLOW

1. Check CoreDNS Pods
   `kubectl get pods -n kube-system -l k8s-app=kube-dns`
2. Check CoreDNS Logs
   `kubectl logs -n kube-system deployment/coredns`
3. Test DNS from a Pod
   `kubectl run dns-test -it --rm --image=busybox -- sh`
   `nslookup kubernetes.default.svc.cluster.local`
4. Check CoreDNS Service
   `kubectl get svc -n kube-system kube-dns`
5. Check CoreDNS Config
   `kubectl -n kube-system get configmap coredns -o yaml`
6. Check Network Policies
   `kubectl get networkpolicies -A`

Follow the workflow from inside the cluster out.

6. USEFUL COMMANDS

Test DNS resolution from Pod
```
nslookup <service>.<namespace>.svc.cluster.local
dig <service>.<namespace>.svc.cluster.local
```
Check CoreDNS Pods
```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```
View CoreDNS Logs
```
kubectl logs -n kube-system deployment/coredns -f
```
Exec into CoreDNS Pod
```
kubectl exec -n kube-system -it <coredns-pod> -- sh
```
Check CoreDNS Config
```
kubectl -n kube-system get configmap coredns -o yaml
```
Check Cluster DNS IP
```
kubectl get svc -n kube-system kube-dns -o wide
```
Test External DNS
```
nslookup google.com 8.8.8.8
```
Describe DNS Service
```
kubectl describe svc kube-dns -n kube-system
```

7. COREDNS COREFILE BASICS

The Corefile defines how CoreDNS handles queries.

It uses plugins in a chain.

Example:

```
.:53 {
  errors
  health
  kubernetes cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
  }
  prometheus :9153
  forward . /etc/resolv.conf
  cache 30
  loop
  reload
  loadbalance
}
```

PLUGIN	PURPOEE
errors	Logs errors
health	Health check endpoint
kubernetes	Resolves Kubernetes names
prometheus	Metrics endpoint
forward	Forwards queries to upstream DNS
cache	Caches responses
loop	Prevents forwarding loops
reload	Auto-reloads Corefile
loadbalance	Load balances upstream servers

BEST PRACTICES

- ☑ Ensure CoreDNS Pods are highly available.
- ☑ Monitor CoreDNS metrics and logs.
- ☑ Keep Corefile simple and correct.
- ☑ Allow UDP/TCP 53 in Network Policies.
- ☑ Test DNS from inside the cluster regularly.

KEY TAKEAWAY

- CoreDNS is critical for Service discovery.
- DNS failures break application communication.
- Use a structured workflow to troubleshoot.
- Monitor, secure, and test DNS continuously.

VERIQTA

FINAL PRINCIPLE

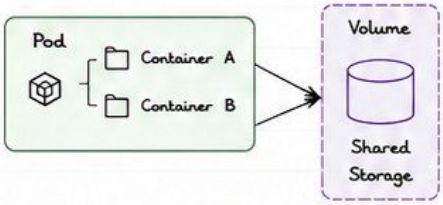
If DNS fails, everything else fails. Keep it fast, reliable, and observable.

24. STORAGE

Kubernetes Storage allows Pods to persist and share data. Understanding Volumes is essential for building reliable systems.

1. STORAGE OVERVIEW

Volumes provide storage that is accessible to containers in a Pod.

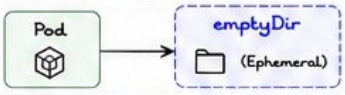


KEY POINTS

- Volumes outlive container restarts.
- Volumes are mounted into containers.
- Data in a Volume persists as configured.
- Some Volumes are ephemeral, others persistent.

2. emptyDir VOLUME

Ephemeral storage that exists as long as the Pod exists.



USE CASES

- Scratch space
- Temporary data
- Caching
- Logs processing

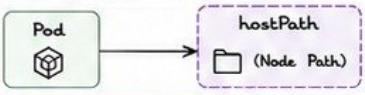
EXAMPLE

```yaml
volumes:
 - name: temp-storage
   emptyDir: {}
containers:
 - name: app
   image: nginx
   volumeMounts:
    - name: temp-storage
      mountPath: /data
```

Data is deleted when the Pod is deleted.

3. hostPath VOLUME

Mounts a file or directory from the Node's filesystem into the Pod.



USE CASES

- Access node files
- Logs collection
- Local PV development
- System configuration

EXAMPLE

```yaml
volumes:
 - name: host-storage
   hostPath:
     path: /var/log
     type: Directory
containers:
 - name: app
   image: nginx
   volumeMounts:
    - name: host-storage
      mountPath: /logs
```

Use with caution. Tightly couples Pod to Node.

4. CONFIGMAP VOLUME

Mount ConfigMap data as files in a Volume.

USE CASES

- Application configuration
- Environment files
- Feature flags

EXAMPLE

```yaml
volumes:
 - name: config-volume
   configMap:
     name: app-config
containers:
 - name: app
   image: nginx
   volumeMounts:
    - name: config-volume
      mountPath: /etc/config
      readOnly: true
```

Files are updated automatically when ConfigMap changes.

5. SECRET VOLUME

Mount Secret data as files in a Volume.

USE CASES

- Credentials
- API keys
- Certificates
- Sensitive data

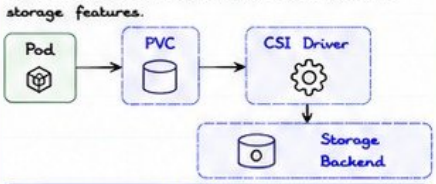
EXAMPLE

```yaml
volumes:
 - name: secret-volume
   secret:
     secretName: db-secret
containers:
 - name: app
   image: nginx
   volumeMounts:
    - name: secret-volume
      mountPath: /etc/secret
      readOnly: true
```

Data is base64 encoded in etcd. Always use RBAC to protect Secrets.

6. CSI (CONTAINER STORAGE INTERFACE)

Standard interface for storage drivers. Enables dynamic provisioning and advanced storage features.



USE CASES

- Persistent storage
- Snapshots
- Volume expansion
- Multi-cloud storage

EXAMPLE

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: fast-ssd
  resources:
    - requests:
        storage: 10Gi
```

CSI enables powerful storage capabilities.

7. VOLUME COMPARISON

VOLUME TYPE	LIFETIME	DATA LOCATION	PERSISTENT	SHARED	TYPICAL USE CASE
emptyDir	Pod Lifetime	Node (Ephemeral)	No	Yes (within Pod)	Temporary data, caching
hostPath	Pod Lifetime	Node filesystem	Yes (until deleted)	Yes (within Pod)	Access node files, logs
ConfigMap	Until deleted	etcd	Yes	Yes (within Pod)	Configuration files
Secret	Until deleted	etcd	Yes	Yes (within Pod)	Sensitive data, credentials
CSI	PVC Lifecycle	Storage backend	Yes	Yes (RWO, RWX, ROX)	Persistent storage, advanced features

8. BEST PRACTICES

- ☑ Use emptyDir for temporary, non-critical data.
- ☑ Avoid hostPath in production unless necessary.
- ☑ Use ConfigMap for non-sensitive configuration.
- ☑ Use Secret for sensitive data only.
- ☑ Prefer CSI with PVC for persistent storage.
- ☑ Use RBAC to secure Secrets and storage resources.

REMEMBER

Choose the right storage type for the right purpose. Wrong choice can lead to data loss or security risks.

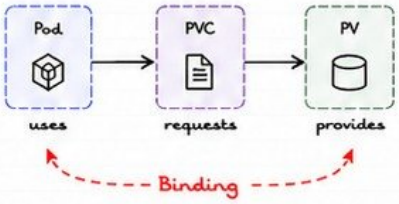
9. TROUBLESHOOTING

1. Pod cannot access files? → Check mountPath.
2. Data not persisting? → Check Volume type.
3. Permission denied? → Check file permissions.
4. hostPath issues? → Verify node path exists.
5. CSI provisioning failed? → Check StorageClass and driver logs.
6. Secret not mounting? → Check secret name and namespace.

25. PERSISTENT VOLUMES

Persistent Volumes (PV) provide cluster storage. Persistent Volume Claims (PVC) request storage. Kubernetes binds PVCs to PVs automatically.

1. CONCEPT OVERVIEW



KEY POINTS

- PV is cluster resource created by admin.
- PVC is namespace resource requested by user.
- PVC and PV are bound based on capacity, access modes, and StorageClass.
- Pod uses storage through the PVC.
- PV lifecycle is independent of Pods.

2. PERSISTENT VOLUME (PV)

Cluster-wide storage resource provisioned by admin.

EXAMPLE PV (NFS)

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: nfs-storage
  nfs:
    server: 10.0.0.20
    path: /data/k8s
status:
  phase: Available
```

PV is Available until it is bound to a PVC.

3. PERSISTENT VOLUME CLAIM (PVC)

Namespace-scoped request for storage.

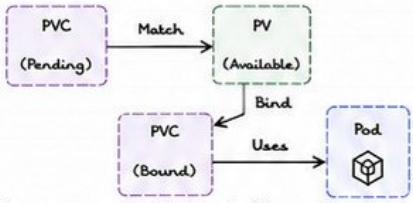
EXAMPLE PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
  storageClassName: nfs-storage
```

PVC requests 5Gi of storage with RWX access using nfs-storage StorageClass.

4. BINDING PROCESS

1. Admin creates PV (Available).
2. User creates PVC (Pending).
3. Kubernetes matches PVC with PV based on:
   - ☑ Storage capacity
   - ☑ Access modes
   - ☑ StorageClass (if specified)
4. PVC is bound to PV.
5. Pod can now use the storage.



Once bound, PVC and PV remain bound until released.

5. RECLAIM POLICIES

Defines what happens to PV when PVC is deleted.

POLICY	BEHAVIOR
Retain	PV is kept. Admin must manually reclaim data and reuse.
Delete	PV and underlying storage are deleted automatically.
Recycle (Deprecated)	PV is cleaned (basic rm -rf) and made Available again.

EXAMPLE PV WITH RETAIN

```yaml
spec:
  persistentVolumeReclaimPolicy: Retain
```

⚠ Use Retain for critical or stateful data.
⚠ Use Delete for dynamic, non-critical data.

6. STORAGE CLASSES

StorageClass defines how PVs are provisioned (dynamic or static).

EXAMPLE STORAGECLASS

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp2
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

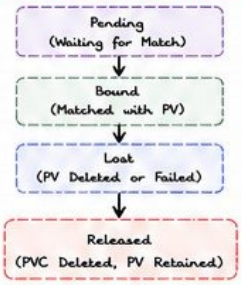
If PVC specifies a StorageClass, dynamic provisioning creates PV automatically.

7. ACCESS MODES

ACCESS MODE	DESCRIPTION
ReadWriteOnce (RWO)	Mounted as read-write by a single node.
ReadOnlyMany (ROX)	Mounted read-only by multiple nodes.
ReadWriteMany (RWX)	Mounted read-write by multiple nodes.

Choose the correct access mode based on application requirements.

8. PVC STATUS LIFECYCLE

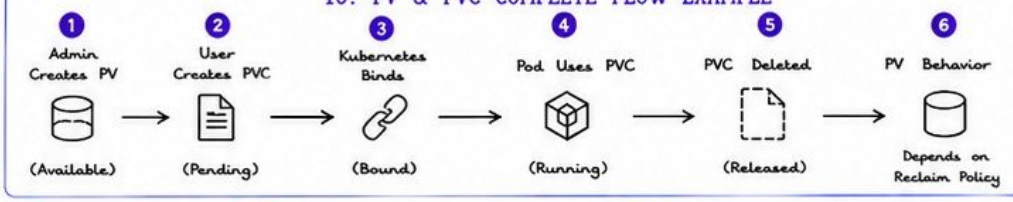


9. TROUBLESHOOTING

Common issues and checks.

1. PVC stuck in Pending
   - Check if matching PV exists.
   - Verify StorageClass and capacity.
   - Check access modes.
2. PVC not binding
   - Check PV labels and StorageClass.
   - Ensure PV is Available.
3. Pod cannot mount volume
   - Check PVC is Bound.
   - Check PV attaches and backend storage.
4. PV not released after PVC delete
   - Check reclaimPolicy.
   - For Retain, clean up manually.
5. Storage full
   - Check PV capacity and usage.
   - Expand PVC if StorageClass supports it.

10. PV & PVC COMPLETE FLOW EXAMPLE



REMEMBER

- ☑ PV is cluster-wide.
- ☑ PVC is namespace-scoped.
- ☑ Binding happens automatically.
- ☑ Reclaim policy controls lifecycle.
- ☑ Choose correct access mode.
- ☑ Monitor storage usage always.

26. STORAGE CLASSES

StorageClasses define how Persistent Volumes (PV) are provisioned dynamically.
They allow administrators to offer different storage types without pre-creating PVs.

1. STORAGE CLASS OVERVIEW

StorageClass is a cluster resource that defines provisioners, parameters, and policies for dynamic storage.

KEY POINTS

- Enables dynamic provisioning of PVs.
- Decouples storage admin from app teams.
- Supports different performance, size, and cost requirements.
- Can set default class for automatic use.

2. DYNAMIC PROVISIONING

PVC references a StorageClass.
If no matching PV exists, a new PV is created automatically.

FLOW

1. User creates PVC with StorageClass.
2. Kubernetes finds no matching PV.
3. Storage provisioner creates PV.
4. PV is bound to PVC.
5. Pod can now use the storage.

EXAMPLE STORAGECLASS

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp2
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Dynamic provisioning saves time and avoids manual PV management.

3. DEFAULT STORAGE CLASSES

One StorageClass can be marked as default.
PVCs without a class will use it.

ANNOTATION	DESCRIPTION
storageclass.kubernetes.io/is-default-class: "true"	Marks this StorageClass as the default class.

EXAMPLE DEFAULT CLASS

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Only one StorageClass should be default at a time.

4. VOLUME EXPANSION

If allowVolumeExpansion is true, PVC size can be increased.

REQUIREMENTS

- StorageClass must set allowVolumeExpansion: true
- The provisioner must support expansion.
- Filesystem must support online expansion.
- For some FS, manual resize is required in Pod.

EXAMPLE (STORAGECLASS)

```yaml
allowVolumeExpansion: true
```

Expansion doesn't work for all storage types. Check provider documentation.

5. STORAGE CLASS PARAMETERS

Parameters are passed to the provisioner to customize storage.

PARAMETER	EXAMPLE	DESCRIPTION
type	gp3	Disk type or performance tier
iops	3000	IOPS for the disk
throughput	125	Throughput in MB/s
encrypted	true	Enable encryption
fsType	ext4	Filesystem type
zone	us-east-1a	Availability zone

EXAMPLE (WITH PARAMETERS)

```yaml
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
  fsType: ext4
```

Parameters vary by storage provisioner. Check your cloud provider docs.

6. TROUBLESHOOTING

Common issues when using StorageClasses.

ISSUE	SYMPTOMS	WHAT TO CHECK
PVC Pending	PVC not bound to PV	Check StorageClass name, provisioner, and events
Provisioning Failed	PVC events show provisioning errors	Check provisioner logs and parameters
No Matching Provisioner	PVC stuck in Pending	Ensure provisioner is installed
Default Class Not Used	PVC Pending without class	Check default annotation on StorageClass
Expansion Not Working	PVC size updated but not expanded	Check allowVolumeExpansion, FS support, and resize logs

USEFUL COMMANDS

```
kubectl get storageclass
kubectl describe storageclass <name>
kubectl get pvc
kubectl describe pvc <name>
kubectl get events --sort-by='.metadata.creationTimestamp'
```

7. STORAGE CLASS COMPARISON

FEATURE	STATIC PROVISIONING	DYNAMIC PROVISIONING (WITH STORAGECLASS)
PV Creation	Manual (Admin creates PV first)	Automatic (Created when PVC is requested)
Ease of Use	Complex	Simple
Scalability	Hard to scale	Easy to scale
Customization	Limited	High (via parameters)
Best For	Special cases, manual setups	Most applications
Default Support	N/A	Can set default StorageClass

BEST PRACTICES

- ☑ Use dynamic provisioning for most workloads.
- ☑ Mark one StorageClass as default.
- ☑ Use allowVolumeExpansion where supported.
- ☑ Monitor provisioner logs and events.
- ☑ Choose the right storage type for performance and cost.

KEY TAKEAWAY

- StorageClasses enable dynamic, automated storage.
- They define provisioners, parameters, and policies.
- Use the right class, monitor health, and automate.

VERIQTA
BUILD. AUTOMATE. SCALE.

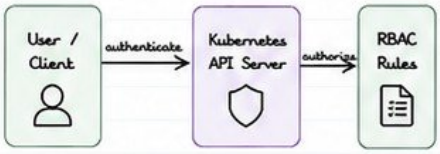
FINAL PRINCIPLE

Good storage design ensures performance, reliability, and scalability. StorageClasses make it simple and consistent.

27. AUTHENTICATION

Authentication verifies who you are. Kubernetes supports multiple authentication methods to secure the cluster and control access.

1. AUTHENTICATION OVERVIEW



KEY POINTS

- Authentication is the first step before authorization (RBAC).
- The API Server validates identity.
- After authentication, RBAC decides what you can do.
- Always use strong authentication methods.

2. CERTIFICATES

Certificates provide strong, cluster-wide authentication for users and components.

TYPES OF CERTIFICATES

- Client Certificates (users, admins, components)
- Server Certificates (API Server, etcd)
- CA Certificate (Certificate Authority)
- kubelet Certificate
- kube-proxy Certificate

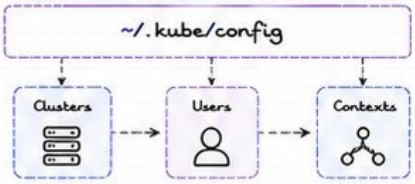
EXAMPLE CLIENT CERTIFICATE AUTH

```
# Generate key and CSR
openssl genrsa -out user.key 2048
openssl req -new -key user.key -out user.csr
# Sign with CA
openssl x509 -req -in user.csr -CA ca.crt
-CAkey ca.key -CAcreateserial -out user.crt
-days 365
```

Certificates are secure and support fine-grained access control.

3. KUBECONFIG

kubectl uses kubeconfig to connect and authenticate to a cluster.



kubectl uses the current context to connect to the cluster.

KEY COMPONENTS

- Clusters : Cluster endpoint and CA cert
- Users : Authentication info (certs, tokens)
- Contexts : Combination of cluster + user
- Current Context : Active context used by kubectl

COMMON COMMANDS

```
kubectl config get-contexts
kubectl config use-context <name>
kubectl config current-context
kubectl config view
```

Protect your kubeconfig file. It contains sensitive access information.

4. SERVICE ACCOUNTS

Service Accounts provide identities for applications running in Pods.

KEY POINTS

- Every namespace has a default ServiceAccount.
- Pods automatically use the default SA unless specified.
- Custom ServiceAccounts can be created.
- Used by workloads to access the Kubernetes API.

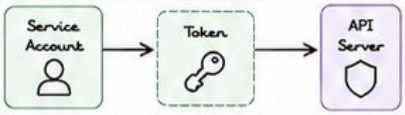
EXAMPLE

```
# Create a ServiceAccount
kubectl create serviceaccount my-app-sa
# Use in a Pod
kubectl run my-app --image=nginx --serviceaccount=my-app-sa
```

Use dedicated ServiceAccounts for each application. Do not use the default SA in production.

5. TOKENS

Tokens are used by ServiceAccounts and users for API authentication.



TOKEN TYPES

- Legacy Tokens (long-lived, stored as Secrets)
- Bound Tokens (short-lived, projected to Pods)
- TokenRequest API (preferred for dynamic tokens)

EXAMPLE: CREATE TOKEN (MANUAL)

```
# Create token secret (legacy method)
kubectl create token my-app-sa
# Or use TokenRequest (bound token)
kubectl create token my-app-sa \
  --duration=1h
```

Prefer Bound Tokens. They are short-lived and automatically rotated.

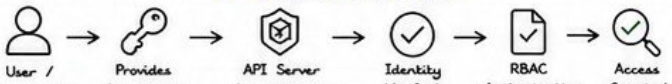
6. AUTHENTICATION METHODS SUMMARY

METHOD	DESCRIPTION
Client Certificates	Strong, secure; used by users and components.
kubeconfig	Holds connection and authentication details.
Service Accounts	Identities for applications running in Pods.
Tokens	Used by ServiceAccounts and users for API access.

BEST PRACTICES

- ☑ Use certificates for human and component access.
- ☑ Use kubeconfig securely and restrict file access.
- ☑ Always use dedicated ServiceAccounts.
- ☑ Prefer bound tokens over legacy tokens.
- ☑ Rotate certificates and tokens regularly.
- ☑ Follow least privilege with RBAC.

7. AUTHENTICATION FLOW



REMEMBER

- ☑ Authentication answers: WHO ARE YOU?
- ☑ Authorization answers: WHAT CAN YOU DO?
- ☑ Both are required for secure Kubernetes access.

8. TROUBLESHOOTING

- ☑ Check certificate validity: openssl x509 -in <cert> -noout -dates
- ☑ Verify kubeconfig context and user.
- ☑ Ensure ServiceAccount exists and token is mounted.
- ☑ Check RBAC permissions after authentication.
- ☑ Check API Server logs for authentication errors.

VERIQTA

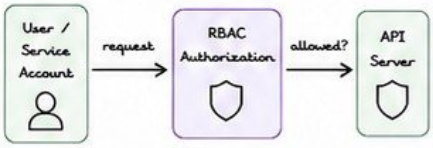
BUILD. AUTOMATE. SCALE.

28. AUTHORIZATION

Authorization determines what you are allowed to do. Kubernetes uses RBAC (Role-Based Access Control) to manage permissions.

1. AUTHORIZATION OVERVIEW

After authentication, Kubernetes checks whether you are allowed to perform an action using RBAC rules.

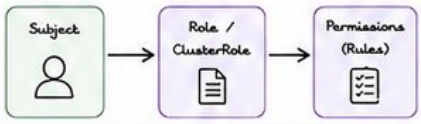


KEY POINTS

- Authorization answers: "Can you do this?"
- RBAC is the native authorization mode.
- Based on Roles, ClusterRoles, and Bindings.
- Always follow the principle of least privilege.

2. RBAC OVERVIEW

RBAC uses rules to grant permissions to users, groups, or ServiceAccounts.



RBAC COMPONENTS

- Subjects : Users, Groups, ServiceAccounts
- Roles : Namespace-scoped permissions
- ClusterRoles : Cluster-wide permissions
- Bindings : Attach Roles to Subjects
- Rules : Define allowed actions

KEY POINTS

- Roles are limited to a namespace.
- ClusterRoles work across the cluster.
- Bindings connect subjects to roles.

3. ROLES (NAMESPACE-SCOPED)

Roles define permissions within a single namespace.

EXAMPLE ROLE

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log", "pods/status"]
  verbs: ["get", "list", "watch"]
```

EXPLANATION

This Role allows reading Pods in the "dev" namespace only.

Use Roles for namespace-specific access.

4. CLUSTERROLES (CLUSTER-WIDE)

ClusterRoles define permisation across the entire cluster.

EXAMPLE CLUSTERROLE

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-admin-readonly
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["get", "list", "watch"]
```

EXPLANATION

This ClusterRole allows read-only access to all resources in the entire cluster.

Use ClusterRoles for cluster-wide permissions.

5. BINDINGS

Bindings attach Roles or ClusterRoles to subjects (users, groups, or ServiceAccounts).

5A. ROLEBINDING (NAMESPACE-SCOPED)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: dev
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

5B. CLUSTERROLEBINDING (CLUSTER-WIDE)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin-binding
subjects:
- kind: Group
  name: devops-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
```

Use RoleBinding for namespaces.
Use ClusterRoleBinding for cluster-wide access.

6. RULES (PERMISSIONS)

Rules define what actions are allowed.

RULE STRUCTURE

- apiGroups : API group ("" for core API)
- resources : Resource names (pods, services, ...)
- verbs : Allowed actions (get, list, create, ...)
- resourceNames (optional): Specific resource names

COMMON VERBS

get	Get a single resource
list	List resources
watch	Watch for changes
create	Create a resource
update	Update a resource
patch	Partially update a resource
delete	Delete a resource

EXAMPLE RULE

```yaml
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

Grant only the verbs and resources needed.
Avoid wildcard access.

7. LEAST PRIVILEGE PRINCIPLE

Grant only the minimum permisions required.

DO	DON'T
☑ Use specific resources	⊗ Use wildcard * unless necessary
☑ Use limited verbs	⊗ Grant cluster-admin without reason
☑ Use Roles instead of ClusterRoles when possible	⊗ Over-permission Service Accounts
☑ Review permissions regularly	

Least privilege reduces risk and improves security.

8. RBAC COMPONENTS SUMMARY

COMPONENT	SCOPE	PURPOSE	EXAMPLE
Role	Namespace	Define permissions in a namespace	Read pods in dev
ClusterRole	Cluster-wide	Define permissions across cluster	Read all nodes
RoleBinding	Namespace	Bind Role to a subject in namespace	User alice -> Role
ClusterRoleBinding	Cluster-wide	Bind ClusterRole to a subject	Group -> ClusterRole
Subject	N/A	User, Group, or ServiceAccount	DevOps team
Rule	N/A	Define allowed actions	get, list, watch

9. RBAC FLOW

Authentication identifies you. Authorization decides what you are allowed to do.

10. TROUBLESHOOTING AUTHORIZATION

- ☑ Error: "Forbidden"
  → Check your Role/ClusterRole permissions.
- ☑ Use: kubectl auth can-i <verb> <resource> [--as user]
  → Example: kubectl auth can-i get pods --as alice
- ☑ Verify RoleBindings and ClusterRoleBindings.
- ☑ Ensure you are using the correct namespace.
- ☑ Check API Server logs for authorization errors.

REMEMBER

- ☑ Authenticate first. Authorize second.
- ☑ RBAC controls what you can do.
- ☑ Use least privilege always.

VERIQTA
BUILD. AUTOMATE. SCALE.

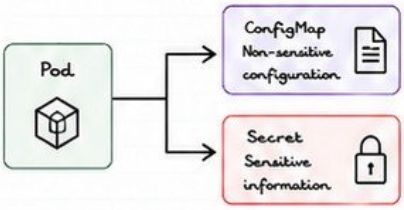
Good authorization is not about giving power, it is about giving the right power to the right person. Nothing more, nothing less.

29. SECRETS & CONFIGMAPS

Secrets store sensitive data. ConfigMaps store non-sensitive configuration.
Use them to keep applications flexible and secure.

1. OVERVIEW

Secrets and ConfigMaps decouple configuration from container images and keep sensitive data out of images and manifests.



KEY POINTS

- Do not store sensitive data in images or code.
- Use Secrets for passwords, tokens, keys, certs.
- Use ConfigMaps for settings and feature flags.
- Mount or inject them into Pods as needed.
- Follow least privilege and RBAC for access.

2. SECURE MANAGEMENT

- Store sensitive data in Secrets, not in ConfigMaps or plain YAML.
- Enable RBAC and restrict access to Secrets and ConfigMaps.
- Encrypt Secrets at rest using etcd encryption.
- Do not log or expose Secret data.
- Rotate Secrets regularly.

CREATE SECRET (EXAMPLE)

```
# Generic Secret
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=MyP@sswOrd

# TLS Secret
kubectl create secret tls tls-secret \
  --cert=cert.crt --key=key.key
```

Use dedicated ServiceAccounts and RBAC to protect Secret and ConfigMap access.

3. MOUNTING INTO PODS

Secrets and ConfigMaps can be mounted as volumes or used as environment variables.

VOLUME MOUNT

```
volumes:
 - name: app-config
   configMap:
     name: app-config

volumeMounts:
  - name: app-config
    mountPath: /etc/config
    readOnly: true
```

ENVIRONMENT VARIABLES

```
env:
 - name: DB_USER
   valueFrom:
     secretKeyRef:
       name: db-secret
       key: username
```

Mount as readonly to prevent accidental modification.

4. CONFIGMAPS (NON-SENSITIVE DATA)

Store application configuration, properties, or feature flags.

CREATE CONFIGMAP (EXAMPLE)

```
kubectl create configmap app-config \
  --from-literal=app.mode=production \
  --from-literal=featureX=true
OR
kubectl create configmap app-config \
  --from-file=config.properties
```

EXAMPLE USAGE

```
env:
 - name: APP_MODE
   valueFrom:
     configMapKeyRef:
       name: app-config
       key: app.mode
```

ConfigMaps are not encrypted. Do not store sensitive data in ConfigMaps.

5. ENVIRONMENT VARIABLES

Inject configuration or secrets into container environment variables.

FROM CONFIGMAP

```
env:
 - name: LOG_LEVEL
   valueFrom:
     configMapKeyRef:
       name: app-config
       key: log.level
```

FROM SECRET

```
env:
 - name: DB_PASSWORD
   valueFrom:
     secretKeyRef:
       name: db-secret
       key: password
```

Environment variables are visible in Pod descriptions. Avoid storing highly sensitive data.

6. UPDATES & RELOADS

How updates to ConfigMaps and Secrets affect running Pods.

RESOURCE	UPDATE BEHAVIOR	POD IMPACT
ConfigMap	Mounted as volume: updates reflect automatically.	Pod does not restart.
ConfigMap	Used as env var: updates require Pod restart.	Pod must be restarted.
Secret	Mounted as volume: updates reflect automatically.	Pod does not restart.
Secret	Used as env var: updates require Pod restart.	Pod must be restarted.

Use rolling updates or restart Pods to pick up env-var changes.

7. PRACTICAL EXAMPLE

1. Create ConfigMap — app-config
2. Create Secret — db-secret
3. Reference in Pod — mount / env
4. Application Uses — config + secret
5. Update & Reload — if needed

8. TROUBLESHOOTING

- ☑ Secret/ConfigMap not found → check name and namespace.
- ☑ Pod not picking up updates → check mount type and restart.
- ☑ Permission denied → check RBAC and ServiceAccount.
- ☑ Incorrect key → verify key name in manifest.
- ☑ Data not showing → ensure readonly mount and correct path.

REMEMBER

- ☑ Use Secrets for sensitive data.
- ☑ Use ConfigMaps for non-sensitive config.
- ☑ Follow least privilege and limit access.
- ☑ Rotate Secrets and keep them up to date.

VERIQTA
BUILD. AUTOMATE. SCALE.

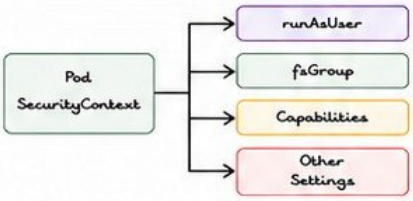
Good configuration is flexible.
Good security is non-negotiable.
Use Secrets and ConfigMaps wisely.

30. SECURITY CONTEXTS

Security Contexts define privilege and access settings for Pods and containers. They help enforce the principle of least privilege and improve cluster security.

1. OVERVIEW

SecurityContext settings control what a container can do and what it can access.



KEY POINTS

- Reduce risk by dropping unnecessary privileges.
- Prevent privilege escalation and container escape.
- Apply at Pod level or Container level.
- Use the minimum permissions required.

Always follow the principle of least privilege.

2. runAsUser

Defines the user ID that runs the container process inside the container.

EXAMPLE

```
securityContext:
  runAsUser:  10001
  runAsGroup:  10001
  runAsNonRoot:  true
```

PROPERTIES

PROPERTY	DESCRIPTION
runAsUser	User ID to run the container.
runAsGroup	Primary group ID.
runAsNonRoot	Prevent running as root (UID 0).
runAsUserName*	(K8s 1.28+) Run as a user name.

Use runAsNonRoot: true to ensure containers do not run as root.

- Requires image to support /etc/passwd or NSS.

3. fsGroup

Sets the group ownership for files mounted in the Pod volumes.

EXAMPLE

```
securityContext:
  fsGroup:  2000
  fsGroupChangePolicy:  "OnRootMismatch"
```

PROPERTIES

PROPERTY	DESCRIRTION
fsGroup	Group ID for volume ownership.
fsGroupChangePolicy	Controls how ownership changes are applied.

- Ensures shared write access for non-root users.
- OnRootMismatch (default): changes only when root mismatch is detected.
- Always: always changes ownership (slower).

Use fsGroup for shared volumes (e.g., PVC).

4. CAPABILITIES

Linux capabilities allow finer control over what system actions a container can perform.

COMMON CAPABILITIES

CAPABILITY	PURPOSE
CHOWN	Change file ownership.
NET_BIND_SERVICE	Bind to privileged ports (<1024).
NET_ADMIN	Network administration.
SYS_TIME	Set system clock.
DAC_OVERRIDE	Bypass file read, write, execute permission checks.
SYS_PTRACE	Trace processes.

ADD CAPABILITY

```
securityContext:
  capabilities:
  add:  ["NET_BIND_SERVICE"]
```

DROP CAPABILITIES

```
securityContext:
  capabilities:
  drop:  ["ALL"]
```

Drop all by default and add only what is required.

5. PRIVILEGED CONTAINERS

Running in privileged mode gives all capabilities and access to the host.

PRIVILEGED: TRUE

- Full access to host.
- Bypass all kernel security.
- High risk.

PRIVILEGED: FALSE

- Limited access.
- Isolated from host.
- Recommended.

EXAMPLE

```
securityContext:
  privileged:  false
```

USE PRIVILEGED ONLY WHEN ABSOLUTELY NECESSARY!

- Not allowed in most production clusters.
- Breaks container isolation.
- Increases attack surface.

6. READONLYROOTFILESYSTEM

Mount the container's root filesystem as read-only to prevent modifications.

EXAMPLE

```
securityContext:
  readOnlyRootFilesystem:  true
  allowPrivilegeEscalation:  false
```

EFFECTS

- ☑ Prevents writes to the container image layer.
- ☑ Blocks malware from modifying binaries.
- ☑ Improves security and immutability.
- ☑ Applications must write to mounted volumes (e.g., emptyDir, PVC).

Combine with writable volumes for /tmp, /var/log, or /data as needed.

7. OTHER IMPORTANT SETTINGS

SETTING	PURPOSE
allowPrivilegeEscalation: false	Prevents gaining more privileges (e.g., via setuid binaries).
seccompProfile: type: RuntimeDefault	Apply default syscall filter.
appArmorProfile: type: RuntimeDefault	Apply AppArmor profile (if supported).
readOnlyRootFilesystem: true	Make root filesystem read-only.
privileged: false	Disallow privileged mode.
runAsNonRoot: true	Prevent running as root.
capabilities.drop: ["ALL"]	Drop all capabilities by default.

8. PRACTICAL EXAMPLE (POD SECURITY CONTEXT)

```
apiVersion:  v1
kind:  Pod
metadata:
  name:  secure-pod
spec:
  securityContext:
    runAsUser:  10001
    runAsGroup:  10001
    fsGroup:  2000
    fsGroupChangePolicy:  OnRootMismatch
  containers:
  - name:  app
    image:  nginx:alpine
    securityContext:
      allowPrivilegeEscalation:  false
      capabilities:
        drop:  ["ALL"]
        add:  ["NET_BIND_SERVICE"]
      readOnlyRootFilesystem:  true
    volumeMounts:
    - name:  app-data
      mountPath:  /data
  volumes:
  - name:  app-data
    emptyDir:  {}
```

WHAT THIS DOES

- ☑ Runs as non-root user.
- ☑ Sets group ownership for volumes.
- ☑ Drops all capabilities, adds only required one.
- ☑ Prevents privilege escalation.
- ☑ Makes root filesystem read-only.
- ☑ Uses writable volume at /data.

RESULT

Minimal privileges.
Better isolation.
Stronger security.

9. SECURITY CONTEXT FLOW

10. TROUBLESHOOTING

- ☑ Container fails to start → Check runAsUser and permissions.
- ☑ Permission denied → Check fsGroup and volume ownership.
- ☑ App cannot bind to port → Add NET_BIND_SERVICE capability.
- ☑ Writes fail → Check readOnlyRootFilesystem and volume mounts.
- ☑ Unexpected privilege → Verify capabilities and privileged: false.

REMEMBER

- ☑ Use the minimum privileges needed.
- ☑ Drop all capabilities and add selectively.
- ☑ Avoid privileged containers.
- ☑ Review and test security contexts regularly.

VERIQTA
BUILD. AUTOMATE. SCALE.

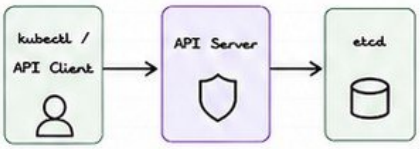
Security Contexts are your first line of defense inside Kubernetes.
Least privilege = Less risk.

31. ADMISSION CONTROL

Admission Control intercepts API requests before objects are persisted.
It enforces policies to keep the cluster secure, stable, and consistent.

1. OVERVIEW

The API server runs admission controllers that can allow, modify, or reject requests to create or update cluster objects.



KEY POINTS

- Controllers run in a defined order.
- A request can be allowed, denied, or mutated.
- Built-in and third-party controllers exist.
- You can enable/disable controllers.
- Order matters for security and functionality.

2. ADMISSION CONTROLLERS

Built-in admission controllers provided by Kubernetes.

CONTROLLER	PURPOSE
AlwaysAdmit	Allows all requests. (not secure)
AlwaysPullImages	Ensures images are always pulled before start.
DenyExecOnPrivileged	Blocks exec into privileged containers.
NamespaceLifecycle	Prevents operations on terminating namespaces.
NodeRestriction	Restricts kubelet from modifying nodes.
PodSecurity	Deprecated PSA predecessor.
PodSecurityAdmission	Enforces Pod Security Standards.
ResourceQuota	Enforces resource quotas.
LimitRanger	Sets default limits and validates resources.
ServiceAccount	Auto-attach default SA.
DefaultStorageClass	Sets default StorageClass.
MutatingAdmissionWebhook	Calls external webhook to mutate objects.
ValidatingAdmissionWebhook	Calls external webhook to validate (allow/deny).

Not all controllers are enabled by default.
Check your cluster configuration.

3. POD SECURITY ADMISSION (PSA)

Enforces security standards on Pods at admission time.

THREE SECURITY LEVELS

PRIVILEGED	BASELINE	RESTRICTED
Unrestricted policy.	Minimally restrictive.	Highly restrictive.

PSA MODES

- enforce – Reject non-compliant Pods.
- audit – Allow but log warnings.
- warn – Show warning to users.

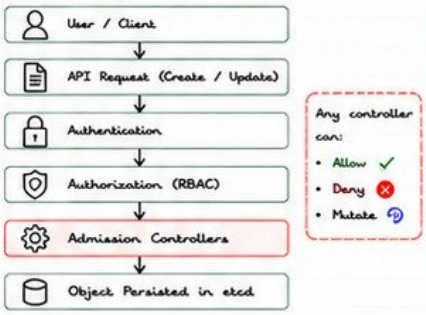
LABEL CONFIGURATION (EXAMPLE)

```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    pod-security.kubernetes.io/enforce: "restricted"
    pod-security.kubernetes.io/audit: "baseline"
    pod-security.kubernetes.io/warn: "baseline"
```

Use PSA to block insecure Pods at the door, before they run.

4. ADMISSION CONTROL FLOW

Request lifecycle from client to etcd.



If any controller denies the request, the object is not created.

5. ENABLING / DISABLING CONTROLLERS

Configured on the API server.

CHECK ENABLED CONTROLLERS

```
kube-apiserver -h | grep enable-admission-plugins
```

EXAMPLE: ENABLE / DISABLE

```
# Enable PodSecurityAdmission
--enable-admission-plugins=...,PodSecurityAdmission,...
# Disable AlwaysPullImages
--disable-admission-plugins=AlwaysPullImages
```

IMPORTANT NOTES

- Some controllers cannot be disabled.
- Changing controllers requires API server restart.
- Be careful: disabling security controllers can expose the cluster.

Validate configuration in non-production before applying to production.

6. PRACTICAL EXAM SCENARIOS

Common CKA exam tasks.

SCENARIO	WHAT TO DO
Deny privileged containers	Enable PodSecurityAdmission with enforce=restricted (or baseline).
Block hostPath volumes	Use PSA restricted policy. (hostPath not allowed)
Limit allowed images	Enable ImagePolicyWebhook or AlwaysPullImages.
Restrict high resources	Use ResourceQuota + LimitRanger.
Deny exec into privileged Pods	Enable DenyExecOnPrivileged.
Audit insecure Pods	Set PSA audit=baseline or warn.

Always verify with kubectl describe and API server logs.

7. ADMISSION CONTROLLER ORDER (SAMPLE)

Order matters. Request flows through controllers in sequence.

If any step denies, the request is rejected.

8. ADMISSION CONTROL BEST PRACTICES

- ☑ Enable PodSecurityAdmission and use "baseline" or "restricted".
- ☑ Keep only necessary controllers enabled.
- ☑ Use RBAC + PSA together for strong security.
- ☑ Use ResourceQuota and LimitRanger for resource governance.
- ☑ Audit before enforcing strict policies.
- ☑ Test in staging before applying to production.

REMEMBER

- ☑ Admission Control is your first line of defense.
- ☑ PSA = Simple and powerful.
- ☑ Least privilege keeps your cluster safe.

VERIQTA
BUILD. AUTOMATE. SCALE.

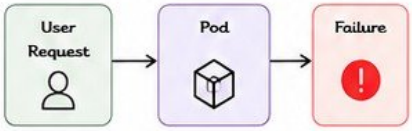
Secure admission today prevents incidents tomorrow. Control what enters your cluster.

32. APPLICATION FAILURES

Application failures occur when containers cannot start, images cannot be pulled, or Pods remain in Pending state. This page covers the most common scenarios you must troubleshoot in the CKA exam.

1. OVERVIEW

Application failures prevent your workloads from running as expected.
Your goal is to identify the root cause quickly and restore service.



KEY POINTS

- Understand Pod status and conditions.
- Read events and logs for root cause.
- Fix configuration, image, resources, or cluster issues.
- Verify the solution with a new Pod.

Always follow a systematic troubleshooting approach.

2. CRASHLOOPBACKOFF

Container starts, crashes, and restarts repeatedly.
Kubernetes backs off the restart.

COMMON CAUSES

- Application exits with error.
- Misconfiguration.
- Failed health checks.
- Missing dependencies.
- Out of memory.

TROUBLESHOOTING STEPS

1. kubectl get pods -o wide
2. kubectl describe pod <pod-name>
3. kubectl logs <pod-name>
4. kubectl logs <pod-name> -c <container> --previous
5. Check events for warnings or errors.
6. Fix the issue and restart the Pod.

EXAMPLE OUTPUT

```
$ kubectl get pods
NAME                      READY   STATUS             RESTARTS   AGE
my-app-6d7d5f7b7c-abcde   0/1     CrashLoopBackOff   7          3m
```

Look at logs and events to find the real error.

3. IMAGEPULLBACKOFF

Kubelet cannot pull the container image.
It keeps retrying and backs off.

COMMON CAUSES

- Incorrect image name or tag.
- Image does not exist.
- Private registry authentication failure.
- Network or DNS issues.
- Image pull secret missing.

TROUBLESHOOTING STEPS

1. kubectl describe pod <pod-name>
2. Check events for image pull errors.
3. Verify image name and registry.
4. Check imagePullSecrets.
5. Test image pull manually.

EXAMPLE EVENT

Failed to pull image "myregistry.com/app:v1": rpc error: code = Unknown desc = requested access to the resource is denied

Confirm image name, registry, and credentials.
Test pull outside the cluster if needed.

4. PENDING PODS

Pod is not running and remains in Pending state.
It has not been scheduled to a node.

COMMON CAUSES

- Insufficient resources.
- Node selectors / affinity mismatch.
- Taints not tolerated.
- Missing PersistentVolume.
- PodSecurity / Admission issues.

TROUBLESHOOTING STEPS

1. kubectl get pods -o wide
2. kubectl describe pod <pod-name>
3. Check events for scheduling errors.
4. Verify resource requests and limits.
5. Review node selectors and tolerations.

Events will show the exact reason why the Pod cannot be scheduled.

5. FAILED SCHEDULING

Scheduler cannot place the Pod on any node.
Events show the exact reason.

COMMON REASONS

REASON	DESCRIPTION
Insufficient CPU	Not enough CPU on any node.
Insufficient Memory	Not enough memory on any node.
NodeSelector Mismatch	Pod requires a label that nodes do not have.
Taint Not Tolerated	All nodes have taints the Pod does not tolerate.
No nodes available	All nodes are cordoned or down.

TROUBLESHOOTING COMMANDS

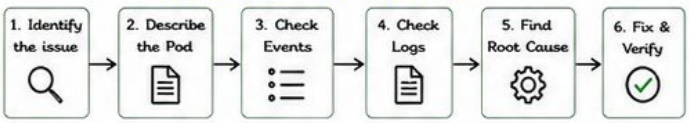
- kubectl describe pod <pod-name>
- kubectl get nodes --show-labels
- kubectl get events --sort-by=.lastTimestamp
- kubectl top nodes

Read the events carefully. They explain why the Pod cannot be scheduled.

6. QUICK REFERENCE COMMANDS

PURPOSE	COMMAND
Check Pod status	kubectl get pods -o wide
Describe Pod	kubectl describe pod <pod-name>
Pod logs	kubectl logs <pod-name>
Previous logs	kubectl logs <pod-name> -c <c> --previous
Check events	kubectl get events --sort-by=.lastTimestamp
Check nodes	kubectl get nodes -o wide
Check node labels	kubectl get nodes --show-labels
Check resources	kubectl top nodes
Check storage	kubectl get pv,pvc
Test image pull	kubectl run test --image=<image> --rm -it --restart=Never

7. TROUBLESHOOTING FLOW



Follow this order every time. Do not guess the issue.

8. EXAM TIPS

- ☑ Always start with: kubectl describe pod <pod-name>
- ☑ Events are your best friend.
- ☑ Check logs (including previous logs) for application errors.
- ☑ Understand resource requests and limits.
- ☑ Check node selectors, affinity, taints, and tolerations.
- ☑ Practice fixing all scenarios in a real cluster.

REMEMBER

- ☑ Read events.
- ☑ Check logs.
- ☑ Verify configuration and resources.
- ☑ Test the fix with a new Pod.

VERIQTA
BUILD. AUTOMATE. SCALE.

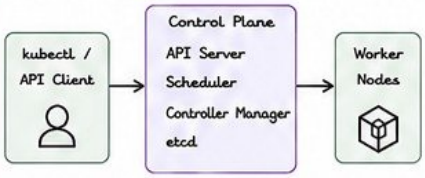
A systematic approach solves problems faster.
Stay calm, read carefully, and trust the data.

33. CONTROL PLANE TROUBLESHOOTING

The control plane runs the brain of Kubernetes. When components fail, the entire cluster can be affected. Know how to diagnose and fix issues.

1. OVERVIEW

The Kubernetes control plane consists of core components that manage cluster state and decisions.



KEY POINTS

- All components are critical.
- Issues in one component can cascade.
- Logs, health checks, and metrics are your best tools.
- Backup etcd regularly.

Always check the symptom, identify the component, then investigate.

2. API SERVER

The API Server is the entry point for all kubectl and component communication.

COMMON ISSUES

- API Server not responding.
- High latency or timeouts.
- Certificate or authentication errors.
- Apiserver down.

TROUBLESHOOTING STEPS

1. Check API Server Pod status.
2. Check Pod logs.
3. Verify API Server endpoint.
4. Check certificate expiration.
5. Check etcd connectivity.
6. Review API Server resource usage.

USEFUL COMMANDS

```
kubectl get pods -n kube-system -l component=kube-apiserver
kubectl logs <kube-apiserver-pod> -n kube-system
kubectl get --raw='/livez'
kubectl get --raw='/readyz'
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -dates
curl -k https://127.0.0.1:6443/healthz
```

HEALTH CHECKS

- Livez -> kubelet is alive.
- Readyz -> dependencies are ready.
- Healthz -> overall health.

If API Server is down, the cluster is unusable. Fix it immediately.

3. SCHEDULER

The Scheduler assigns Pods to nodes based on resources and constraints.

COMMON ISSUES

- Pods stuck in Pending.
- No nodes available.
- Scheduling failures.
- Incorrect node selection.

TROUBLESHOOTING STEPS

1. Check Scheduler Pod status.
2. Check Scheduler logs.
3. Check Pending Pods events.
4. Verify node resources and taints.
5. Check Pod node selectors, affinity, and tolerations.

USEFUL COMMANDS

```
kubectl get pods -n kube-system -l component=kube-scheduler
kubectl logs <kube-scheduler-pod> -n kube-system
kubectl get pods --all-namespaces | grep Pending
kubectl describe pod <pod-name>
kubectl get nodes
kubectl top nodes
```

Pending Pods? Check events first. Events show the real scheduling reason.

4. CONTROLLER MANAGER

The Controller Manager runs core controllers that keep the cluster in desired state.

COMMON ISSUES

- Controllers not reconciling.
- Resource objects not created/updated.
- High CPU or memory usage.
- Controller crashes or restarts.

TROUBLESHOOTING STEPS

1. Check Controller Manager Pod status.
2. Check logs for errors.
3. Identify affected controller.
4. Check API Server connectivity.
5. Review metrics for high load.

USEFUL COMMANDS

```
kubectl get pods -n kube-system -l component=kube-controller-manager
kubectl logs <kube-controller-manager-pod> -n kube-system
kubectl get events --all-namespaces --sort-by='.lastTimestamp'
kubectl get --raw /metrics | grep controller
kubectl top pod <kube-controller-manager-pod> -n kube-system
```

Look for repeated errors in logs. They tell you which controller is failing.

5. ETCD

etcd stores all cluster data. If etcd is down, the cluster loses its brain.

COMMON ISSUES

- etcd not responding.
- High disk usage.
- Slow performance.
- Data corruption.

TROUBLESHOOTING STEPS

1. Check etcd Pod status.
2. Check etcd logs.
3. Check etcd endpoint health.
4. Check disk space and inode usage.
5. Check etcd compaction status.
6. Verify backup integrity.

USEFUL COMMANDS

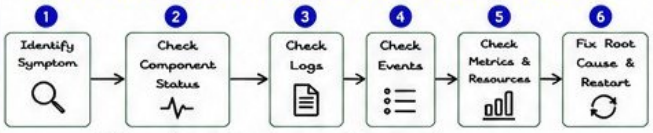
```
kubectl get pods -n kube-system -l component=etcd
kubectl logs <etcd-pod> -n kube-system
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 endpoint health
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 member list
df -h
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 status
```

No etcd = No cluster. Keep backups and monitor disk usage always.

6. QUICK REFERENCE COMMANDS

PURPOSE	COMMAND
Check all control plane Pods	`kubectl get pods -n kube-system`
Check Pod logs	`kubectl logs <pod> -n kube-system`
Check events	`kubectl get events --all-namespaces --sort-by='.lastTimestamp'`
Check node status	`kubectl get nodes`
Check component health	`kubectl get --raw='/livez'` `kubectl get --raw='/readyz'`
Check API resources	`kubectl api-resources`
Check cluster info	`kubectl cluster-info`
Check etcd members	`ETCDCTL_API=3 etcdctl member list`

7. CONTROL PLANE TROUBLESHOOTING FLOW



Follow this order. Do not skip steps!

8. BEST PRACTICES

- ☑ Monitor all control plane components.
- ☑ Alert on restarts, high CPU, high memory.
- ☑ Keep etcd backups and test restore regularly.
- ☑ Use dedicated nodes for control plane.
- ☑ Keep certificates valid and rotate before expiry.
- ☑ Document troubleshooting steps for your cluster.

REMEMBER

- ☑ API Server is the entry point.
- ☑ Scheduler places Pods on nodes.
- ☑ Controller Manager maintains desired state.
- ☑ etcd stores everything.
- ☑ If one fails, the cluster suffers.

VERIQTA

BUILD. AUTOMATE. SCALE.

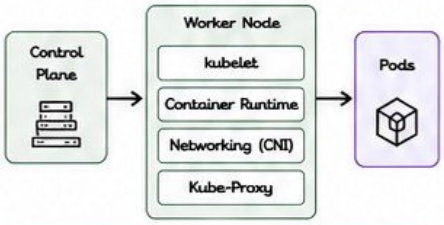
A healthy control plane means a healthy cluster. Monitor, troubleshoot, and keep it running strong.

34. WORKER NODE TROUBLESHOOTING

Worker nodes run your workloads. When a node has issues, Pods cannot start or run properly.
This page covers the most common worker node problems and fixes.

1. OVERVIEW

Each worker node has several critical components. When any of them fails, Pods on the node are affected.



KEY POINTS

- Node problems affect all Pods on the node.
- Check node status before checking individual Pods.
- Logs are your best friend.
- Fix the root cause, not just the symptoms.

Always start with: kubectl get nodes -o wide

2. KUBELET

kubelet ensures Pod containers are running on the node.

COMMON ISSUES

- kubelet service is down.
- kubelet cannot reach API Server.
- Invalid configuration.
- Certificate or permission errors.
- Disk pressure or resource issues.

TROUBLESHOOTING STEPS

1. Check kubelet service status.
2. Check kubelet logs.
3. Verify kubelet configuration.
4. Check connectivity to API Server.
5. Check node resources.
6. Restart kubelet if needed.

USEFUL COMMANDS

```
systemctl status kubelet
journalctl -u kubelet -xe
ps aux | grep kubelet
cat /var/lib/kubelet/config.yaml
kubectl get --raw='/healthz'
kubectl get nodes
```

If kubelet is down, Pods cannot run. Fix kubelet first.

3. CONTAINER RUNTIME

The container runtime pulls images and runs containers.

COMMON ISSUES

- Runtime service is down.
- Cannot pull images.
- Containers fail to start.
- Runtime crashes or hangs.
- Disk or storage issues.

TROUBLESHOOTING STEPS

1. Check runtime service status.
2. Check runtime logs.
3. Check image pull errors.
4. Verify runtime configuration.
5. Check disk space and inode usage.
6. Restart runtime if needed.

USEFUL COMMANDS

```
systemctl status containerd  # or docker
journalctl -u containerd -xe
crictl info
crictl ps -a
crictl images
crictl logs <container-id>
```

Use crictl for containerd and docker commands for Docker runtime.

4. NETWORKING (CNI)

CNI plugin provides Pod networking. Network issues can block Pods.

COMMON ISSUES

- CNI plugin pods are not running.
- Pod IP allocation failure.
- No network connectivity.
- DNS not working.
- kube-proxy issues.

TROUBLESHOOTING STEPS

1. Check CNI plugin pods in kube-system.
2. Check CNI logs.
3. Verify network connectivity.
4. Check kube-proxy status.
5. Check node firewall rules.
6. Restart CNI pods if needed.

USEFUL COMMANDS

```
kubectl get pods -n kube-system
kubectl logs -n kube-system <cni-pod>
ip a
ip route
iptables -L -n -v
systemctl status kube-proxy
```

No network = No Pod communication. Check CNI and kube-proxy.

5. NODE NOTREADY

When a node shows NotReady, Kubernetes will stop scheduling new Pods.

COMMON REASONS

- kubelet not running.
- Network unreachable.
- Disk pressure.
- Memory pressure.
- PID pressure.
- Kubelet cannot reach API Server.

TROUBLESHOOTING STEPS

1. Check node status and events.
2. Check kubelet logs.
3. Check resource usage.
4. Check network connectivity.
5. Check disk space.
6. Restart kubelet or reboot node if required.

USEFUL COMMANDS

```
kubectl get nodes -o wide
kubectl describe node <node-name>
top node <node-name>
df -h
free -h
dmesg | tail -n 50
```

Always read node events. They tell you the real reason.

6. EVENTS & LOGS

Events and logs help you find the root cause faster.

KEY CHECKS

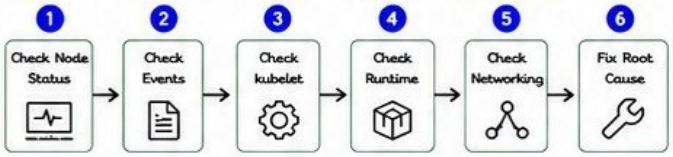
- Node events
- kubelet logs
- Runtime logs
- System logs
- CNI logs
- kube-proxy logs

USEFUL COMMANDS

```
kubectl get events --sort-by=.lastTimestamp
kubectl describe node <node-name>
journalctl -u kubelet -xe
journalctl -u containerd -xe
journalctl -u kube-proxy -xe
tail -f /var/log/syslog
```

Correlate events and logs to find the root cause.

7. WORKER NODE TROUBLESHOOTING FLOW



Follow the order. Do not skip steps.

8. QUICK FIXES

- [x] Restart kubelet: systemctl restart kubelet
- [x] Restart runtime: systemctl restart containerd
- [x] Restart kube-proxy: systemctl restart kube-proxy
- [x] Restart CNI pods: kubectl -n kube-system rollout restart ds
- [x] Reboot node if required
- [x] Ensure disk space and memory are sufficient
- [x] Fix network connectivity

REMEMBER

- [x] Healthy nodes = Healthy workloads.
- [x] Monitor node resources continuously.
- [x] Check logs before making changes.
- [x] Fix the root cause, not the symptom.
- [x] Document issues and solutions.

VERIQTA
BUILD. AUTOMATE. SCALE.

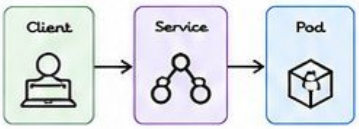
Healthy worker nodes keep your cluster reliable, stable, and high performing.

35. CLUSTER NETWORKING ISSUES

Networking problems can break communication between Pods, Services, and external users. This page covers the most common scenarios and fixes.

1. OVERVIEW

Kubernetes networking allows Pods, Services, and external clients to communicate.



KEY POINTS

- All Pods should reach each other.
- Services provide stable endpoints.
- DNS resolves Service and Pod names.
- Ingress exposes applications externally.
- Network issues affect availability and performance.

Always verify DNS, CNI, Services, and Ingress in this order.

2. DNS FAILURES

DNS issues prevent name resolution inside the cluster.

COMMON CAUSES

- CoreDNS Pods not running
- Wrong ConfigMap or plugin error
- Network policies blocking DNS
- DNS service IP unreachable

TROUBLESHOOTING STEPS

1. Check CoreDNS Pod status.
2. Check CoreDNS logs.
3. Test DNS lookup from a Pod.
4. Verify DNS Service IP.
5. Check NetworkPolicy rules.

USEFUL COMMANDS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system deployment/coredns
kubectl run dns-test --image=busybox -it --rm --sh
nslookup kubernetes.default.svc.cluster.local
dig @<dns-ip> kubernetes.default.svc.cluster.local
```

If DNS fails, name resolution for Services and Pods will also fail.

3. CNI FAILURES

CNI plugin issues break Pod to Pod communication.

COMMON CAUSES

- CNI plugin Pods not running
- IP address pool exhausted
- CNI configuration error
- Node network unreachable
- CNI crashed or restarted

TROUBLESHOOTING STEPS

1. Check CNI plugin Pod status.
2. Check CNI plugin logs.
3. Check node network status.
4. Verify IP address allocation.
5. Restart CNI plugin if needed.

USEFUL COMMANDS

```
kubectl get pods -n kube-system
kubectl logs -n kube-system -l k8s-app=calico-node
ip a
ip route
kubectl get nodes -o wide
kubectl describe node <node-name>
```

CNI issues usually affect all Pods on the affected node or cluster.

4. SERVICE CONNECTIVITY

Services provide stable access to applications. Connectivity issues affect availability.

COMMON CAUSES

- Wrong selector in Service
- Endpoints not available
- Service ClusterIP unreachable
- Network policies blocking traffic
- Port mismatch

TROUBLESHOOTING STEPS

1. Check Service and Endpoints.
2. Check Pod labels and selector.
3. Test from inside the cluster.
4. Verify NetworkPolicy rules.
5. Check target Pod readiness.

USEFUL COMMANDS

```
kubectl get svc
kubectl get endpoints <svc-name> -o wide
kubectl describe svc <svc-name>
kubectl run curl-test --image=curlimages/curl
    --rm -it --restart=Never -sh
curl http://<svc-name>:<port>/
```

Services work only if Endpoints and Pods are healthy.

5. INGRESS DEBUGGING

Ingress exposes applications outside the cluster. Issues can block external access.

COMMON CAUSES

- Ingress controller not running
- Incorrect Ingress rules
- DNS not pointing to Ingress IP
- TLS certificate issues
- Backend Service/Pod unhealthy

DEBUGGING STEPS

1. Check Ingress controller Pods.
2. Check Ingress resource.
3. Verify Ingress events.
4. Check DNS record.
5. Test external access.
6. Check backend Service and Pods.

USEFUL COMMANDS

```
kubectl get pods -n ingress-nginx
kubectl logs -n ingress-nginx deployment/ingress-nginx-controller
kubectl get ingress -A
kubectl describe ingress <ingress-name>
dig <domain-name>
curl -I http://<domain-name>
```

Ingress works only if DNS, TLS, Service, and Pods are all correct.

6. NETWORK POLICY BLOCKING

NetworkPolicies control traffic. Wrong rules can block communication.

COMMON CAUSES

- Default deny policy blocking traffic
- Missing egress rules
- Incorrect namespace selector
- Blocking DNS traffic

TROUBLESHOOTING STEPS

1. List all network policies.
2. Check applied policies on Pods.
3. Verify allowed ingress/egress rules.
4. Test connectivity after changes.

USEFUL COMMANDS

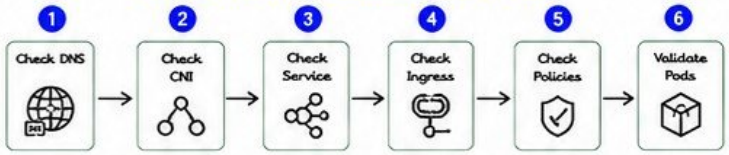
```
kubectl get networkpolicies -A
kubectl describe networkpolicy <policy-name> -n <ns>
kubectl get pods -n <ns> -o wide
kubectl exec -it <pod-name> -n <ns> -- curl http://<ip>:<port>
```

Always allow DNS and required traffic when using strict policies.

7. QUICK REFERENCE COMMANDS

PURPOSE	COMMAND
Check all Pods	kubectl get pods -A -o wide
Check all Services	kubectl get svc -A
Check Endpoints	kubectl get endpoints -A -o wide
Check CoreDNS	kubectl get pods -n kube-system -l k8s-app=kube-dns
Check CNI Pods	kubectl get pods -n kube-system
Check Ingress	kubectl get ingress -A
Check Network Policies	kubectl get networkpolicies -A
Test DNS	nslookup kubernetes.default.svc.cluster.local
Test Connectivity	kubectl run curl --image=curlimages/curl --rm -it --restart=Never -- sh
Check Node Network	kubectl describe node <node-name> ip a && ip route

8. NETWORK TROUBLESHOOTING FLOW



Follow the order. Fix each layer before moving to the next.

9. BEST PRACTICES

- ☑ Keep CoreDNS and CNI healthy.
- ☑ Monitor network performance and errors.
- ☑ Use NetworkPolicies carefully and test.
- ☑ Ensure DNS rules allow required traffic.
- ☑ Document network architecture and rules.
- ☑ Test connectivity during deployments.

REMEMBER

- ☑ No DNS = No Service discovery.
- ☑ No CNI = No Pod communication.
- ☑ No Service = No stable access.
- ☑ No Ingress = No external access.
- ☑ No Policies review = Hidden blocks.

VERIQTA

BUILD. AUTOMATE. SCALE.

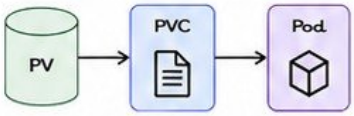
A healthy network means a healthy cluster.
Monitor, troubleshoot, and secure it always.

36. STORAGE FAILURES

Storage problems prevent Pods from starting or cause applications to fail.
This page covers the most common storage failures and how to fix them.

1. OVERVIEW

Kubernetes uses PersistentVolumes (PV) to store data for Pods.
When storage fails, Pods cannot start or data becomes unavailable.



KEY POINTS

- Storage is critical for stateful workloads.
- PVC binds to a suitable PV.
- Storage failures cause Pod startup or runtime issues.
- Always check events, PVC, PV, and CSI components.

Always follow the troubleshooting order on this page.

2. PVC PENDING

PVC is not bound to any PV.

COMMON CAUSES

- No matching PersistentVolume.
- StorageClass mismatch.
- Insufficient storage capacity.
- Access mode not supported.
- Incorrect PVC configuration.

TROUBLESHOOTING STEPS

1. kubectl get pvc
2. kubectl describe pvc <pvc-name>
3. Check StorageClass
4. kubectl get pv
5. Verify available storage
6. Check events

USEFUL COMMANDS

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get pv
kubectl get sc
kubectl describe sc <storageclass>
kubectl get events --sort-by=.lastTimestamp
```

PVC remains Pending until it binds to a matching PV.

3. MOUNT FAILURES

Pod starts but volume cannot be mounted.

COMMON CAUSES

- Incorrect mount path.
- Permission denied.
- NFS server unavailable.
- Volume not formatted.
- ReadOnly filesystem.

TROUBLESHOOTING STEPS

1. kubectl describe pod <pod-name>
2. Check events for mount errors
3. Verify volume and PV status
4. Check node logs
5. Verify path and permissions
6. Test manual mount on node

USEFUL COMMANDS

```
kubectl describe pod <pod-name>
kubectl get events --sort-by=.lastTimestamp
kubectl exec -it <pod> -- ls -l <mount-path>
kubectl logs <pod>
journalctl -u kubelet -xeu
```

Mount failures usually show permission or path issues.

4. VOLUME ATTACHMENT

PV exists but cannot attach to the node.

COMMON CAUSES

- Cloud disk limits reached.
- Volume in use by another node.
- AZ or zone mismatch.
- Disk attachment timeout.
- Node not ready.

TROUBLESHOOTING STEPS

1. kubectl describe pod <pod-name>
2. Check events for attach errors
3. Verify node status
4. Check cloud provider limits
5. Detach volume if stuck
6. Restart kubelet if needed

USEFUL COMMANDS

```
kubectl describe pod <pod-name>
kubectl get nodes
kubectl get pv
kubectl describe pv <pv-name>
kubectl get events --sort-by=.lastTimestamp
kubectl cordon <node> && kubectl drain <node>
```

Volumes stuck in "attached" state block new Pod scheduling.

5. CSI ISSUES

Container Storage Interface (CSI) driver fails to provision or manage volumes.

COMMON CAUSES

- CSI driver not running.
- CSI controller crash.
- Wrong driver configuration.
- Incompatible CSI version.
- Network issues to storage backend.

TROUBLESHOOTING STEPS

1. Check CSI pods in kube-system
2. Check CSI controller logs
3. Check events for CSI errors
4. Verify StorageClass configuration
5. Test CSI provisioning
6. Check network connectivity

USEFUL COMMANDS

```
kubectl get pods -n kube-system | grep csi
kubectl logs -n kube-system <csi-pod>
kubectl get csidrivers
kubectl describe csidriver <driver-name>
kubectl describe sc <storageclass>
kubectl get events --sort-by=.lastTimestamp
```

CSI issues affect provisioning, attachment, and volume operations.

6. EVENTS EXAMPLES

Events show the real reason behind storage failures.

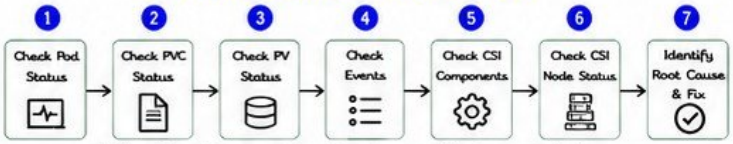
TYPE	EXAMPLE EVENT
PVC Pending	waiting for a volume to be created, either by external provisioner or manually created
Mount Failure	MountVolume.SetUp failed for volume "data" : permission denied
Attach Failure	AttachVolume.Attach failed for volume "disk-1" : timeout
CSI Error	rpc error: code = Internal desc = failed to provision volume with CSI driver

Always read events carefully. They point to the root cause.

7. QUICK REFERENCE COMMANDS

PURPOSE	COMMAND
Check PVCs	kubectl get pvc
Describe PVC	kubectl describe pvc <pvc-name>
Check PVs	kubectl get pv
Describe PV	kubectl describe pv <pv-name>
Check StorageClass	kubectl get sc
Check CSI Pods	kubectl get pods -n kube-system | grep csi
Check Events	kubectl get events --sort-by=.lastTimestamp
Describe Pod	kubectl describe pod <pod-name>
Check Nodes	kubectl get nodes
Drain Node (if needed)	kubectl drain <node> --ignore-daemonsets --delete-emptydir-data

8. STORAGE TROUBLESHOOTING FLOW



Follow this order every time. Do not guess. Check events first!

9. BEST PRACTICES

- ☑ Use the correct StorageClass and access mode.
- ☑ Ensure enough storage capacity is available.
- ☑ Monitor storage health and events.
- ☑ Use Redistribute/Replica Storage for critical workloads.
- ☑ Document storage architecture and limits.
- ☑ Test failover and recovery regularly.

REMEMBER

- ☑ No PVC = No storage.
- ☑ No events = No direction.
- ☑ No root cause = No fix.
- ☑ Check events, then act.
- ☑ Fix the cause, not the symptom.

VERIQTA
BUILD. AUTOMATE. SCALE.

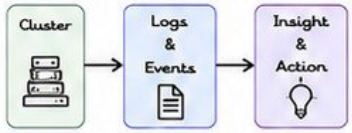
Reliable storage means reliable applications.
Troubleshoot smart, fix the root cause, and keep your cluster healthy.

37. LOGGING & EVENTS

Logs and events tell you what is happening inside your cluster.
This page covers logs, events, describe, and debug containers.

1. OVERVIEW

When something goes wrong, logs and events are your first source of truth. They help you understand what happened, when, and why.



KEY POINTS

- Logs show what the application or containers are doing.
- Events show what Kubernetes is doing.
- Always check events first for scheduling, mounting, and permission issues.
- Use logs to debug application behavior.
- Use describe for full object details.

Events show the story.
Logs show the details.

2. KUBECTL LOGS

View logs from containers in Pods.

COMMON USAGE

- View current logs
- Follow logs in real time
- Previous container logs
- Logs from all containers in a Pod
- Logs with timestamps

EXAMPLES

```
kubectl logs <pod-name>
kubectl logs -f <pod-name>
kubectl logs -f <pod-name> -c <container>
kubectl logs <pod-name> -c <container>
kubectl logs <pod-name> --previous
kubectl logs -l app=myapp --all-containers
kubectl logs <pod-name> --timestamps
```

USEFUL FLAGS

- -f : follow logs
- --previous : previous container logs
- -c : specific container
- -l : label selector
- --timestamps : include timestamps

Use -f to watch logs live during issues.

3. EVENTS

Events show what Kubernetes is doing inside the cluster.

COMMON USE CASES

- Pod scheduling issues
- Volume mounting issues
- Image pull failures
- Permission problems
- Node and resource issues

EXAMPLES

```
kubectl get events
kubectl get events --sort-by=.lastTimestamp
kubectl get events -A
kubectl get events -n <namespace>
kubectl describe events --sort-by=.lastTimestamp
```

FILTER EXAMPLES

```
kubectl get events --field-selector type=Warning
kubectl get events --field-selector reason=Failed
kubectl get events
--field-selector involvedObject.name=<pod-name>
```

Always sort events by time to see the most recent issues first.

4. DESCRIBE

Get detailed information about any Kubernetes object.

WHAT YOU GET

- Configuration
- Status
- Events
- Conditions
- Warnings
- Resource usage

EXAMPLES

```
kubectl describe pod <pod-name>
kubectl describe pod <pod-name> -n <ns>
kubectl describe node <node-name>
kubectl describe pvc <pvc-name>
kubectl describe deployment <deploy-name>
kubectl describe svc <service-name>
```

WHY IT'S USEFUL

- Shows events related to the object
- Shows why Pods are Pending
- Shows mount and permission errors
- Shows image pull errors

Describe is your first deep debugging tool.

5. DEBUG CONTAINERS

Debug running containers without modifying the image.

COMMON METHODS

- kubectl exec
- Ephemeral debug containers
- Interactive troubleshooting

EXAMPLES

```
kubectl exec -it <pod-name> -- /bin/sh
kubectl exec -it <pod-name> -- /bin/bash
kubectl exec -it <pod-name> -c <container> -- /bin/sh
```

EPHEMERAL DEBUG CONTAINER

```
kubectl debug -it <pod-name> --image=busybox --target=<container-name> -- /bin/sh
```

Ephemeral containers help debug without changing the running Pod.

6. LOGGING BEST PRACTICES

BEST PRACTICES

- ☑ Log important events and errors.
- ☑ Use structured logs (JSON) when possible.
- ☑ Include timestamps in logs.
- ☑ Do not log sensitive information.
- ☑ Use log aggregation for production.
- ☑ Monitor logs continuously.
- ☑ Set log retention policies.
- ☑ Correlate logs with metrics and events.

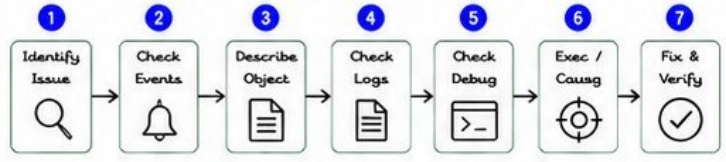
Good logging reduces MTTR and improves reliability.

7. COMMON LOG & EVENT SCENARIOS

SCENARIO	CHECK	COMMAND
Pod is Pending	Events	kubectl describe pod <pod-name>
Image pull failure	Events + Logs	kubectl describe pod <pod-name> kubectl logs <pod-name>
CrashLoopBackOff	Logs + Events	kubectl logs -f <pod-name> kubectl describe pod <pod-name>
Mount failure	Events	kubectl describe pod <pod-name>
Permission denied	Logs + Describe	kubectl logs <pod-name> kubectl describe pod <pod-name>
Service not reachable	Describe + Logs	kubectl describe svc <svc-name> kubectl logs <pod-name>

Events explain the "why". Logs show the "what".

8. LOGGING & EVENTS TROUBLESHOOTING FLOW



Follow the order. Do not guess. Check events and logs first.

9. QUICK REFERENCE COMMANDS

PURPOSE	COMMAND
View Pod logs	kubectl logs <pod-name>
Follow Pod logs	kubectl logs -f <pod-name>
Logs from container	kubectl logs <pod-name> -c <container>
Previous logs	kubectl logs <pod-name> --previous
All events	kubectl get events --sort-by=.lastTimestamp
Describe Pod	kubectl describe pod <pod-name>
Exec into Pod	kubectl exec -it <pod-name> -- /bin/bash
Ephemeral debug	kubectl debug -it <pod-name> --image=busybox --target=<container> -- /bin/sh

REMEMBER

- ☑ Check events before logs.
- ☑ Describe for full context.
- ☑ Logs show the runtime details.
- ☑ Use debug containers when needed.
- ☑ Good logs save hours of debugging.

VERIQTA
BUILD. AUTOMATE. SCALE.

Great logging and events lead to fast debugging, stable applications, and happy users.

38. RESOURCE PRESSURE

When the cluster or nodes run out of resources, Pods get throttled, evicted, or stuck.
This page covers CPU, Memory, Disk pressure, and Evictions.

1. OVERVIEW

Nodes have limited resources. When resources are overused, Kubernetes protects the node and the cluster.

KEY POINTS

- Resource pressure affects stability.
- Kubernetes tries to throttle first, then evict Pods.
- Always monitor usage and set proper requests and limits.
- Evictions can impact applications.
- Disk pressure can stop scheduling.

Monitor, alert, and plan capacity. Don't wait for the node to fail.

2. CPU PRESSURE

High CPU usage causes throttling and impacts performance.

SIGNS

- High CPU usage on node
- CPU throttling of Pods
- High load average
- Application slowness
- Increased response time

TROUBLESHOOTING STEPS

1. Check node CPU usage.
2. Check top CPU consuming Pods.
3. Check CPU limits and requests.
4. Look for CPU throttling.
5. Optimize or scale workloads.
6. Consider adding more nodes.

USEFUL COMMANDS

```
kubectl top nodes
kubectl top pods -A
kubectl describe node <node-name>
kubectl describe pod <pod-name>
kubectl get pods -A --sort-by=.spec.containers[0].resources.requests.cpu
kubectl get pods -A --sort-by=.spec.containers[0].resources.limits.cpu
```

High CPU = Throttling = Slow applications.

3. MEMORY PRESSURE

High memory usage triggers evictions or OOM kills.

SIGNS

- High memory usage on node
- OOMKilled Pods
- Pod restarts
- Evictions
- System instability

TROUBLESHOOTING STEPS

1. Check node memory usage.
2. Check memory usage of Pods.
3. Look for OOMKilled events.
4. Review memory requests/limits.
5. Optimize application memory.
6. Scale or add more memory.

USEFUL COMMANDS

```
kubectl top nodes
kubectl top pods -A
kubectl describe node <node-name>
kubectl describe pod <pod-name>
kubectl get events -A --sort-by=.lastTimestamp | grep -i oom
```

Set limits. OOMKilled Pods waste resources and cause outages.

4. DISK PRESSURE

Low disk space can stop Pods from starting or cause evictions.

SIGNS

- Node Condition: DiskPressure
- Pods Pending
- Image pulls fail
- Write errors
- Evictions

TROUBLESHOOTING STEPS

1. Check disk usage on nodes.
2. Identify large files/containers.
3. Clean up unused data.
4. Check container log size.
5. Check filesystem inodes.
6. Increase disk capacity if needed.

USEFUL COMMANDS

```
kubectl describe node <node-name>
df -h
du -sh /var/lib/docker/*
kubectl get pods -A
kubectl get events -A --sort-by=.lastTimestamp | grep -i disk
```

Disk full = New Pods fail. Always keep space free.

5. EVICTIONS

Kubernetes evicts Pods to protect the node and keep system stable.

COMMON REASONS

- Memory pressure
- Disk pressure
- PID pressure
- Node pressure
- System critical Pods need resources

TROUBLESHOOTING STEPS

1. Check node conditions.
2. Check recent events.
3. Identify evicted Pods.
4. Check resource usage.
5. Fix the root cause.
6. Prevent with limits and requests.

USEFUL COMMANDS

```
kubectl get nodes
kubectl describe node <node-name>
kubectl get events -A --sort-by=.lastTimestamp | grep -i evict
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
```

Evictions protect the node, but impact your applications.

6. EVENTS EXAMPLES

Events tell you what Kubernetes is doing. Always check events first.

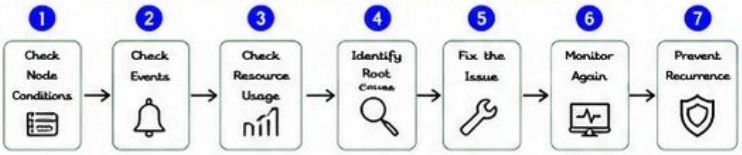
TYPE	REASON	MESSAGE EXAMPLE
Warning	NodePressure	Node memory pressure
Warning	DiskPressure	Node disk pressure
Warning	Evicted	The node was low on resource: memory. Container <name> was evicted
Warning	OOMKilled	Container killed due to out-of-memory
Warning	FailedScheduling	0/3 nodes are available: insufficient memory
Normal	Killing	Stopping container <name>
Normal	Pulled	Successfully pulled image <image>

Events are your first clue. Always check them before digging deeper.

7. QUICK REFERENCE COMMANDS

PURPOSE	COMMAND
Check node conditions	kubectl get nodes
Describe node	kubectl describe node <node-name>
Check resource usage (nodes)	kubectl top nodes
Check resource usage (pods)	kubectl top pods -A
Check events (all namespaces)	kubectl get events -A --sort-by=.lastTimestamp
Check events (filter evictions)	kubectl get events -A --sort-by=.lastTimestamp | grep -i evict
Describe pod	kubectl describe pod <pod-name>
Previous logs (evicted/OOMKilled)	kubectl logs <pod-name> --previous
Check disk usage on node	df -h
Check inodes	df -i

8. RESOURCE PRESSURE TROUBLESHOOTING FLOW



Follow the order. Don't skip steps. Verify after every fix.

9. BEST PRACTICES

- ☑ Set resource requests and limits for all containers.
- ☑ Monitor CPU, memory, and disk continuously.
- ☑ Keep at least 20% free resources on every node.
- ☑ Use Horizontal Pod Autoscaler for dynamic scaling.
- ☑ Keep logs under control. Rotate and archive.
- ☑ Clean up unused images and containers.
- ☑ Alert before pressure becomes critical.
- ☑ Test your cluster under real load.

REMEMBER

- ☑ Pressure → Throttle → Evict.
- ☑ Evictions = Application Impact.
- ☑ Monitor Early. Fix Early.
- ☑ Requests & Limits Prevent Problems.
- ☑ Healthy Resources = Healthy Cluster.

VERIQTA
BUILD. AUTOMATE. SCALE.

Manage resources wisely.
Stable resources.
Stable applications.
Stable business.

39. FAST KUBECTL TECHNIQUES

Speed comes from repetition and the right shortcuts.
These techniques help you work faster and debug smarter.

1. ALIASES

Create short aliases for the commands you use every day.

ALIAS	EXPANDS TO
k	kubectl
kgp	kubectl get pods
kgn	kubectl get nodes
kgs	kubectl get svc
kga	kubectl get all
kd	kubectl describe
kl	kubectl logs -f
kctx	kubectl config use-context
kns	kubectl config set-context --current --namespace
ka	kubectl apply -f

HOW TO SET ALIASES

1. Open ~/.bashrc or ~/.zshrc
2. Add aliases:
   alias k=kubectl
   alias kgp='kubectl get pods'
   alias kga='kubectl get all'
3. Reload: source ~/.bashrc

Use aliases that make sense to YOU.
Keep them simple and consistent.

2. AUTOCOMPLETE

Autocomplete saves time and prevents typing errors.

BASH (LINUX / MAC)

1. Install bash-completion (if needed)

```
sudo apt install bash-completion
# or
brew install bash-completion
```

2. Enable kubectl autocomplete

```
source <(kubectl completion bash)
echo "source <(kubectl completion bash)"
>> ~/.bashrc
```

3. Reload your shell

```
source ~/.bashrc
```

ZSH

1. Enable in oh-my-zsh

```
echo 'plugins=(... kubectl)' >> ~/.zshrc
```

2. Or enable manually

```
source <(kubectl completion zsh)
echo "source <(kubectl completion zsh)"
>> ~/.zshrc
```

3. Reload

```
source ~/.zshrc
```

Now press TAB to autocomplete resources, namespaces, options and more.

3. JSONPATH

Extract exactly what you need from Kubernetes objects.

USE CASE	EXAMPLE
Get all Pod names	kubectl get pods -o jsonpath ='{.items[*].metadata.name}'
Get Pod IPs	kubectl get pods -o jsonpath ='{.items[*] .status.podIP}'
Get Node names	kubectl get nodes -o jsonpath ='{.items[*].metadata.name}'
Get Node CPU capacity	kubectl get nodes -o jsonpath ='{.items[*].status.capacity.cpu}'
Get first Container image	kubectl get pods -o jsonpath ='{.items[0].spec.containers[0].image}'
Get Service ClusterIP	kubectl get svc my-svc -o jsonpath ='{.spec.clusterIP}'
Get all Namespaces	kubectl get ns -o jsonpath ='{.items[*].metadata.name}'

JSONPATH TIPS

- Use quotes around the expression.
- Use [.items[*]] to loop over all items.
- Combine with | tr ' ' '\n' for line output.
- See more: kubectl get pods -o json | jq .

JSONPath helps you script, automate, and debug faster.

4. IMPERATIVE COMMANDS

Make quick changes without editing YAML.
Great for testing and production fixes.

ACTION	EXAMPLE COMMAND
Create a Pod	kubectl run nginx --image=nginx
Create a Deployment	kubectl create deploy myapp --image=nginx
Expose a Deployment	kubectl expose deploy myapp --port=80 --type=NodePort
Scale a Deployment	kubectl scale deploy myapp --replicas=5
Set Image	kubectl set image deploy/myapp nginx=nginx:1.25
Rolling Restart	kubectl rollout restart deploy/myapp
Edit Resource	kubectl edit deploy/myapp
Label Resource	kubectl label pod mypod env=prod
Annotate Resource	kubectl annotate pod mypod team=devops
Delete Resource	kubectl delete pod mypod

Imperative commands = fast, direct and powerful.
Perfect for one-time or urgent tasks.

5. TIME SAVERS

Small tricks that save minutes every day.

- ☑ Use k alias for kubectl.
- ☑ Use -n <namespace> instead of --namespace.
- ☑ Use -o wide to see more info.
- ☑ Use --watch to monitor changes.
- ☑ Use -l <label-selector> to filter.
- ☑ Use --field-selector for server-side filtering.
- ☑ Use | less or | grep to narrow output.
- ☑ Use --dry-run=client -o yaml to preview.
- ☑ Use !! to repeat the last command.
- ☑ Use history | grep kubectl to find old cmds.
- ☑ Use kga to see all resources at once.
- ☑ Use kdp <name> to describe fast.

Speed is a habit.
Build shortcuts that match your workflow.

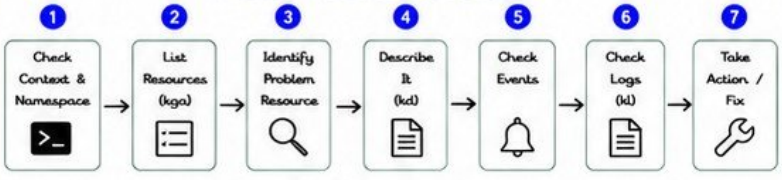
6. POWERFUL COMBINATIONS

Put multiple techniques together.

GOAL	COMMAND EXAMPLE
List all pods with IPs	kubectl get pods -o wide
Monitor pod restarts	kubectl get pods -w
Get failed pods	kubectl get pods --field-selector status.phase=Failed
Top nodes	kubectl top nodes
Top pods	kubectl top pods
Get pod events	kubectl get events --field-selector involvedObject.name=mypod
Filter logs by keyword	kubectl logs -f mypod | grep ERROR
Get pod CPU usage	kubectl top pod mypod
Describe + logs shortcut	kubectl describe pod mypod ; kubectl logs -f mypod
Get pods not ready	kubectl get pods --field-selector status.phase!=Running

Combinations turn kubectl into a superpower.
Use them in debugging and daily operations.

7. FAST KUBECTL WORKFLOW



Follow this flow every time.
It saves time and prevents guesswork.

8. MUST-REMEMBER COMMANDS

PURPOSE	COMMAND
See everything	kubectl get all -A
Watch changes	kubectl get pods -w -A
Describe resource	kubectl describe <resource> <name>
View logs	kubectl logs <pod-name> -f
Get events	kubectl get events --sort-by=.lastTimestamp
Exec into pod	kubectl exec -it <pod-name> -- /bin/bash
Copy file from pod	kubectl cp <pod>:<path> ./
Port forward	kubectl port-forward <pod> 8080:80
Delete everything in namespace	kubectl delete all --all -n <namespace>

REMEMBER

- ☑ Shortcuts build speed.
- ☑ Speed builds confidence.
- ☑ Confidence builds impact.
- ☑ Practice daily.
- ☑ Make kubectl your strength.

VERIQTA
BUILD. AUTOMATE. SCALE.

Small shortcuts today.
Big efficiency tomorrow.
Work smart.
Ship faster.

40. YAML EDITING STRATEGIES

Good YAML saves time, prevents errors, and makes production changes safe.
This page shows fast ways to create, edit, reuse, and manage manifests.

1. FAST MANIFEST GENERATION

Generate a manifest, then edit it.

GOAL	COMMAND
Create Deployment	kubectl create deployment myapp --image=nginx --dry-run=client -o yaml
Create Service	kubectl expose deployment myapp --port=80 --target-port=80 --dry-run=client -o yaml
Create ConfigMap	kubectl create configmap app-config --from-literal=key=value --dry-run=client -o yaml
Create Secret (generic)	kubectl create secret generic mysecret --from-literal=password=pass123 --dry-run=client -o yaml
Create Ingress	kubectl create ingress my-ingress --rule=app.example.com/=/ --dry-run=client -o yaml
Generate from Existing Object	kubectl get deployment myapp -o yaml > myapp.yaml

Always use --dry-run=client -o yaml to generate clean manifests. Then edit before applying.

2. COPY-EDIT WORKFLOW

A safe and fast way to reuse existing manifests.

1. Get the existing manifest

```
kubectl get deployment myapp -o yaml
> myapp-copy.yaml
```

2. Edit the file

```
vi myapp-copy.yaml
```

3. Change what you need

- image tag
- replicas
- resources
- environment variables
- labels / annotations

4. Apply the changes

```
kubectl apply -f myapp-copy.yaml
```

5. Verify the update

```
kubectl rollout status deployment/myapp
```

Copy → Edit → Apply → Verify.
Simple, safe, repeatable.

3. VI SHORTCUTS FOR YAML

Edit faster with these essential vi keys.

ACTION	VI COMMAND
Open file	vi filename.yaml
Insert mode	i
Append after cursor	a
Append at end of line	A
Delete line	dd
Copy line	yy
Paste below	p
Undo	u
Save and exit	:wq
Quit without saving	:q!
Search text	/text
Go to line number	:123
Visual select	v (select) then y (copy)
Indent line	>>
Outdent line	<<

Master these and you can edit any manifest in seconds.

4. COMMON TEMPLATES

Reusable templates for everyday resources.

DEPLOYMENT (MINIMAL)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
        image: nginx:1.25
        ports:
          - containerPort: 80
```

SERVICE (CLUSTERIP)

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-svc
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
    - name: http
      port: 80
      targetPort: 80
```

CONFIGMAP (LITERAL)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "prod"
  LOG_LEVEL: "info"
```

Keep templates in a /templates folder and reuse them.

5. YAML BEST PRACTICES

- ☑ Use 2 spaces for indentation. Never use tabs.
- ☑ Validate before apply.

```
kubectl apply -f file.yaml --dry-run=client
```

- ☑ Use meaningful names and labels.
- ☑ Keep manifests small and focused.
- ☑ Use comments to explain non-obvious settings.
- ☑ Store in Git and version your changes.
- ☑ Use kustomize or Helm for environments when needed.
- ☑ Test changes in dev before prod.

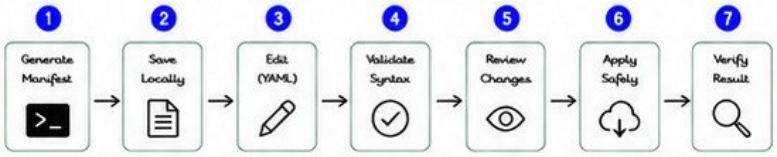
Clean YAML today prevents outages tomorrow.

6. USEFUL EDITING PATTERNS

PATTERN	EXAMPLE
Change image tag	image: nginx:1.25 → nginx:1.26
Change replicas	replicas: 2 → 5
Add environment var	env: - name: ENV value: "prod"
Add resource limits	resources: limits: cpu: "500m" memory: "512Mi"
Add node selector	nodeSelector: disktype: ssd
Change container port	containerPort: 80 → 8080
Add labels	labels: team: platform env: production
Add annotation	annotations: reloader.stakater.com/auto: "true"

Small changes. Big impact. Edit with intent.

7. YAML EDITING WORKFLOW



Generate → Edit → Validate → Review → Apply → Verify.
Follow this every time.

8. MUST-REMEMBER COMMANDS

PURPOSE	COMMAND
Validate YAML syntax	kubectl apply -f file.yaml --dry-run=client
Apply manifest	kubectl apply -f file.yaml
View applied object	kubectl get all
Describe object	kubectl describe <type> <name>
Diff changes (server-side)	kubectl diff -f file.yaml
Delete resource	kubectl delete -f file.yaml
View last applied config	kubectl get <type> <name> -o yaml
Explain field	kubectl explain <resource>.<field>

- ☑ Generate clean manifests.
- ☑ Edit with confidence.
- ☑ Validate before applying.
- ☑ Version everything.
- ☑ Small, safe changes → stable clusters.

VERIQTA
BUILD. AUTOMATE. SCALE.

Good YAML is not luck.
It is a habit.
Edit smart. Deploy safe.
Operate like a pro.

41. EXAM TIME MANAGEMENT

Time is your most limited resource in the exam.
Plan it. Protect it. Use it smart.

1. QUESTION PRIORITIZATION

Tackle questions in the best order.

PRIORITY	TYPE	EXAMPLES
P1	Easy Quick Wins	Direct recall, commands, definitions, concepts
P2	Medium Built Questions	Troubleshooting steps, YAML edits, diagrams
P3	Hard Complex	Multi-step scenarios, performance, design
P4	Park & Return	Unsure, time heavy, low confidence

STRATEGY

- Do P1 questions first (bank easy marks).
- Move to P2 when comfortable.
- Attempt P3 with remaining time.
- Park P4 and come back if time allows.
- Never get stuck on one question.

Bank points early. Momentum builds confidence and speed.

2. TIME BUDGETING

Plan your time before you start.

EXAM OVERVIEW

- Total Time: 180 minutes
- Questions: 60
- Average Time per Question: 3 minutes

SUGGESTED TIME SPLIT

TIME RULES

- Do not spend > 4 minutes on any question.
- Mark and move if stuck.
- Keep last 15 minutes for review.
- Watch the clock at each 30 min mark.

Time that is not managed, will manage you.

3. RECOVERY STRATEGY

When you get stuck or fall behind.

IF YOU GET STUCK

- Stop. Take a deep breath.
- Read the question again carefully.
- Eliminate wrong options.
- Think: What is Kubernetes trying to achieve?
- Take an educated guess and move.

IF YOU FALL BEHIND

- Skip current question.
- Focus on easier questions.
- Increase pace, but stay accurate.
- Do not panic - you can recover.
- Protect the review time.

MINDSET REMINDERS

- ☑ Calm mind > perfect knowledge
- ☑ Progress > perfection
- ☑ Move forward, then refine
- ☑ You can always come back

Do not let one hard question steal time from many easy ones.

4. VERIFICATION CHECKLIST

Review smart, not twice.

BEFORE YOU SUBMIT

- ☐ Read every question.
- ☐ Attempted all questions (no blank).
- ☐ YAML is valid and indented.
- ☐ Commands are correct.
- ☐ Names, labels, namespaces correct.
- ☐ Resource types spelled correctly.
- ☐ Ports, paths, and values correct.
- ☐ Concepts match Kubernetes behavior.
- ☐ Diagram labels and flow correct.
- ☐ Time remaining ≥ 1 minute.

FINAL 5 MINUTE REVIEW

- ☐ Recheck flagged questions.
- ☐ Scan for silly mistakes.
- ☐ Ensure answers are complete.
- ☐ Submit with confidence.

A 5 minute review can save 5+ marks.

5. TIME ALLOCATION BY QUESTION TYPE

Use this as a guideline.

QUESTION TYPE	TIME PER Q	EXAMPLES
Recall / Concept	1 - 2 min	Definitions, features, components
Commands	2 - 3 min	kubectl, logs, exec, get, describe
YAML Editing	3 - 4 min	Fix indentation, labels, resources
Troubleshooting	3 - 5 min	Logs, events, causes, solutions
Scenarios / Design	4 - 6 min	Architecture, scaling, security
Performance	4 - 6 min	Tuning, limits, metrics

Adjust based on question difficulty.
Do not overspend time.

6. EXAM FLOW STRATEGY

Follow this proven approach.

Quick Scan (2-3 min)	Read all questions. Understand overall difficulty. Mark easy ones.
Answer P1 Questions (60 min)	Complete all easy questions quickly and accurately.
Answer P2 Questions (70 min)	Focus on medium questions. Stay consistent.
Attempt P3 Questions (35 min)	Use logic, eliminate wrong options. Do your best.
Return to P4 Questions (10 min)	Revisit parked questions if time allows.
Review & Submit (15 min)	Use the verification checklist. Submit confidently.

A plan followed is better than perfect knowledge without a plan.

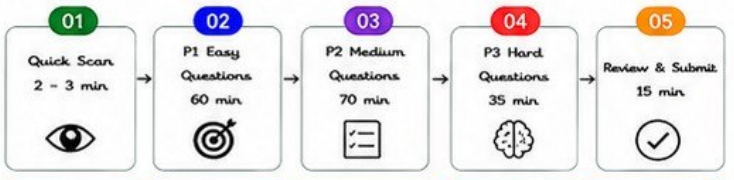
7. COMMON TIME KILLERS

Avoid these mistakes.

TIME KILLER	BETTER ALTERNATIVE
Reading question carelessly	Read slowly and understand what is asked.
Spending too long on one question	Move on. Return later.
Overthinking	Use elimination. Make a choice.
Not checking time	Check every 30 minutes.
Editing answers repeatedly	Edit once. Move forward.
Skipping review	Reserve time to review.
Leaving blank answers	Make an educated guess.

Avoid time traps. Protect your focus and rhythm.

8. SAMPLE TIME PLAN (180 MINUTES)



Stay disciplined. Stick to the plan.

9. SUCCESS MINDSET

- ☑ Prepare well before the exam.
- ☑ Sleep well the night before.
- ☑ Eat light and stay hydrated.
- ☑ Arrive early and stay calm.
- ☑ Read instructions carefully.
- ☑ Trust your preparation.
- ☑ Stay positive till the end.
- ☑ Review, verify, and submit with confidence.

Calm mind.
Smart plan.
Maximum score.

REMEMBER

- ☑ Plan your time.
- ☑ Protect your time.
- ☑ Use your time wisely.
- ☑ Recover when needed.
- ☑ Finish strong and review.

VERIQTA
BUILD. AUTOMATE. SCALE.

Your time management today is your success tomorrow.
Plan it. Protect it. Perform.
You've got this!

42. COMMON EXAM MISTAKES

Small mistakes cost big marks.
Avoid these to save time and score higher.

1. NAMESPACE ERRORS

Working in the wrong namespace is a top reason for failure.

MISTAKE	EXAMPLE
Using wrong namespace	Commands run in default instead of app namespace
Forgetting -n flag	kubectl get pods (no -n specified)
Creating in wrong namespace	Resources created in default unintentionally
Not switching namespace	Still in previous namespace context

AVOID IT

- ☑ Always check: kubectl config get-context --current
  kubectl get ns
- ☑ Use -n <namespace> or set context.
- ☑ Read the question carefully.

Wrong namespace = Wrong answer.

2. WRONG CONTEXT

Wrong context means wrong cluster, wrong user, wrong namespace.

MISTAKE	EXAMPLE
Using wrong cluster	kubectl config use-context prod instead of exam
Wrong user/identity	RBAC permissions don't match the context
Not checking context	Assuming context is already correct
Changing context accidentally	Switching context and forgetting to switch back

AVOID IT

- ☑ Check current context at the start.
  kubectl config current-context
- ☑ Verify cluster, user, and namespace.
- ☑ Don't change context unless needed.

Right context = Right results.

3. RBAC MISTAKES

RBAC questions test details.
Small errors break access.

MISTAKE	EXAMPLE
Wrong apiGroups	Using v1 instead of apps or rbac.authorization.k8s.io
Wrong resource names	pods instead of pod
Wrong verbs	Using get instead of list/watch
Binding to wrong subject	User instead of ServiceAccount
Wrong namespace for Role/RoleBinding	Creating Role in default instead of target ns
ClusterRole instead of Role	Scope does not match requirements

AVOID IT

- ☑ Check apiGroups, resources, and verbs.
- ☑ Match the subject exactly.
- ☑ Check scope: Role vs ClusterRole.
- ☑ Bind in the correct namespace.

RBAC is all about exact match.

4. SERVICE SELECTOR MISTAKES

Services find Pods using selectors.
Wrong selector = No endpoints.

MISTAKE	EXAMPLE
Wrong label key	selector: app: web but pods have tier: web
Wrong label value	selector: app: api but pods have app: API
Mixing labels	Using different labels than in Pod template
Missing selector	Service created without selector (ClusterIP)
Editing selector after creation	Changing selector doesn't update existing endpoints

AVOID IT

- ☑ Check Pod labels first: kubectl get pods --show-labels
- ☑ Ensure Service selector matches exactly.
- ☑ Labels are case-sensitive.

Correct selector = Traffic flows.
Wrong selector = No endpoints.

5. LABEL MISMATCHES

Labels drive everything in Kubernetes.
Small typos break resources.

MISTAKE	EXAMPLE
Typos in keys	app: weeb (extra e) instead of app: web
Typos in values	env: prodc instead of env: prod
Case sensitivity	App: Web instead of app: web
Inconsistent labels	Deployment uses app: api Service uses app: API
Missing required labels	Pod missing label expected by Service/Selector
Not updating labels	Service/NetworkPolicy still uses old label

AVOID IT

- ☑ Copy labels exactly from the question.
- ☑ Use kubectl get pods --show-labels to verify.
- ☑ Keep labels consistent across all resources.

Labels must match exactly.
One small mismatch breaks everything.

6. BONUS: OTHER COMMON MISTAKES

- Forgetting resource quotas/limits
  Solution may be blocked by limits.
- Not reading all requirements
  Missing one requirement = Wrong answer.
- Wrong resource type
  Using Deployment instead of DaemonSet, or Service type mismatch.
- Missing flags/options
  Forget -n, --dry-run, or selectors.
- Not verifying results
  Not checking get/describe outputs.
- Incorrect file names/paths
  Saving YAML to wrong file name.

Slow down, read carefully, and verify your work.

7. ERROR TO OUTCOME: WHAT GOES WRONG?

MISTAKE	WHAT HAPPENS	HOW TO FIX	VERIFY
Wrong Namespace	Resource not found No pods/endpoints	Use correct -n flag or switch context	kubectl get pods -n <ns> kubectl get all -n <ns>
Wrong Context	No permission Wrong cluster	Use correct context and verify user	kubectl config current-context
RBAC Mistakes	Access denied Forbidden errors	Fix Role/RoleBinding and verbs	kubectl auth can-i --list kubectl get <resource>
Service Selector	No endpoints Service unreachable	Fix selector to match Pod labels	kubectl get endpoints <svc> kubectl describe svc <svc>
Label Mismatches	Resources not linked Policies not applied	Align labels across resources	kubectl get pods --show-labels kubectl describe <resource>

8. EXAM MISTAKE PREVENTION CHECKLIST

- ☐ Read the question carefully (all parts)
- ☐ Identify namespace to use
- ☐ Check current context
- ☐ Verify resource type and scope
- ☐ Check required labels/selectors
- ☐ Verify RBAC requirements
- ☐ Plan before typing commands
- ☐ Test and verify your solution
- ☐ Double-check file names and flags
- ☐ Review final state before moving on

REMEMBER

- ☑ Read carefully. Understand fully.
- ☑ Check context and namespace.
- ☑ Match labels and selectors exactly.
- ☑ Verify RBAC with correct verbs.
- ☑ Test before final submission.
- ☑ Small mistakes = Big mark loss.

VERIQTA
BUILD. AUTOMATE. SCALE.

Attention to detail is a superpower in this exam.
Avoid mistakes.
Score big. Succeed.

43. MOCK EXAM 1

Realistic CKA-style questions. 60 minutes. 20 Questions.
Choose the best answer for each question.

EXAM INFO

Total Questions	:	20
Time Limit	:	60 Minutes
Passing Score	:	70% (14/20)
Exam Type	:	CKA Practice

1. NAMESPACE

Which command creates a namespace named devops?

A. kubectl create ns devops
B. kubectl create namespace devops
C. kubectl ns create devops
D. kubectl new namespace devops

2. CONTEXT

Which command switches to the context named prod?

A. kubectl switch prod
B. kubectl config use prod
C. kubectl config use-context prod
D. kubectl set-context prod

3. PODS

Which command lists Pods in namespace monitoring?

A. kubectl get pods -n monitoring
B. kubectl get po monitoring
C. kubectl pods --namespace monitoring
D. kubectl list pods -n monitoring

4. DEPLOYMENT

Which command creates a deployment named web with image nginx?

A. kubectl run web --image=nginx
B. kubectl create deploy web --image=nginx
C. kubectl create deployment web nginx
D. kubectl deploy web --image nginx

5. SERVICE

Which type of Service exposes a pod externally using a cloud provider LoadBalancer?

A. ClusterIP
B. NodePort
C. LoadBalancer
D. ExternalName

6. YAML

Which field defines the number of Pod replicas in a Deployment?

A. spec.count
B. spec.replicas
C. spec.podCount
D. spec.instances

7. PVC

A PVC is stuck in Pending state. What is the most likely reason?

A. Pod is not running
B. StorageClass does not exist
C. Node has no kubelet
D. Service selector mismatch

8. RBAC

Which resource is used to grant permissions in a namespace?

A. ClusterRoleBinding
B. RoleBinding
C. ServiceAccount
D. ConfigMap

9. NETWORKING

Which component usually provides cluster DNS?

A. CoreDNS
B. kube-proxy
C. kube-scheduler
D. Flannel

10. POD LOGS

Which command shows logs of pod web-7d4f9 in namespace app?

A. kubectl logs web-7d4f9 -n app
B. kubectl get logs web-7d4f9 -n app
C. kubectl log web-7d4f9 app
D. kubectl show logs web-7d4f9

11. NODE

Which command drains a node named worker1?

A. kubectl drain worker1
2. kubectl cordon worker1
C. kubectl drain node worker1 --ignore-daemonsets
D. kubectl evict node worker1

12. CONFIGMAP

Which command creates a ConfigMap from a file named app.conf?

A. kubectl create configmap app --file=app.conf
B. kubectl configmap create app app.conf
C. kubectl create cm app --from-file=app.conf
D. kubectl create configmap app app.conf

13. SECRETS

How do you create a secret named mysecret from a literal value?

A. kubectl create secret mysecret value=pass
B. kubectl create secret generic mysecret --from-literal=password=pass
C. kubectl secret create mysecret --value=pass
D. kubectl create secret mysecret --from-pass=pass

14. ROLLOUT

Which command checks the rollout status of deployment web?

A. kubectl rollout status deploy/web
B. kubectl rollout check web
C. kubectl status rollout web
D. kubectl rollout history web

15. KUBECTL APPLY

What does this command do?
kubectl apply -f app.yaml

A. Deletes the resources
B. Creates or updates resources
C. Only updates existing resources
D. Validates the YAML only

16. POD FAILURE

A Pod is in CrashLoopBackOff. What should you check first?

A. Service configuration
B. Pod logs
C. Node taints
D. NetworkPolicy

17. PRACTICAL LAB (5 POINTS)

You are given the following file: nginx-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.25
```

1. Create tthe fusing the hasks:
2. Expose the Pod using a Service named nginx-svc of type NodePort, -port 80.
3. Verify that the Service is created.
4. Delete the Pod.
5. Delete the Service.

(1 point each)

18. DEBUGGING SCENARIO (3 POINTS)

A Deployment named api has 3 replicas. Users report intermittent 503 errors.

Answer the following:

1. List 3 possible causes.
2. Commands you would run to investigate.
3. What would you check in the Pod logs?

(1 point each)

19. YAML EXAM QUESTION (3 POINTS)

What will be the result of this YAML?

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
  type: ClusterIP
```

A. Exposes Pods with label app=web on port 80 inside the cluster.
B. Exposes Pods with label app=web on port 8080 externally.
C. Creates a Deployment.
D. Creates a NodePort Service.

20. CONCEPTUAL QUESTION (2 POINTS)

Explain the difference between NodePort and LoadBalancer Services. When would you use each?

ANSWER SHEET

Mark your answers in the boxes below.

1	2	3	4	5	6	7	8	9	10
A	B	B	A	A	A	A	B	C	D

11	12	13	14	15	16	17	18	19	20
A	B	C	B	A	C	C	C	C	D

EXAM TIPS

- ☑ Read each question carefully.
- ☑ Manage your time wisely.
- ☑ Use kubectl explain and --help in real exams.
- ☑ Verify your work before moving on.
- ☑ Stay calm and focused.

44. MOCK EXAM 2

Realistic CKA-style questions. 60 minutes. 20 Questions.

Choose the best answer for each question.

EXAM INFO

Total Questions	:	20
Time Limit	:	60 Minutes
Passing Score	:	70% (14/20)
Exam Type	:	CKA Practice

1. NAMESPACE

Which command lists all namespaces?

A. kubectl get all -A
B. kubectl get namespaces
C. kubectl get ns
D. kubectl list ns

2. CONTEXT

Which command shows the current context?

A. kubectl config view
B. kubectl config current-context
C. kubectl config get-contexts
D. kubectl context current

3. PODS

Which command shows all pods in all namespaces?

A. kubectl get pods --all-namespaces
B. kubectl get pods -A
C. kubectl get pods --namespace all
D. kubectl get all --all-namespaces

4. DEPLOYMENT

Which command scales a deployment named api to 5 replicas?

A. kubectl scale deployment api --replicas=5
B. kubectl set replicas deployment api 5
C. kubectl edit deployment api
D. kubectl update deployment api --replicas=5

5. SERVICE

Which Service type exposes a pod externally on a specific port of a node?

A. ClusterIP
B. NodePort
C. LoadBalancer
D. ExternalName

6. YAML

Which field sets environment variables in a container?

A. env
B. environment
C. variables
D. config

7. PVC

Which access mode allows ReadWrite by many nodes?

A. ReadWriteOnce
B. ReadOnlyMany
C. ReadWriteMany
D. ReadWriteAll

8. RBAC

Which RoleBinding refers to a ClusterRole?

A. RoleBinding
B. ClusterRoleBinding
C. ServiceBinding
D. PermissionBinding

9. NETWORKING

Which Kubernetes component implements NetworkPolicy?

A. kube-proxy
B. CoreDNS
C. CNI plugin
D. kube-scheduler

10. POD LOGS

Which command streams logs from a container named app?

A. kubectl logs app
B. kubectl logs pod/app
C. kubectl logs -f app
D. kubectl logs -f pod/app

11. NODE

Which command shows node resource usage?

A. kubectl top node
B. kubectl describe node
C. kubectl get node -o wide
D. kubectl logs node/<name>

12. CONFIGMAP

Which command creates a ConfigMap from a file named app.conf?

A. kubectl create configmap app --from-file=app.conf
B. kubectl apply -f app.conf
C. kubectl create cm app app.conf
D. kubectl configmap app app.conf

13. SECRETS

Which command decodes a secret value stored in base64?

A. base64 --decode <value>
B. kubectl decode secret <name>
C. kubectl get secret <name> -o yaml
D. kubectl secret decode <name>

14. ROLLOUT

Which command undoes the last deployment rollout?

A. kubectl rollout undo deployment/<name>
B. kubectl rollout restart deployment/<name>
C. kubectl undo deployment/<name>
D. kubectl rollback deployment/<name>

15. KUBECTL APPLY

What does kubectl apply do?

A. Creates resources only
B. Replaces resources only
C. Creates or updates resources declaratively
D. Deletes resources

16. POD FAILURE

A Pod is in ImagePullBackOff. What is the most likely cause?

A. Wrong image name or tag
B. Insufficient CPU
C. Node not ready
D. Service not found

17. PRACTICAL LAB (5 POINTS)

You are given the following YAML.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginxtest
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
```

Perform the following:

1. Create the Pod.
2. Expose the Pod using a ClusterIP Service named nginx-svc on port 80.
3. Verify the Service endpoints.
4. Clean up all resources.

(1.25 points each)

18. DEBUGGING SCENARIO (3 POINTS)

A Deployment named web has 3 replicas. Users report intermittent 502 errors.

Answer the following:

1. List 3 possible causes.
2. What commands will you run to investigate?
3. What would you check in the Pod logs?

(1 point each)

19. YAML EXAM QUESTION (3 POINTS)

What will be the result of this YAML?

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
    spec:
      containers:
      - name: app
        image: nginx:stable
```

A. 1 Pod will run
B. 2 Pods will run
C. 0 Pods will run
D. 2 Deployments will run

20. CONCEPTUAL QUESTION (2 POINTS)

Explain the difference between ReadWriteOnce and ReadWriteMany. Provide one use case for each.

ANSWER SHEET

Mark your answers in the boxes below.

1	2	3	4	5	6	7	8	9	10
A	B	B	A	A	B	A	B	C	D

11	12	13	14	15	16	17	18	19	20
A	B	C	B	C	D	C	B	C	D

EXAM TIPS

- ☑ Read each question carefully.
- ☑ Use kubectl explain and --help.
- ☑ Verify your work before moving on.
- ☑ Manage time wisely.
- ☑ Stay calm and focused.

45. MOCK EXAM 3

Realistic CKA-style questions. 60 minutes. 20 Questions.
Choose the best answer for each question.

EXAM INFO

Total Questions	:	20
Time Limit	:	60 Minutes
Passing Score	:	70% (14/20)
Exam Type	:	CKA Practice

1. NAMESPACE

Which command creates a namespace named prod?

A. kubectl create ns prod
B. kubectl ns prod
C. kubectl new namespace prod
D. kubectl create namespace prod

2. CONTEXT

Which command lists all contexts?

A. kubectl context list
B. kubectl config contexts
C. kubectl get-contexts
D. kubectl config view contexts

3. PODS

Which command shows all Pods in all namespaces?

A. kubectl get pods -A
B. kubectl get pods --all
C. kubectl get pods --all-namespaces
D. kubectl list pods -A

4. DEPLOYMENT

Which command updates image of a deployment named web?

A. kubectl set image deploy/web nginx:1.25
B. kubectl update deployment web --image nginx:1.25
C. kubectl edit deploy/web --image nginx:1.25
D. kubectl modify deployment web --image nginx:1.25

5. SERVICE

Which Service type exposes a Pod externally on a cloud provider LoadBalancer?

A. ClusterIP
B. NodePort
C. LoadBalancer
D. ExternalName

6. YAML

Which field sets resource limits for a container?

A. resources
B. limits
C. capacity
D. constraints

7. PVC

Which access mode allows ReadWrite by a single node?

A. ReadWriteOnce
B. ReadOnlyMany
C. ReadWriteMany
D. ReadWriteAll

8. RBAC

Which resource defines permissions for a ServiceAccount?

A. RoleBinding
B. ClusterRoleBinding
C. Role
D. PermissionSet

9. NETWORKING

Which Kubernetes component implements NetworkPolicy?

A. kube-proxy
B. CoreDNS
C. CNI plugin
D. kube-scheduler

10. POD LOGS

Which command shows logs from container api in Pod api-5f7d?

A. kubectl logs api-5f7d -c api
B. kubectl get logs api-5f7d -c api
C. kubectl logs api -c api-5f7d
D. kubectl show logs api-5f7d -c api

11. NODE

Which command shows allocatable resources on a node?

A. kubectl describe node <name>
B. kubectl get node <name> -o wide
C. kubectl top node <name>
D. kubectl node info <name>

12. CONFIGMAP

Which command creates a ConfigMap from a file named app.conf?

A. kubectl create configmap app --from-file=app.conf
B. kubectl apply configmap app --from-file=app.conf
C. kubectl create cm app app.conf
D. kubectl create config app --file app.conf

13. SECRETS

Which command decodes a secret value from secret.yaml?

A. kubectl decode secret -f secret.yaml
B. kubectl get secret -f secret.yaml -o yaml
C. kubectl get secret -o yaml secret.yaml
D. kubectl view secret secret.yaml

14. ROLLOUT

Which command checks rollout status of deployment web?

A. kubectl rollout status deployment/web
B. kubectl rollout check deployment web
C. kubectl status rollout deployment/web
D. kubectl rollout verify deployment/web

15. KUBECTL APPLY

What does kubectl apply -f do?

A. Create only if not exists
B. Replace the resource
C. Create or update the resource
D. Delete the resource

16. POD FAILURE

A Pod is in CrashLoopBackOff.
What is the most likely cause?

A. Wrong command or entrypoint
B. Insufficient CPU
C. Node cordoned
D. Service not found

17. PRACTICAL LAB (5 POINTS)

You are given the following YAML.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: busybox
spec:
  containers:
  - name: box
    image: busybox
    command: ["sleep", "3600"]
```

Perform the following:

1. Create the Pod.
2. Execute into the Pod.
3. List files in /.
4. Delete the Pod.
5. Verify the Pod is removed.

(1 point each)

18. DEBUGGING SCENARIO (3 POINTS)

A Deployment named auth has 3 replicas. Users report intermittent 503 errors.

Answer the following:

1. List 3 possible causes.
2. Commands you would run to investigate.
3. What would you check in the Pod logs?

(1 point each)

19. YAML EXAM QUESTION (3 POINTS)

What will be the result of this YAML?

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
```

A. ClusterIP on port 80
B. NodePort on a random high port
C. NodePort on port 80
D. LoadBalancer on port 8080

20. CONCEPTUAL QUESTION (2 POINTS)

Explain the difference between ReadWriteOnce and ReadWriteMany. Provide one use case for each.

ANSWER SHEET

Mark your answers in the boxes below.

1	2	3	4	5	7	8	9	10
A	B	C	A	A	A	B	C	B
11	12	13	14	15	16	17	19	20
A	B	C	D	A	C	C	A	B

EXAM TIPS

- ☑ Read each question carefully.
- ☑ Manage your time wisely.
- ☑ Use kubectl explain and --help.
- ☑ Verify your work before moving on.
- ☑ Stay calm and focused.

Preparation builds confidence.
Confidence leads to success.
You've trained hard.
Now go test and prove it!
CKA is yours.

46. SCENARIO-BASED TROUBLESHOOTING 1

Real-world scenario. Follow the investigation path. Identify the root cause and apply the fix.

SCENARIO INFO

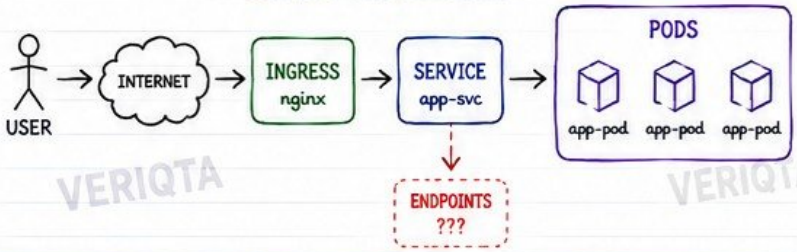
Type	:	Troubleshooting
Focus Area	:	Application Not Accessible
Difficulty	:	Medium
Exam Relevance	:	High
Time Suggested	:	15 Minutes

SCENARIO

A team reports that their application is not accessible from the browser.

- The Deployment was working yesterday.
- No recent code changes.
- The Service and Ingress exist.
- Users get "Connection Timed Out".

CURRENT ARCHITECTURE



INVESTIGATION PATH (FOLLOW IN ORDER)

1. Check Pod Status
Verify if pods are running and ready.

```
kubectl get pods -n <ns> -o wide
```

2. Check Pod Details
Inspect pod events and container status.

```
kubectl describe pod <pod-name> -n <ns>
```

3. Check Service
Verify service exists and has correct selector.

```
kubectl get svc -n <ns>
kubectl describe svc <svc-name> -n <ns>
```

4. Check Endpoints
Ensure service has endpoints (pods behind it).

```
kubectl get endpoints <svc-name> -n <ns>
kubectl describe endpoints <svc-name> -n <ns>
```

5. Check Ingress
Verify ingress rules and backend service.

```
kubectl get ingress -n <ns>
kubectl describe ingress <ingress-name> -n <ns>
```

6. Check Events
Look for recent cluster or namespace events.

```
kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp
```

7. Test Connectivity
Test service from inside the cluster.

```
kubectl run test-pod --image=busybox -it --rm -- sh
# Inside pod:
wget -O- http://<svc-name>.<ns>.svc.cluster.local
```

POSSIBLE ROOT CAUSES

1. Pods are not running or crashing.
2. Service selector does not match pods.
3. Endpoints are empty.
4. Ingress points to wrong service/port.
5. NetworkPolicy blocking traffic.
6. Node or resource pressure causing failures.

QUICK COMMANDS CHEAT SHEET

- Get all resources in namespace

```
kubectl get all -n <ns>
```

- Describe deployment

```
kubectl describe deploy <deploy-name> -n <ns>
```

- Check logs of a pod

```
kubectl logs <pod-name> -n <ns>
```

- Describe node

```
kubectl describe node <node-name>
```

- Top pods and nodes

```
kubectl top pods -n <ns>
kubectl top nodes
```

COMMON SYMPTOMS & WHAT THEY MEAN

SYMPTOM	LIKELY CAUSE
Pod in CrashLoopBackOff	Application error or misconfiguration
Service has no Endpoints	Selector mismatch or pods not ready
Connection Timed Out	NetworkPolicy / Ingress / Firewall
503 / 502 Errors	Backend pods not healthy
Pods Pending	Insufficient resources / scheduling issue

FIX SUMMARY (EXAMPLES)

- ✓ Fix deployment / application error.
- ✓ Correct service selector to match pod labels.
- ✓ Ensure pods are Ready and endpoints exist.
- ✓ Update ingress backend service or port.
- ✓ Adjust NetworkPolicy to allow required traffic.
- ✓ Add resources or fix node pressure issues.

SENIOR TIP

Always validate the traffic path step-by-step:
User → Ingress → Service → Endpoints → Pods
Don't guess. Verify.

VERIFICATION CHECKLIST

- ☐ Pods are Running and Ready
- ☐ Service has valid Endpoints
- ☐ Ingress routes to correct Service
- ☐ Application responds from inside cluster
- ☐ Application accessible from browser

NOTES

EXAM TIP

In exams, mark the most likely root cause first, then list at least 2 additional possible causes and how you would verify each. Show your thinking process.

Veriqta. Learn. Practice. Succeed.

47. SCENARIO-BASED TROUBLESHOOTING 2

Real-world scenario. Follow the investigation path. Identify the root cause and apply the fix.

SCENARIO INFO

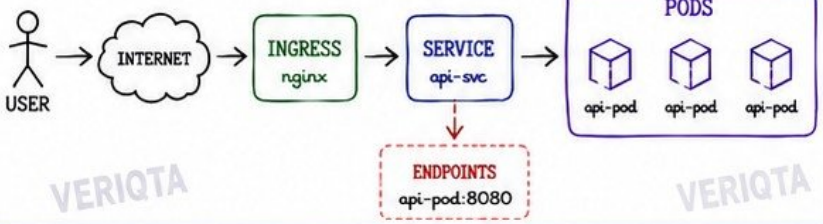
Type	:	Troubleshooting
Focus Area	:	Pod CrashLoopBackOff
Difficulty	:	Medium
Exam Relevance	:	High
Time Suggested	:	15 Minutes

SCENARIO

A backend API Pod is in CrashLoopBackOff.

- The deployment was working earlier.
- Recent change: New image was deployed.
- Pods fail to start and keep restarting
- Users experience 500 errors.

CURRENT ARCHITECTURE



INVESTIGATION PATH (FOLLOW IN ORDER)

1. Check Pod Status
Verify pods are in CrashLoopBackOff.

```
kubectl get pods -n <ns>
kubectl get pods -n <ns> -o wide
```

2. Check Pod Description
Look for events and reason.

```
kubectl describe pod <pod-name> -n <ns>
```

3. Check Pod Logs
Inspect container logs for errors.

```
kubectl logs <pod-name> -n <ns>
kubectl logs <pod-name> -n <ns> --previous
```

4. Check Events
Look for ImagePull, BackOff, or errors.

```
kubectl get events -n <ns> --sort-by='.metadata.creationTimestamp
```

5. Check Deployment
Verify rollout status and replica count.

```
kubectl rollout status deployment/<deploy-name> -n <ns>
kubectl get deployment -n <ns>
```

6. Check Config
Confirm ConfigMap/Secret or env variables.

```
kubectl get configmap,secret -n <ns>
kubectl describe configmap <cm-name> -n <ns>
```

7. Check Resource Usage
Ensure node has enough resources.

```
kubectl top pods -n <ns>
kubectl describe node <node-name>
```

8. Test Image Locally
Pull and run the image locally to validate.

```
docker pull <image:tag>
docker run --rm -it <image:tag>
```

POSSIBLE ROOT CAUSES

1. Application error at startup.
2. Missing or incorrect environment variables.
3. ConfigMap or Secret not found.
4. Image not found or pull error.
5. Port misconfiguration.
6. Database or external dependency unavailable.
7. Insufficient memory or CPU.
8. Liveness/Readiness probe failing.

QUICK COMMANDS CHEAT SHEET

- Get pods in CrashLoopBackOff
  `kubectl get pods -n <ns>`
- Describe pod
  `kubectl describe pod <pod-name> -n <ns>`
- View logs (current and previous)
  `kubectl logs <pod-name> -n <ns> --previous`
- Get events
  `kubectl get events -n <ns>`
- Check deployment rollout
  `kubectl rollout status deployment/<deploy-name> -n <ns>`
- Check node resources
  `kubectl top nodes`

COMMON SYMPTOMS & WHAT THEY MEAN

SYMPTOM	LIKELY CAUSE
CrashLoopBackOff	App crash / startup failure
ImagePullBackOff	Image not found / auth / registry issue
ErrImagePull	Cannot pull image / bad tag
BackOff	Repeated failures with backoff
OOMKilled	Out of memory

FIX SUMMARY (EXAMPLES)

- ✓ Review logs and fix application error.
- ✓ Verify environment variables and secrets.
- ✓ Ensure ConfigMap/Secret exists and is correct.
- ✓ Correct image name/tag or registry access.
- ✓ Fix port and probe configuration.
- ✓ Check external dependencies (DB, API, etc).
- ✓ Adjust resource requests/limits.
- ✓ Redeploy and verify the fix.

SENIOR TIP

Logs tell you what happened.
Events tell you why.
Descriptions show the whole story.
Always read in this order:
Logs → Events → Describe → Architecture → Fix.
Stay methodical. Don't guess.
Verify after every change.

VERIQTA

VERIFICATION CHECKLIST

- ☐ Pods are Running and Ready
- ☐ Deployment rollout is successful
- ☐ Service has valid Endpoints
- ☐ Application is responding correctly
- ☐ No errors in logs or events

VERIQTA

NOTES

EXAM TIP

In exams, read the scenario carefully, identify the area (Pod / Service / Network / RBAC / Storage), then follow a logical path. Check, Observe, Analyze, Fix, Verify.

Veriqta. Learn. Practice. Succeed.

48. SCENARIO-BASED TROUBLESHOOTING 3

Real-world scenario. Follow the investigation path.
Identify the root cause and apply the fix.

SCENARIO INFO

Type	:	Troubleshooting
Focus Area	:	Intermittent Connectivity
Difficulty	:	Medium
Exam Relevance	:	High
Time Suggested	:	15 Minutes

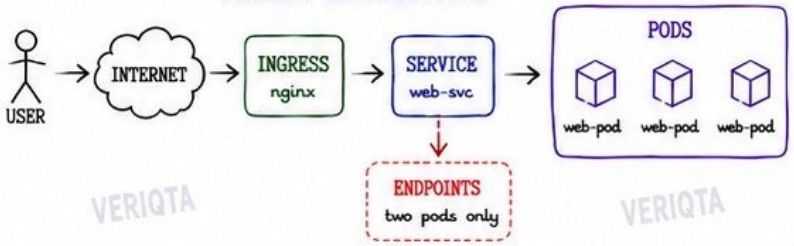
VERIQTA

SCENARIO

Users report intermittent timeouts when accessing the application.

- Service and Pods are running.
- Issue happens randomly.
- No code or deployment changes.
- Some requests succeed, some fail.

CURRENT ARCHITECTURE



INVESTIGATION PATH (FOLLOW IN ORDER)

1 Check Pod Status
Ensure all pods are Running.

```
kubectl get pods -n <ns> -o wide
```

2 Check Events
Look for warnings or restart events.

```
kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp
```

3 Check Endpoints
Verify endpoints include all healthy pods.

```
kubectl get endpoints <svc-name> -n <ns>
kubectl describe endpoints <svc-name> -n <ns>
```

4 Check Pod Readiness
Confirm readiness probe is passing.

```
kubectl get pods -n <ns> -o wide
kubectl describe pod <pod-name> -n <ns>
```

5 Check Service Selector
Verify selector matches pod labels.

```
kubectl describe svc <svc-name> -n <ns>
```

6 Check Network Policy
Ensure policies allow required traffic.

```
kubectl get networkpolicy -n <ns>
kubectl describe networkpolicy <np-name> -n <ns>
```

7 Test Pod Connectivity
Test from inside another pod.

```
kubectl run test-pod -n <ns> --image=busybox -it --rm -- sh
# Inside pod:
wget -qO- http://<svc-name>.<ns>.svc.cluster.local
```

8 Check Ingress Logs
Look for upstream errors or timeouts.

```
kubectl logs -n <ingress-ns> <ingress-pod> --tail=100
```

POSSIBLE ROOT CAUSES

1. Some pods failing readiness probe.
2. Endpoints missing healthy pods.
3. Service selector does not match pod labels.
4. NetworkPolicy intermittently blocking traffic.
5. Backend service slow or timing out.
6. DNS resolution issues.
7. Resource limits causing throttling.
8. LoadBalancer / Ingress unstable.

QUICK COMMANDS CHEAT SHEET

- Get pods with labels
  ```
  kubectl get pods --show-labels -n <ns>
  ```
- Describe service and endpoints
  ```
  kubectl describe svc <svc-name> -n <ns>
  kubectl describe endpoints <svc-name> -n <ns>
  ```
- Top pods and nodes
  ```
  kubectl top pods -n <ns>
  kubectl top nodes
  ```
- Check DNS from pod
  ```
  nslookup <svc-name>.<ns>.svc.cluster.local
  ```
- Check logs (deployment)
  ```
  kubectl logs -l app=<app-label> -n <ns> --tail=100
  ```

COMMON SYMPTOMS & WHAT THEY MEAN

SYMPTOM	LIKELY CAUSE
Intermittent Timeouts	Backend slow / pod not ready in time
Some Requests Fail	Only some pods unhealthy / missing endpoints
DNS Resolution Fails	DNS issue / CoreDNS problem
Connection Reset	NetworkPolicy / Firewall issue
High Latency	Resource saturation / throttling

FIX SUMMARY (EXAMPLES)

- ✓ Fix and validate readiness & liveness probes.
- ✓ Ensure Service selector matches pod labels.
- ✓ Confirm all healthy pods in endpoints.
- ✓ Adjust NetworkPolicies to allow required traffic.
- ✓ Increase resources if pods are throttled.
- ✓ Investigate backend dependencies.
- ✓ Monitor after fix to ensure stability.

SENIOR TIP

Intermittent issues are the hardest.
Think: Is it a pattern, a race condition, or a partial failure?
Use metrics + events + logs together.
Observe before you change.
Fix the real problem, not the noise.

VERIQTA

VERIFICATION CHECKLIST

- ☐ Pods are Running and Ready
- ☐ Endpoints include all healthy pods
- ☐ Service selector matches pod labels
- ☐ NetworkPolicies allow traffic
- ☐ Users can access application reliably

VERIQTA

NOTES

EXAM TIP

In exams, read the scenario carefully.
Check pods → endpoints → service → network → backend.
Follow the logical path. Verify after each fix.
Veriqta. Learn. Practice. Succeed.

49. ESSENTIAL KUBECTL CHEAT SHEET

Your go-to reference for Kubernetes commands.
Save this. Use it daily. Master it.

QUICK CONTEXT

- Current Context : `kubectl config current-context`
- Current Namespace : `kubectl config view --minify | grep namespace`
- Switch Namespace : `kubectl config set-context --current --namespace=<ns>`
- All Contexts : `kubectl config get-contexts`
- All Namespaces : `kubectl get ns`

VERIQTA

1. RESOURCE COMMANDS (GET / LIST)

List all pods (all namespaces)
```
kubectl get pods -A
```
List pods in namespace
```
kubectl get pods -n <ns>
```
List pods with wide output
```
kubectl get pods -n <ns> -o wide
```
List all resources in namespace
```
kubectl get all -n <ns>
```
List all resources (all namespaces)
```
kubectl get all -A
```
List resources with labels
```
kubectl get pods -l app=web -n <ns>
```
Watch resources
```
kubectl get pods -n <ns> -w
```

VERIQTA

2. DESCRIBE & DETAILS

Describe a pod
```
kubectl describe pod <pod-name> -n <ns>
```
Describe a deployment
```
kubectl describe deployment <deploy-name> -n <ns>
```
Describe a service
```
kubectl describe svc <svc-name> -n <ns>
```
Describe nodes
```
kubectl describe nodes
```
Describe namespace
```
kubectl describe ns <ns>
```
Describe events (namespace)
```
kubectl get events -n <ns>
--sort-by=.metadata.creationTimestamp
```

VERIQTA

3. LOGS & DEBUGGING

Pod logs
```
kubectl logs <pod-name> -n <ns>
```
Follow logs
```
kubectl logs -f <pod-name> -n <ns>
```
Logs from previous container
```
kubectl logs <pod-name> -n <ns> --previous
```
Logs from specific container
```
kubectl logs <pod-name> -c <container-name> -n <ns>
```
Tail last N lines
```
kubectl logs <pod-name> -n <ns> --tail=100
```
Logs with timestamps
```
kubectl logs <pod-name> -n <ns> -t
```

VERIQTA

4. EXEC & DEBUG INTO CONTAINERS

Exec into pod (bash)
```
kubectl exec -it <pod-name> -n <ns> -- /bin/bash
```
Exec into pod (sh)
```
kubectl exec -it <pod-name> -n <ns> -- /bin/sh
```
Exec into specific container
```
kubectl exec -it <pod-name> -c <container> -n <ns> -
-- /bin/bash
```
Run command in pod
```
kubectl exec <pod-name> -n <ns> -- <command>
```
Copy file from pod
```
kubectl cp <ns>/<pod-name>:/path/to/file ./
```
Copy file to pod
```
kubectl cp ./file <ns>/<pod-name>:/path/to/file
```

VERIQTA

5. CREATE, APPLY & EDIT

Apply a manifest
```
kubectl apply -f file.yaml
```
Apply multiple manifests
```
kubectl apply -f directory/
```
Create from stdin
```
kubectl apply -f - < file.yaml
```
Edit a resource
```
kubectl edit <resource> <name> -n <ns>
```
Create a deployment
```
kubectl create deploy <name> --image=<image> -n <ns>
```
Create a service
```
kubectl create svc <name> --tcp=80:80 -n <ns>
```
Dry run (client)
```
kubectl apply -f file.yaml --dry-run=client -o yaml
```
Dry run (server)

VERIQTA

6. UPDATE & PATCH

Set image in deployment
```
kubectl set image deploy/<deploy-name> <container>=<image> -n <ns>
```
Scale deployment
```
kubectl scale deploy/<deploy-name> --replicas=3 -n <ns>
```
Patch resource (strategic merge)
```
kubectl patch <resource> <name> -n <ns> -p '{"spec":{"replicas":3}}'
```
Patch resource (JSON)
```
kubectl patch <resource> <name> -n <ns> --type='json' \
-p='[{"op":"replace","path":"/spec/replicas","value":3}]'
```
Rolling restart deployment
```
kubectl rollout restart deploy/<deploy-name> -n <ns>
```

VERIQTA

7. DELETE & CLEAUNP

Delete a pod
```
kubectl delete pod <pod-name> -n <ns>
```
Delete by label
```
kubectl delete pods -l app=web -n <ns>
```
Delete a deployment
```
kubectl delete deploy <deploy-name> -n <ns>
```
Delete all resources in namespace
```
kubectl delete all --all -n <ns>
```
Force delete pod (grace period 0)
```
kubectl delete pod <pod-name> -n <ns> --grace-period=0 --force
```

VERIQTA

8. ROLLOUT & HISTORY

Show rollout status
```
kubectl rollout status deploy/<deploy-name> -n <ns>
```
Show rollout history
```
kubectl rollout history deploy/<deploy-name> -n <ns>
```
Rollout history with revision
```
kubectl rollout history deploy/<deploy-name> --revision=2 -n <ns>
```
Undo to previous revision
```
kubectl rollout undo deploy/<deploy-name> -n <ns>
```
Pause rollout
```
kubectl rollout pause deploy/<deploy-name> -n <ns>
```
Resume rollout
```
kubectl rollout resume deploy/<deploy-name> -n <ns>
```

9. NODES & CLUSTER INFO

List nodes
```
kubectl get nodes
```
Nodes (wide)
```
kubectl get nodes -o wide
```
Node details
```
kubectl describe node <node-name>
```
Cluster info
```
kubectl cluster-info
```
API resources
```
kubectl api-resources
```
Top nodes (metrics-server required)
```
kubectl top nodes
```
Top pods
```
kubectl top pods -A
kubectl top pods -n <ns> --sort-by=.metadata.creationTimestamp
kubectl top nodes
```

VERIQTA

10. CONFIGURATION & HELP

View kubeconfig
```
kubectl config view
```
View kubeconfig (minified)
```
kubectl config view --minify
```
Set context
```
kubectl config use-context <context-name>
```
Set namespace
```
kubectl config set-context --current --namespace=<ns>
```
Explain a resource
```
kubectl explain pods
```
Explain a field
```
kubectl explain pods.spec.containers
```
kubectl help
```
kubectl --help
```

VERIQTA

SENIOR TIP

- ☑ Always use -n <ns> to avoid mistakes.
- ☑ Use -o wide for better visibility.
- ☑ Describe before you delete.
- ☑ Check events when things break.
- ☑ Logs + Describe = 80% of debugging.
- ☑ Test changes in non-prod first.

VERIQTA

MOST USED COMMANDS (TOP 10)

1. `kubectl get pods -A`
2. `kubectl describe pod <pod> -n <ns>`
3. `kubectl logs -f <pod> -n <ns>`
4. `kubectl exec -it <pod> -n <ns> -- /bin/bash`
5. `kubectl apply -f file.yaml`
6. `kubectl get all -n <ns>`
7. `kubectl rollout status deploy/<name> -n <ns>`
8. `kubectl delete pod <pod> -n <ns>`
9. `kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp`
10. `kubectl top nodes`

VERIQTA

50. YAML PATTERNS

Common Kubernetes YAML patterns you must understand and use in real-world environments.

YAML QUICK NOTES

- YAML is indentation-sensitive.
- Use 2 spaces for indentation.
- Key: value pairs define configuration.
- --- separates multiple YAML documents.
- # starts a comment.

VERIQTA

1. POD

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
  - name: my-container
    image: nginx:1.25
    ports:
    - containerPort: 80
```

Use case: Run a single container.

2. DEPLOYMENT

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-container
        image: nginx:1.25
        ports:
        - containerPort: 80
```

Use case: Manage replica pods with rolling updates.

3. SERVICE

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 80
    protocol: TCP
```

Use case: Expose pods internally.

4. CONFIGMAP

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: my-app
  LOG_LEVEL: info
  FEATURE_FLAG: "true"
```

Use case: Store non-sensitive configuration data.

5. SECRET

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: cm9vdA==
  password: cGFzc3dvcmQ=
```

Use case: Store sensitive data (base64 encoded).

6. NAMESPACE

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: production
  labels:
    purpose: prod
```

Use case: Isolate resources by environment or team.

7. SERVICE ACCOUNT

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
```

Use case: Provide identity for pods.

8. ROLE & ROLEBINDING

Role

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: read-pods
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

RoleBinding

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: production
roleRef:
  kind: Role
  name: read-pods
  apiGroup: rbac.authorization.k8s.io
```

Use case: Grant permissions within a namespace.

9. INGRESS

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-service
            port:
              number: 80
```

Use case: Expose HTTP/HTTPS routes externally.

10. VOLUME & VOLUME MOUNT

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-volume
spec:
  containers:
  - name: app
    image: nginx:1.25
    volumeMounts:
    - name: app-data
      mountPath: /usr/share/nginx/html
  volumes:
  - name: app-data
    emptyDir: {}
```

Use case: Persist or share data with containers.

11. RESOURCE LIMITS & REQUESTS

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-limited-pod
spec:
  containers:
  - name: app
    image: nginx:1.25
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "256Mi"
```

Use case: Control resource usage and ensure stability.

YAML BEST PRACTICES

- ✓ Use meaningful names & labels.
- ✓ Keep indentation consistent (2 spaces).
- ✓ Use comments to explain complex sections.
- ✓ Separate configs using --- for multi-doc files.
- ✓ Validate YAML before applying.
- ✓ Use ConfigMaps & Secrets appropriately.
- ✓ Follow least privilege in RBAC.
- ✓ Version control all YAML files.

EXAMPLE: MULTI-DOCUMENT YAML

```yaml
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: info
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
```

Use --- to combine related resources in one YAML file.

VERIQTA

COMMON LABELS

Label	Purpose
app	Application name
env	Environment (dev, prod, staging)
tier	Layer (frontend, backend, db)
team	Owning team
version	Application version
component	Component name

51. NETWORKING QUICK REVIEW

A quick review of essential Kubernetes networking concepts, resources, and troubleshooting.

QUICK NOTES

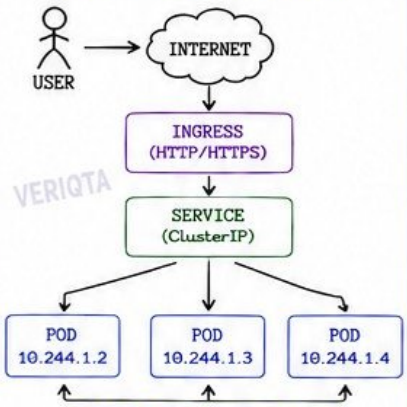
- Understand traffic flow.
- Validate DNS, Services, Endpoints, and Pods.
- NetworkPolicies control traffic between pods.
- Ingress manages external HTTP/HTTPS access.
- CNI plugin provides pod-to-pod connectivity.
- When in doubt, test from inside the cluster.

VERIQTA

CORE NETWORKING CONCEPTS

Concept	Explanation
Pod Networking	Every pod gets its own IP address. Pods communicate directly.
Service	Stable endpoint (ClusterIP) for pods. Load balances traffic.
DNS	CoreDNS resolves service names to ClusterIP.
Ingress	HTTP/HTTPS access to services from outside the cluster.
NetworkPolicy	Controls traffic between pods and namespaces.
CNI Plugin	Provides networking connectivity for pods (e.g., Calico, Cilium).

TRAFFIC FLOW OVERVIEW



COMMON NETWORK RESOURCES

Resource	Purpose
Pod	The smallest deployable unit with its own IP.
Service	Stable network endpoint to a set of pods.
Endpoints	List of pod IPs behind a service.
Ingress	External access to services (L7 routing).
NetworkPolicy	Allow/Deny traffic between pods.
DNS (CoreDNS)	Resolves service names inside the cluster.

DNS QUICK CHECK

Kubernetes uses CoreDNS.
All services are available via DNS.

Query Type	Example
Service (default ns)	my-service.default.svc.cluster.local
Service (other ns)	my-service.my-namespace.svc.cluster.local
Pod (A record)	<pod-ip>
Test DNS	nslookup my-service

COMMAND

```
nslookup my-service.default.svc.cluster.local
```

SERVICE TYPES

Type	Use Case
ClusterIP	Internal access within the cluster (default).
NodePort	Expose service on each node's IP and port.
LoadBalancer	Expose service externally using cloud LB.
ExternalName	Map service to external DNS name.
Headless	No ClusterIP. Used for stateful sets.

NETWORKPOLICY TYPES

Policy Type	Description
Ingress	Controls incoming traffic to pods.
Egress	Controls outgoing traffic from pods.
Ingress + Egress	Controls both directions.
Default Deny	Deny all traffic unless explicitly allowed.

EXAMPLE COMMAND

```
kubectl get networkpolicy -n <namespace>
```

COMMON CNI PLUGINS

CNI	Highlights
Calico	Policy + networking + observability.
Cilium	eBPF based, high performance and security.
Flannel	Simple and lightweight overlay network.
Weave Net	Easy to install, good for small clusters.

VALIDATION COMMANDS (MUST KNOW)

Check nodes	kubectl get nodes -o wide
Check pods	kubectl get pods -A -o wide
Check services	kubectl get svc -A
Check endpoints	kubectl get endpoints -A
Check ingress	kubectl get ingress -A
Check network policies	kubectl get networkpolicy -A
Test DNS inside pod	kubectl exec -it <pod> -- nslookup <svc>
Test connectivity	kubectl exec -it <pod> -- curl <svc>:<port>
Describe resource	kubectl describe <resource> <name> -n <ns>

COMMON PORTS & PROTOCOLS

Port	Protocol	Used For
53	UDP/TCP	DNS (CoreDNS)
80	TCP	HTTP
443	TCP	HTTPS
6443	TCP	Kubernetes API Server
10250	TCP	Kubelet
30000-32767	TCP	NodePort range
7946	TCP/UDP	Flannel (VXLAN)
4789	UDP	VXLAN

NETWORKING TROUBLESHOOTING CHECKLIST

Step	What to Check
1 User/Client	Is the request correct? DNS or URL?
2 Ingress	Ingress controller running? Rules correct?
3 Service	Service exists? Correct selector? Type?
4 Endpoints	Endpoints exist? Healthy pod IPs?
5 Pods	Pods running? Ready? Correct labels?
6 NetworkPolicy	Policies blocking traffic? Check allow rules.
7 DNS	Can pods resolve service names?
8 Connectivity	Test from inside cluster (curl/ping).
9 Node / CNI	CNI healthy? Nodes NotReady?

KEY TAKEAWAYS

- ✓ Pods get their own IP and communicate directly.
- ✓ Services provide stable endpoints to pods.
- ✓ DNS is the foundation of service discovery.
- ✓ Ingress handles HTTP/HTTPS traffic.
- ✓ NetworkPolicies secure pod-to-pod traffic.
- ✓ Always validate each layer of the traffic path.
- ✓ Use commands and logs to find the root cause.
- ✓ Test from inside the cluster first.
- ✓ Document and learn from every issue.

SENIOR TIP

When troubleshooting networking:

- ✓ Follow the traffic path.
- ✓ Check one layer at a time.
- ✓ Use cluster tools.
- ✓ Collect evidence.
- ✓ Fix the real problem, not the symptom.

VERIQTA

52. STORAGE QUICK REVIEW

A quick review of essential Kubernetes storage concepts, resources, storage classes, and troubleshooting.

QUICK NOTES

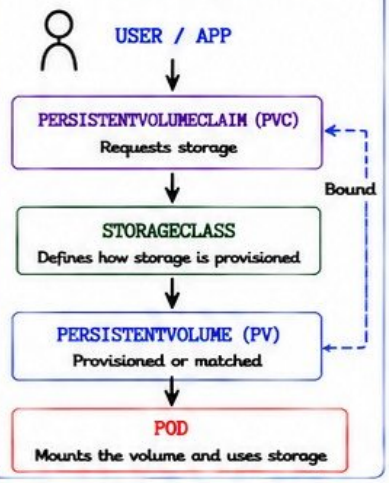
- Understand persistent storage in Kubernetes.
- Know storage classes and volume types.
- Always match the right storage to the workload.
- Bind a PersistentVolume to a PersistentVolumeClaim.
- Monitor usage, performance, and capacity.
- Test backups and restore procedures.

VERIQTA

CORE STORAGE CONCEPTS

Concept	Explanation
Volume	A directory with data, accessible to containers in a Pod.
PersistentVolume (PV)	Cluster resource that represents a piece of storage in the cluster.
PersistentVolumeClaim (PVC)	Request for storage by a user. Binds to a PV.
StorageClass	Defines storage parameters and provisions PVs dynamically.
Reclaim Policy	What happens to PV when PVC is deleted (Retain, Delete).
Access Modes	How the volume can be mounted (RWO, ROX, RWX).

STORAGE WORKFLOW



COMMON STORAGE CLASSES

Type	Use Case	Notes
gp3 (AWS EBS)	General purpose workloads	Balanced performance and cost
io1 / io2 (AWS EBS)	High IOPS databases	Low latency, consistent IOPS
pd-ssd (GCP)	General purpose workloads	Good performance
pd-balanced (GCP)	Balanced workloads	Cost optimized
premium-rwo (Azure Disk)	Production workloads	High performance
azurefile	Shared file storage	Supports RWX

VOLUME TYPES

Type	Description	Access Modes
emptyDir	Ephemeral storage lifetime of Pod	RWO
hostPath	Mount file or directory from node	RWO
configMap	Mount ConfigMap as a volume	R/O
secret	Mount Secret as a volume	R/O
persistentVolume	Use a PV with PVC	RWO, ROX, RWX
nfs	Mount NFS share	RWX
csi	CSI driver based volumes	Depends on driver

ACCESS MODES

Mode	Description
RWO ReadWriteOnce	Volume can be mounted as read-write by a single node.
ROX ReadOnlyMany	Volume can be mounted as read-only by many nodes.
RWX ReadWriteMany	Volume can be mounted as read-write by many nodes.

Note: Not all storage classes support all access modes.

RECLAIM POLICIES

Policy	Behavior
Retain	PV is not deleted when PVC is deleted. Data is retained.
Delete	PV and underlying storage is deleted when PVC is deleted.
Recycle	Deprecated. (Previously cleaned data and made available.)

EXAMPLE: PVC AND POD

PVC (my-pvc)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: gp3
```

POD (my-pod)

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: my-pvc
```

STORAGE TROUBLESHOOTING CHECKLIST

#	Check	What to Verify
1	PVC Status	Check if PVC is Bound.
2	StorageClass	Verify StorageClass exists and is default (if needed).
3	PV Status	Check PV status (Available / Bound).
4	Events	kubectl get events | grep -i pvc
5	Quota	Check storage quotas in the namespace.
6	Node Issues	Check node disk pressure and conditions.
7	CSI Driver	Check CSI driver pods and logs.
8	Mount Path	Verify mountPath inside the container.
9	Permissions	Check file permissions and ownership.

USEFUL COMMANDS

kubectl get pv	List all PVs
kubectl get pvc -A	List all PVCs
kubectl describe pv <pv-name>	Details of a PV
kubectl describe pvc <pvc-name>	Details of a PVC
kubectl get sc	List StorageClasses
kubectl describe sc <sc-name>	Details of a SC
kubectl get pods -A	List all pods
kubectl logs -n kube-system -l app=csi-driver	Check CSI driver logs
df -h	Check node disk usage

SENIOR TIP

- ✓ Choose the right storage type for the workload.
- ✓ Always use StorageClass for dynamic provisioning.
- ✓ Monitor capacity, IOPS, and latency.
- ✓ Test volume backups and restore.
- ✓ Clean unused PVs to avoid cost.
- ✓ Document storage and recovery process.

VERIQTA

COMMON TOOLS

- kubectl – Kubernetes CLI
- csi-snapshotter – Volume snapshots
- velero – Backup and restore
- prometheus – Storage metrics
- grafana – Storage dashboards

KEY TAKEAWAYS

- ✓ PersistentVolume provides storage in the cluster.
- ✓ PersistentVolumeClaim requests and binds storage.
- ✓ StorageClass enables dynamic provisioning.
- ✓ Choose proper access modes and reclaim policies.
- ✓ Monitor, backup, and troubleshoot regularly.

STORAGE BEST PRACTICES

- ✓ Use StorageClass (avoid static PVs).
- ✓ Set resource requests and limits.
- ✓ Use least privilege access.
- ✓ Monitor usage and set alerts.
- ✓ Keep regular backups and test restores.

53. SECURITY QUICK REVIEW

A quick review of essential Kubernetes security concepts, resources, security controls, and troubleshooting.

QUICK NOTES

- Security is everyone's responsibility.
- Apply least privilege everywhere.
- Validate policies. Test and monitor regularly.
- Keep Kubernetes, nodes, and images updated.
- Use defense in depth.
- Audit, alert, and respond fast.

VERIQTA

CORE SECURITY CONCEPTS

Concept	Explanation
Authentication	Verifying the identity of users, apps, and systems.
Authorization	Controlling what authenticated identities are allowed to do.
Admission Control	Validating or rejecting requests before objects are persisted.
Secrets Management	Storing sensitive data securely (Secrets, external stores).
Network Security	Controlling network traffic between pods and services.
Image Security	Ensuring container images are trusted and vulnerability-free.
Runtime Security	Protecting containers and nodes at runtime.

KUBERNETES SECURITY LAYERS

USER / CLIENT

↓

AUTHENTICATION
Verify identity (Users, ServiceAccounts)

↓

AUTHORIZATION
RBAC decides what is allowed

↓

ADMISSION CONTROL
Validate and enforce policies

↓

NETWORK SECURITY
NetworkPolicies, Ingress, CNI

↓

RUNTIME SECURITY
Pod Security, Seccomp, AppArmor

↓

CLUSTER & NODE SECURITY
Kubelet, etcd, OS hardening

IMPORTANT SECURITY RESOURCES

Resource	Purpose
RBAC	Role-Based Access Control for authorization.
Role / ClusterRole	Define permissions.
RoleBinding / ClusterRoleBinding	Bind roles to users, groups or SAs.
ServiceAccount	Identity for workloads inside the cluster.
Secret	Store sensitive data (tokens, passwords, etc).
NetworkPolicy	Control pod-to-pod and ingress/egress traffic.
PodSecurity	Enforce security standards at namespace level.
SecurityContext	Define privilege and user settings for pods.

POD SECURITY STANDARDS (PSS)

Profile	Description
Privileged	Unrestricted policy. All features available.
Baseline	Minimally restrictive. Blocks known privilege escalations.
Restricted	Heavily restricted. Follows security best practices.

Apply PSS using labels on namespaces:

```
kubectl label ns <namespace>
pod-security.kubernetes.io/enforce=restricted
```

RBAC QUICK REFERENCE

Object	What it does
Role	Defines permissions within a namespace.
ClusterRole	Defines permissions cluster-wide.
RoleBinding	Grants a Role to users, groups or ServiceAccounts in a namespace.
ClusterRoleBinding	Grants a ClusterRole cluster-wide.

EXAMPLE: VIEW PODS IN A NAMESPACE

```
kubectl create role pod-viewer \
  --verb=get,list,watch \
  --resource=pods

kubectl create rolebinding view-pods \
--role=pod-viewer --user=dev-user
```

NETWORK SECURITY

Component	Purpose
NetworkPolicy	Allow/Deny traffic between pods using labels.
Ingress	Control inbound HTTP/HTTPS traffic to services.
Egress	Control outbound traffic from pods.
CNI Plugin	Implements network connectivity and policies.

SAMPLE: DEFAULT DENY ALL (Egress)

```
apiVersion: networking.k8s.io/v1
kind:  NetworkPolicy
metadata:
  name: default-deny-egress
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress: []
```

SECURITY TROUBLESHOOTING CHECKLIST

	Check	What to Verify
1	AuthN	Can the user or SA authenticate? Check kubeconfig, tokens.
2	AuthZ	Does the identity have required permissions? Check RBAC.
3	Admission	Are requests blocked by admission controllers or policies?
4	Secrets	Are secrets mounted correctly and not exposed?
5	Network	Is NetworkPolicy blocking traffic? Check policies and labels.
6	Images	Is the image trusted and vulnerability-free?
7	Pod Security	Is the pod compliant with the namespace PSS?
8	Node Security	Are nodes hardened and up to date?
9	Audit Logs	Check audit logs for denied or suspicious activity.

USEFUL COMMANDS

`kubectl auth can-i <verb> <res> [--as user]`	Check permissions.
`kubectl get roles,rolebindings -A`	List roles & bindings.
`kubectl get sa -A`	List service accounts.
`kubectl get secrets -A`	List secrets.
`kubectl describe secret <name>`	View secret details.
`kubectl get networkpolicies -A`	List network policies.
`kubectl describe netpol <name>`	View policy details.
`kubectl get pod <pod> -o yaml`	Inspect pod spec.
`kubectl exec -it <pod> -- whoami`	Check container user.
`kubectl logs <pod>`	Check application logs.
`kubectl get events -A --sort-by=.lastTimestamp`	Check recent events.

SECURITY BEST PRACTICES

- ✓ Use RBAC and least privilege.
- ✓ Use dedicated ServiceAccounts.
- ✓ Enable and enforce Pod Security Standards.
- ✓ Use NetworkPolicies (default deny).
- ✓ Store secrets securely (external secrets / sealed-secrets).
- ✓ Scan images and keep base images minimal.
- ✓ Keep Kubernetes and nodes updated.
- ✓ Enable audit logging.
- ✓ Monitor, alert, and respond quickly.

COMMON SECURITY TOOLS

- Falco – Runtime security and threat detection
- Trivy – Image & filesystem vulnerability scanning
- kube-bench – CIS benchmark for Kubernetes
- OPA / Gatekeeper – Policy enforcement
- Kyverno – Policy as code
- HashiCorp Vault – Secrets management

KEY TAKEAWAYS

- ✓ Secure every layer: identity, network, workload, and node.
- ✓ Apply least privilege and verify continuously.
- ✓ Use policies to enforce security at scale.
- ✓ Monitor, audit, and improve your security posture.

SENIOR TIP

Security is not a one-time setup.
It's a continuous process of preventing, detecting, and responding.

Think: PREVENT → DETECT → RESPOND

VERIQTA

54. SCHEDULING QUICK REVIEW

A quick review of essential Kubernetes scheduling concepts, resources, scheduling mechanisms, and troubleshooting.

QUICK NOTES

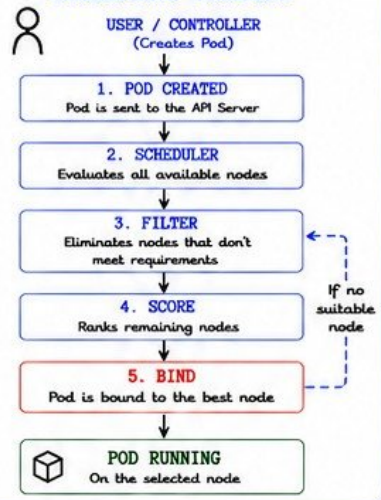
- Scheduler places pods on the best available node.
- It considers resources, policies, constraints, and priorities.
- Good scheduling = performance, reliability, and efficiency.
- When pods don't schedule, always check events!

VERIQTA

CORE SCHEDULING CONCEPTS

Concept	Explanation
Scheduler	Default scheduler assigns pods to the most suitable nodes.
Node	A worker machine where pods run.
Pod	The smallest deployable unit scheduled to a node.
Scheduling Decision	Based on resources, policies, affinity/anti-affinity, taints/tolerations, etc.
Re-Scheduling	Pods can be rescheduled if nodes become unschedulable or unavailable.

SCHEDULING WORKFLOW



IMPORTANT SCHEDULING RESOURCES

Resource	Purpose
nodeSelector	Schedule pods to nodes with specific labels.
nodeAffinity	Advanced rules for scheduling based on node labels.
podAffinity	Schedule pods close to other pods.
podAntiAffinity	Schedule pods away from other pods.
taints & tolerations	Repel or allow pods on specific nodes.
topologySpreadConstraints	Spread pods across failure domains evenly.
priorityClassName	Set scheduling priority for pods.
Resource Requests & Limits	Guarantee and limit resource usage for scheduling.

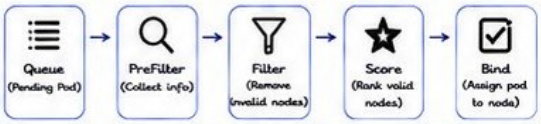
NODE SELECTION METHODS

Method	Use Case
nodeSelector	Simple key-value match (AND conditions).
nodeAffinity	Complex rules: requiredDuring, preferredDuring.
podAffinity	Place pods near other pods (same app/data locality).
podAntiAffinity	Place pods away from certain pods (isolation).
taints & tolerations	Control which pods can run on specific nodes.
topologySpreadConstraints	Ensure even distribution across zones, racks, or nodes.

SCHEDULER CONSIDERATIONS

Factor	Description
Resource Availability	CPU, Memory, Ephemeral Storage, Pods, Extended Resources.
Policies & Constraints	NodeSelector, Affinity, Taints, Tolerations, Topology.
Pod Priorities	priorityClassName affects preemption and placement.
Node Health	Node conditions: Ready, Memory Pressure, Disk Pressure, etc.
Data Locality	Prefer nodes with local data (when applicable).
Load Balancing	Distributes pods to avoid hotspots and improve availability.

DEFAULT SCHEDULER FLOW (Simplified)



EXAMPLE: NODE SELECTOR

```
spec:
  nodeSelector:
    disktype: ssd
    environment: production
```

Pod will be scheduled only on nodes with: disktype=ssd AND environment=production

EXAMPLE: Taints & Tolerations

```
# Node taint
kubectl taint nodes node1
  key=dedicated:NoSchedule
```

```
# Pod toleration
  tolerations:
  - key: "dedicated"
    operator: "Equal"
    value. "true"
    effect: "NoSchedule"
```

SCHEDULING TROUBLESHOOTING CHECKLIST

Check	What to Verify
1 Pod Status	kubectl get pods -o wide
2 Events	kubectl describe pod <pod-name>
3 Node Conditions	kubectl get nodes
4 Resource Availability	CPU, Memory, Pods, Storage
5 Node Labels	kubectl get nodes --show-labels
6 Taints	kubectl describe node <node-name>
7 Tolerations	Check tolerations in pod spec
8 Affinity / Anti-Affinity	Validate rules and label selectors
9 Topology Constraints	Check topologySpreadConstraints rules

VERIQTA

USEFUL COMMANDS

kubectl get pods -o wide	See where pods are scheduled
kubectl get nodes --show-labels	View node labels
kubectl describe pod <pod>	Check events and failures
kubectl describe node <node>	Check taints and conditions
kubectl get events --sort-by=.metadata.creationTimestamp	View recent events
kubectl get pods -n <ns>	Pods in a namespace
kubectl top nodes	Node resource usage
kubectl top pods -n <ns>	Pod resource usage

SENIOR TIP

- ✓ Label your nodes meaningfully.
- ✓ Use taints for special purpose nodes.
- ✓ Prefer affinity/anti-affinity over nodeSelector for flexibility.
- ✓ Use topologySpreadConstraints for high availability.
- ✓ Always check events first when pods stay Pending.

VERIQTA

COMMON SCHEDULING TOOLS

- kubectl – Command line management
- kube-scheduler – Default scheduler
- metrics-server – Resource metrics
- kube-state-metrics – Cluster state
- Prometheus – Monitoring and alerts

KEY TAKEAWAYS

- ✓ Scheduling is the heart of Kubernetes performance.
- ✓ Understand resources, policies, and constraints.
- ✓ Use the right scheduling rules for the workload.
- ✓ Monitor, observe, and troubleshoot continuously.
- ✓ A well-scheduled cluster is a reliable cluster.

SCHEDULING BEST PRACTICES

- ✓ Set accurate resource requests and limits.
- ✓ Use node labels and affinity wisely.
- ✓ Avoid over-constraining your pods.
- ✓ Distribute pods across failure domains.
- ✓ Monitor and review scheduling regularly.

55. CLUSTER ADMINISTRATION QUICK REVIEW

A quick review of essential Kubernetes cluster administration concepts, maintenance tasks, and operational best practices.

QUICK NOTES

- Healthy clusters are secure, observable, and updated.
- Automate repetitive tasks.
- Monitor everything.
- Plan for failure and recovery.
- Document your operational runbooks.

VERIQTA

CORE ADMIN CONCEPTS

Concept	Explanation
Control Plane	Manages the cluster (API Server, etcd, Scheduler, Controller Manager).
Worker Nodes	Run workloads (Pods, containers) and kubelet.
etcd	Distributed key-value store that holds all cluster data.
kube-apiserver	Front-end for the Kubernetes control plane.
kubectl	Command line tool to manage the cluster.
Cluster State	Desired state (spec) vs actual state (status).
High Availability	Multiple control plane nodes and etcd members.

CLUSTER ADMIN WORKFLOW

ADMIN

1. ACCESS CLUSTER
kubectl config use-context <ctx>

2. CHECK CLUSTER HEALTH
kubectl get componentstatuses
kubectl get nodes

3. MONITOR & REVIEW
Logs, metrics, events, alerts

4. MAINTAIN & SECURE
Updates, security, backups

5. AUTOMATE & OPTIMIZE
Scripts, policies, capacity

HEALTHY & RELIABLE CLUSTER

IMPORTANT ADMIN COMPONENTS

Component	Purpose
etcd	Stores all cluster data.
kube-apiserver	Handles all API requests.
kube-scheduler	Assigns Pods to nodes.
kube-controller-manager	Runs control loops to keep cluster in desired state.
kubelet	Ensures containers are running on nodes.
kube-proxy	Maintains network rules on nodes.
CoreDNS	Cluster DNS service.
Ingress Controller	Manages external HTTP/S traffic.

CLUSTER HEALTH CHECKS

Check	Command
Cluster Info	kubectl cluster-info
Nodes	kubectl get nodes -o wide
Component Status	kubectl get componentstatuses
All Pods	kubectl get pods -A
System Pods	kubectl get pods -n kube-system
Events	kubectl get events -A --sort-by=.lastTimestamp
Resource Usage	kubectl top nodes kubectl top pods -A
API Resources	kubectl api-resources
Cluster Version	kubectl version --short

ADMIN MAINTENANCE TASKS

Task	Command / Action
Drain Node	kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
Cordon Node	kubectl cordon <node>
Uncordon Node	kubectl uncordon <node>
Delete Node	kubectl delete node <node>
Rotate Certificates	kubeadm certs renew all
Upgrade Cluster	kubeadm upgrade plan kubeadm upgrade apply vX.Y.Z
Cleanup Unused Resources	kubectl delete pods --all -A --field-selector status.phase=Succeeded kubectl delete pvc --all -A
Backup etcd	etcdctl snapshot save backup.db
View Logs (Control Plane)	journalctl -u kube-apiserver -f journalctl -u etcd -f

CLUSTER SECURITY BASICS

Areas	Best Practices
Access Control	Use RBAC. Avoid cluster-admin for daily tasks.
Authentication	Use strong auth (OIDC, certs, MFA where possible).
Network Security	Use NetworkPolicies to limit pod traffic.
Image Security	Scan images. Use trusted registries.
Secrets Management	Use Kubernetes Secrets. Encrypt at rest.
Audit Logging	Enable audit logs and review regularly.
Updates	Keep Kubernetes, nodes, and add-ons up to date.

ADMIN TROUBLESHOOTING CHECKLIST

	Check	What to Verify
1	Nodes	Are nodes Ready? Any NotReady?
2	Components	Are control plane components healthy?
3	Events	Any Warning or Error events?
4	Pods	Any CrashLoopBackOff or Pending pods?
5	Resources	CPU, Memory, Disk pressure?
6	Networking	CNI status, DNS resolution, NetworkPolicies?
7	Storage	PV status, StorageClass, PVC issues?
8	Certificates	Any certs expired or near expiry?
9	Backups	Are backups recent and restorable?

VERIQTA

USEFUL ADMIN COMMANDS

kubectl get ns	List all namespaces
kubectl get nodes -o wide	List nodes with details
kubectl get pods -A -o wide	List all pods with details
kubectl get all -A	List all resources in all namespaces
kubectl describe node <node>	View node details
kubectl describe pod <pod> -n <ns>	View pod details
kubectl logs <pod> -n <ns>	View pod logs
kubectl logs <pod> -n <ns> -c <ctr> -f	Stream container logs
kubectl exec -it <pod> -n <ns> -- bash	Exec into pod
kubectl get pv,pvc	List PVs and PVCs
kubectl get sc	List StorageClasses
kubectl get events -A --sort-by=.lastTimestamp	Recent events

EXAMPLE: CHECK CLUSTER HEALTH

```
# Basic cluster health check
kubectl cluster-info
kubectl get nodes
kubectl get componentstatuses
kubectl get pods -A
kubectl get events -A --sort-by=.lastTimestamp | tail -20
kubectl top nodes
kubectl top pods -A
```

VERIQTA

SENIOR TIP

- ✓ Automate repetitive admin tasks.
- ✓ Monitor control plane and etcd.
- ✓ Test backups regularly.
- ✓ Document runbooks and procedures.
- ✓ Review and improve continuously.

VERIQTA

COMMON ADMIN TOOLS

- kubectl – Kubernetes CLI
- k9s – Terminal UI
- kube-prometheus-stack – Monitoring
- Velero – Backup & Restore
- etcdctl – etcd management

KEY TAKEAWAYS

- ✓ Know your cluster components and their roles.
- ✓ Monitor health, logs, and events continuously.
- ✓ Secure access and follow least privilege.
- ✓ Automate, backup, and test recovery.
- ✓ A reliable cluster is a well-maintained cluster.

56. MOST FREQUENTLY TESTED TASKS

A quick review of the most commonly tested Kubernetes tasks in exams, interviews, and real-world operations.

QUICK NOTES

- These tasks appear in CKA, CKAD, interviews, and daily operations.
- Practice until you can do them without thinking.
- Use kubectl help and --dry-run for safety.
- Speed + Accuracy = Confidence.
- Understand the why, not just the how.

VERIQTA

POD & WORKLOAD TASKS

Task	Commands / Actions
Run a Pod	kubectl run nginx --image=nginx --restart=Never
Get Pods	kubectl get pods -o wide
Describe a Pod	kubectl describe pod <pod-name>
View Pod Logs	kubectl logs <pod-name> [-c container]
Execute in Pod	kubectl exec -it <pod-name> -- /bin/bash
Delete a Pod	kubectl delete pod <pod-name>
Create Deployment	kubectl create deployment <name> --image=nginx
Scale Deployment	kubectl scale deployment <name> --replicas=3
Update Image	kubectl set image deployment/<name> <container>=<new-image>
Rollout Status	kubectl rollout status deployment/<name>
Rollout Undo	kubectl rollout undo deployment/<name>

SERVICE & NETWORKING TASKS

Task	Commands / Actions
Create ClusterIP Service	kubectl expose deployment <name> --port=80 --target-port=80
Create NodePort Service	kubectl expose deployment <name> --type=NodePort --port=80
Create LoadBalancer Service	kubectl expose deployment <name> --type=LoadBalancer --port=80
Get Services	kubectl get svc -o wide
Describe Service	kubectl describe svc <svc-name>
Get Endpoints	kubectl get endpoints <svc-name>
Create Ingress	kubectl apply -f ingress.yaml
View Ingress	kubectl get ingress -o wide
Test Connectivity	kubectl run curl --image=curlimages/curl -it --rm --restart=Never -- curl <service-name>:<port>

CONFIGURATION & STORAGE TASKS

Task	Commands / Actions
Create ConfigMap	kubectl create configmap <name> --from-literal=key=value
Create Secret	kubectl create secret generic <name> --from-literal=key=value
Use ConfigMap in Pod	Mount as volume or use envFrom
Use Secret in Pod	Mount as volume or use env
Create PVC	kubectl apply -f pvc.yaml
Get PVCs	kubectl get pvc -o wide
Get StorageClass	kubectl get storageclass
Describe PVC	kubectl describe pvc <pvc-name>
Delete PVC	kubectl delete pvc <pvc-name>

NAMESPACE & CONTEXT TASKS

Task	Commands
List Namespaces	kubectl get ns
Create Namespace	kubectl create namespace <name>
Set Namespace	kubectl config set-context --current --namespace=<name>
View Current Context	kubectl config current-context
Switch Context	kubectl config use-context <context>
Get All Contexts	kubectl config get-contexts

CLUSTER ADMIN TASKS

Task	Commands / Actions
Get Nodes	kubectl get nodes -o wide
Describe Node	kubectl describe node <node-name>
Cordon Node	kubectl cordon <node-name>
Drain Node	kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
Uncordon Node	kubectl uncordon <node-name>
View Cluster Info	kubectl cluster-info
Get Component Status	kubectl get componentstastues
Verify System Pods	kubectl get pods -n kube-system
View Events	kubectl get events --sort-by=.lastTimestamp

SECURITY TASKS

Task	Commands / Actions
Create ServiceAccount	kubectl create serviceaccount <name>
Create Role	kubectl create role <name> --verb=get,list --resource=pods
Create RoleBinding	kubectl create rolebinding <name> --role=<role> --serviceaccount=<ns>:<sa>
Create ClusterRole	kubectl create clusterrole <name> --verb=* --resource=*
Create ClusterRoleBinding	kubectl create clusterrolebinding <name> --clusterrole=<role> --user=<user>
View Permissions	kubectl auth can-i get pods --as=<user> -n <ns>
View ServiceAccounts	kubectl get sa -n <namespace>

TROUBLESHOOTING TASKS

	Task	Commands / Actions	
1	Pod Not Running	kubectl describe pod <pod>	kubectl logs <pod>
2	Image Pull Error	kubectl describe pod <pod>	Check image name & secret
3	CrashLoopBackOff	kubectl logs <pod> --previous	Check app logs
4	Pending Pod	kubectl describe pod <pod>	Check events & nodes
5	Node Not Ready	kubectl describe node <node>	Check kubelet status
6	Service Not Working	kubectl get endpoints <svc>	kubectl describe svc <svc>
7	DNS Not Resolving	kubectl run dns-test --image=busybox -it --rm -- nslookup <svc>	
8	PVC Pending	kubectl describe pvc <pvc>	Check StorageClass & events
9	Ingress Not Working	kubectl describe ingress <ing>	Check backend pods
10	Network Connectivity	kubectl run netshoot --image=nicolaka/netshoot -it --rm	

USEFUL COMMANDS (QUICK REFERENCE)

What it does	Command
Get everything in all ns	kubectl get all -A
Get pods with labels	kubectl get pods -l app=nginx
Get pods by field	kubectl get pods --field-selector status.phase=Running
Get logs from all containers	kubectl logs <pod> --all-containers=true
Edit a resource	kubectl edit <resource> <name>
Dry run (client)	kubectl apply -f file.yaml --dry-run=client
Explain a resource	kubectl explain pods.spec.containers
Get resource YAML	kubectl get pod <pod> -o yaml
Delete all pods in ns	kubectl delete pods --all -n <ns>
Top nodes / pods	kubectl top nodes kubectl top pods

PRACTICE MINDSET

- ☑ Practice every task in a safe environment.
- ☑ Time yourself and repeat.
- ☑ Understand the output.
- ☑ Learn from mistakes.
- ☑ Build confidence with repetition.
- ☑ Document what you learn.

MASTERY IS BUILT BY DOING
THE ESSENTIAL TASKS CONSISTENTLY.

COMMON TOOLS

- kubectl – Kubernetes CLI
- k9s – Terminal UI
- kube-prometheus-stack – Monitoring
- metrics-server – Resource metrics
- Velero – Backup & Restore
- etcdctl – etcd management

KEY TAKEAWAYS

- ✓ Master these tasks and you'll ace any exam or interview.
- ✓ Speed comes from practice. Accuracy comes from understanding.
- ✓ Always verify with describe and logs when troubleshooting.
- ✓ Keep learning and improving.
- ✓ Kubernetes rewards those who practice consistently.

SENIOR TIP

- ✓ Build muscle memory.
- ✓ Use aliases for speed.
- ✓ Read docs when stuck.
- ✓ Automate the repetitive.
- ✓ Keep your cluster clean and healthy.

VERIQTA

57. HIGH-YIELD TROUBLESHOOTING FLOWCHARTS

Quick reference flowcharts for solving the most common Kubernetes issues quickly and accurately.

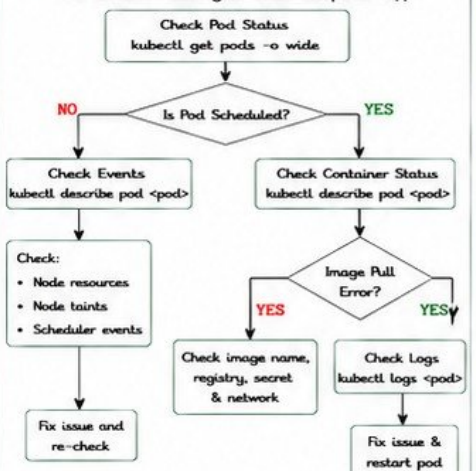
QUICK NOTES

- Follow the flow. Don't skip steps.
- Verify assumptions at every stage.
- Collect evidence before making changes.
- Small issues ignored become big outages.
- Troubleshooting is a skill—practice it.

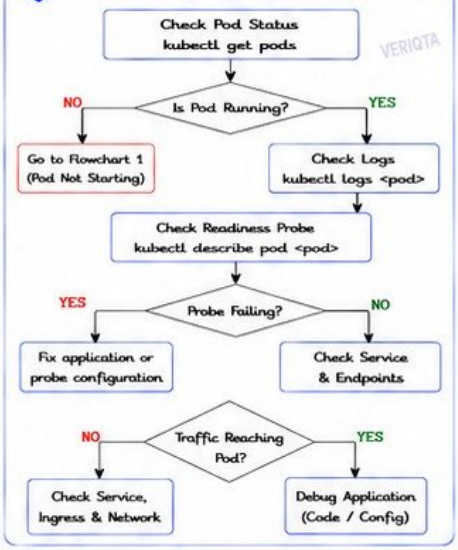
VERIQTA

1. POD NOT STARTING

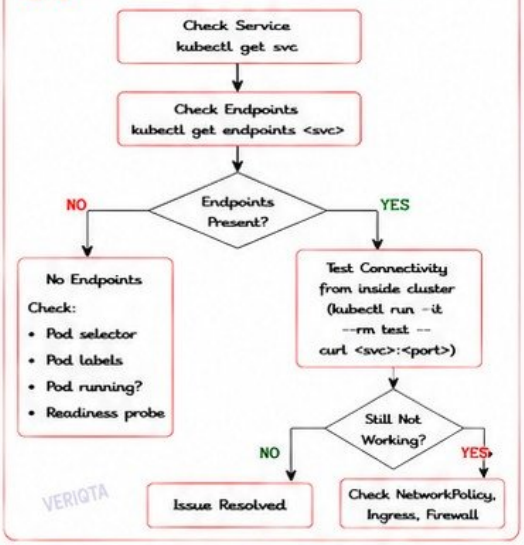
Pod status: Pending or CrashLoopBackOff



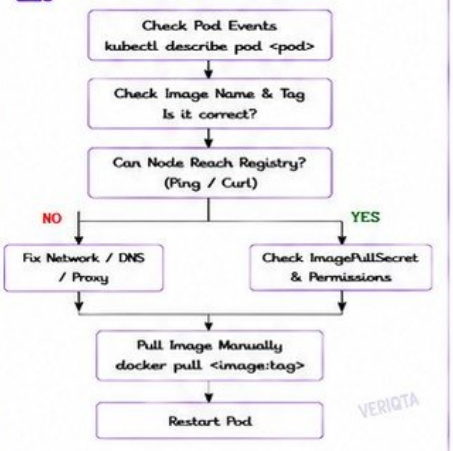
2. POD RUNNING BUT APP NOT WORKING



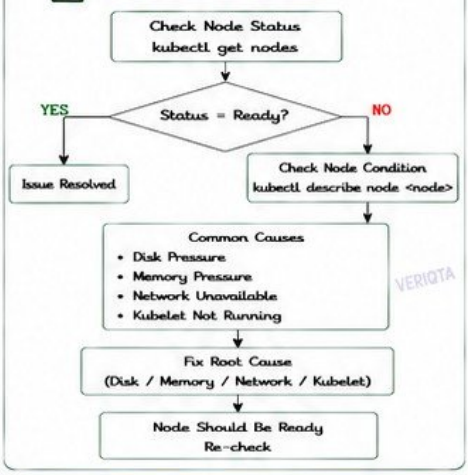
3. SERVICE NOT WORKING



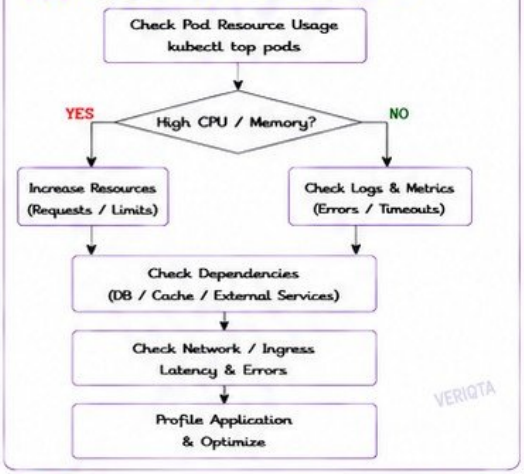
4. IMAGE PULL ERROR



5. NODE NOT READY



6. APP SLOW / HIGH LATENCY



UNIVERSAL TROUBLESHOOTING CHECKLIST

1	Identify the Problem	Understand the symptom clearly.
2	Gather Information	kubectl get, describe, logs, events.
3	Form a Hypothesis	What could be causing this?
4	Test the Hypothesis	Verify with evidence.
5	Isolate the Root Cause	Narrow down the exact issue.
6	Apply the Fix	Confirm the issue is resolved.
7	Verify the Fix	Update runbooks & share knowledge.

USEFUL COMMANDS (QUICK REFERENCE)

Purpose	Command
Get all resources in ns	kubectl get all -n <ns>
Describe a resource	kubectl describe <type> <name> -n <ns>
Get events	kubectl get events -n <ns> --sort-by=.lastTimestamp
Check pod logs	kubectl logs <pod> -n <ns>
Check previous logs	kubectl logs <pod> -n <ns> --previous
Execute in pod	kubectl exec -it <pod> -n <ns> -- /bin/bash
Check node status	kubectl get nodes -o wide
Check resource usage	kubectl top nodes / pods -n <ns>
Check endpoints	kubectl get endpoints <svc> -n <ns>

SENIOR TIP

- ✓ Follow a consistent process every time.
- ✓ Good tools + good process = fast resolution.
- ✓ Don't guess. Verify.
- ✓ Document what you learn.
- ✓ Automation is the best prevention.

VERIQTA

COMMON TOOLS

- kubectl
- k9s
- stern
- kube-shell / krew
- Prometheus
- Grafana
- Jaeger
- Loki

KEY TAKEAWAYS

- ✓ Methodical troubleshooting saves time and stress.
- ✓ Most issues follow patterns—learn them.
- ✓ Collect data first, act second.
- ✓ Practice these flowcharts until they become natural.
- ✓ Great SREs solve problems and prevent them.

TROUBLESHOOTING GOLDEN RULES

- ✓ Check the basics first (status, events, logs).
- ✓ Narrow down before diving deep.
- ✓ One change at a time.
- ✓ Understand the WHY, not just the HOW.
- ✓ Share knowledge and improve the system.

58. COMMAND REFERENCE FOR THE EXAM

High-yield Kubernetes commands you must know cold.

Memorize. Practice. Use them in the right context.

QUICK NOTES

- These commands are exam favorites.
- Know what each command does.
- Understand the output.
- Combine with flags and filters.
- Practice daily until it becomes automatic.

VERIQTA

1. CLUSTER INFORMATION

Command	Purpose
kubectl cluster-info	Displays cluster endpoints
kubectl get nodes -o wide	List all nodes with details
kubectl get componentstatuses	Check control plane components
kubectl get cs	Short form for componentstatuses
kubectl get nodes -o wide	Show nodes with IP and OS
kubectl describe node <node-name>	Detailed node information
kubectl get namespaces	List all namespaces
kubectl get ns	Short form for namespaces
kubectl config current-context	Show current context
kubectl config get-contexts	List all contexts

2. WORKLOADS

Command	Purpose
kubectl get pods	List pods in current namespace
kubectl get pods -o wide	Pods with node and IP info
kubectl describe pod <pod-name>	Detailed pod information
kubectl logs <pod-name>	View pod logs
kubectl logs -f <pod-name>	Stream live logs
kubectl exec -it <pod> -- /bin/bash	Access pod shell
kubectl get deployments	List deployments
kubectl describe deployment <name>	Deployment details
kubectl get replicasets	List ReplicaSets
kubectl get statefulsets	List StatefulSets
kubectl get daemonsets	List DaemonSets
kubectl get jobs	List Jobs
kubectl get cronjobs	List CronJobs

3. SERVICES & NETWORKING

Command	Purpose
kubectl get svc	List services
kubectl get svc -o wide	Services with details
kubectl describe svc <svc-name>	Service details
kubectl get endpoints	List all endpoints
kubectl get ep <svc-name> -o wide	Endpoints for a service
kubectl get ingress	List Ingress resources
kubectl describe ingress <name>	Ingress details
kubectl get networkpolicy	List NetworkPolicies
kubectl describe networkpolicy <name>	NetworkPolicy details
kubectl get pods -l <label-key>=<val>	Filter pods by label
kubectl get svc -l <label-key>=<val>	Filter services by label
kubectl port-forward <pod> 8080:80	Forward local port

4. CONFIGURATION

Command	Purpose
kubectl get configmap	List ConfigMaps
kubectl describe configmap <name>	ConfigMap details
kubectl get secret	List Secrets
kubectl describe secret <name>	Secret details
kubectl get pvc	List PersistentVolumeClaims
kubectl describe pvc <name>	PVC details
kubectl get pv	List PersistentVolumes
kubectl get storageclass	List StorageClasses
kubectl describe storageclass <name>	StorageClass details

5. NAMESPACES & CONTEXT

Command	Purpose
kubectl get ns	List namespaces
kubectl create namespace <name>	Create namespace
kubectl delete namespace <name>	Delete namespace
kubectl config get-contexts	List contexts
kubectl config use-context <name>	Switch context
kubectl config current-context	Show current context
kubectl config set-context --current --namespace=<ns>	Set namespace for context

6. MONITORING & DEBUGGING

Command	Purpose
kubectl top nodes	Node resource usage
kubectl top pods	Pod resource usage
kubectl get events --sort-by=.metadata.creationTimestamp	Recent events
kubectl describe pod <pod-name>	Events & pod details
kubectl get pods --field-selector=status.phase=Failed	List failed pods
kubectl get pods --field-selector=status.phase=Pending	List pending pods

7. APPLICATION LIFECYCLE

Command	Purpose
kubectl apply -f <file.yaml>	Create or update resources
kubectl create -f <file.yaml>	Create resources
kubectl edit <resource> <name>	Edit a resource
kubectl delete -f <file.yaml>	Delete resources in file
kubectl delete <resource> <name>	Delete a resource
kubectl rollout status deployment/<name>	Check rollout status
kubectl rollout history deployment/<name>	View rollout history
kubectl rollout undo deployment/<name>	Rollback deployment

8. ADVANCED / HELPFUL

Command	Purpose
kubectl explain <resource>	Explain resource fields
kubectl explain pod.spec	Explain pod.spec fields
kubectl api-resources	List all API resources
kubectl api-versions	List API versions
kubectl get all	List all common resources
kubectl get all -A	All resources in all namespaces
kubectl get pods --show-labels	Show labels for pods
kubectl get pods -o yaml	Output in YAML
kubectl get pods -o json	Output in JSON

9. LOGGING & DIAGNOSTICS

Command	Purpose
kubectl logs <pod> -c <container>	Logs for a container
kubectl logs <pod> --previous	Logs from previous container
kubectl logs <pod> -f --tail=100	Last 100 lines, follow
kubectl exec <pod> -- env	Show environment vars
kubectl get pods -o jsonpath='{.items[*].metadata.name}'	List pod names (JSONPath)
kubectl get pods -oname | wc -l	Count pods in namespace

COMMON FLAGS (USE OFTEN)

```
-A, --all-namespaces     # Across all namespaces
-o wide                  # Wide output
-o yaml                  # YAML output
-o json                  # JSON output
-f <file>                # File input
-l key=value             # Label selector
--sort-by=...            # Sort output
--field-selector=...     # Field based filtering
```

KEY TAKEAWAYS

- ✓ Know these commands and what they do.
- ✓ Understand the output, not just the command.
- ✓ Use flags to filter, format, and sort.
- ✓ Practice in a real cluster or local environment.
- ✓ Speed + Accuracy + Understanding = Success.

SENIOR TIP

- ✓ Don't just memorize — understand.
- ✓ Practice real scenarios.
- ✓ Combine commands.
- ✓ Read the output carefully.
- ✓ Troubleshoot like a senior engineer.

VERIQTA

59. LAST-HOUR REVISION CHECKLIST

Use this checklist in the final hours before your exam.

Focus, review, and go in with confidence.

QUICK NOTES

- Review high-yield topics only.
- Practice key commands mentally.
- Focus on understanding, not memorizing.
- Stay calm, hydrated, and confident.
- You've prepared. You've got this!

VERIQTA

1. CLUSTER FUNDAMENTALS

- [ ] Understand Kubernetes architecture
- [ ] (Control Plane & Worker Nodes)
- [ ] kubectl basic commands
- [ ] Cluster state: desired vs. actual
- [ ] Nodes, Pods, Components, Health
- [ ] Namespaces and contexts
- [ ] Resource management
- [ ] High Availability concepts

HIGH-YIELD COMMANDS

```
kubectl cluster-info
kubectl get nodes -o wide
kubectl get pods -A
kubectl get componentstatuses
kubectl config get-contexts
```

2. WORKLOADS

- [ ] Pods: create, view, logs, exec
- [ ] Deployments: create, scale, update, rollout
- [ ] ReplicaSets, StatefulSets, DaemonSets
- [ ] Jobs and CronJobs
- [ ] Understand labels and selectors
- [ ] Health Probes: Liveness, Readiness, Startup
- [ ] Resource requests and limits

HIGH-YIELD COMMANDS

```
kubectl run nginx --image=nginx
kubectl get pods -o wide
kubectl logs <pod-name>
kubectl describe pod <pod-name>
kubectl rollout status deployment/<name>
```

3. SERVICES & NETWORKING

- [ ] Service types: ClusterIP, NodePort, LoadBalancer
- [ ] Endpoints and EndpointSlices
- [ ] Ingress: basic concepts and rules
- [ ] NetworkPolicies: allow/deny traffic
- [ ] DNS in Kubernetes
- [ ] CoreDNS and service discovery
- [ ] Troubleshoot connectivity

HIGH-YIELD COMMANDS

```
kubectl get svc
kubectl get endpoints <svc-name>
kubectl get ingress -A
kubectl describe ingress <name>
kubectl run -it --rm test --image=busybox --
nslookup <service>
```

4. CONFIGURATION & STORAGE

- [ ] ConfigMaps: create, use, mount
- [ ] Secrets: create, use, mount
- [ ] PersistentVolumes (PV)
- [ ] PersistentVolumeClaims (PVC)
- [ ] StorageClasses
- [ ] Volume types and access modes
- [ ] Mount volumes in Pods

HIGH-YIELD COMMANDS

```
kubectl create configmap <name>
--from-literal=key=value
kubectl create secret generic <name>
--from-literal=key=value
kubectl get pvc
kubectl get pv
kubectl describe pvc <name>
```

5. ADMINISTRATION & MAINTENANCE

- [ ] Add, cordon, drain, uncordon nodes
- [ ] Taints and tolerations
- [ ] RBAC: Roles, RoleBindings, ClusterRoles
- [ ] ServiceAccounts
- [ ] kubectl auth can-i
- [ ] Pod disruption and eviction
- [ ] Cluster upgrades (basics)

HIGH-YIELD COMMANDS

```
kubectl cordon <node-name>
kubectl drain <node-name> --ignore-daemonsets --
-delete-emptydir-data
kubectl uncordon <node-name>
kubectl get roles,rolebindings -A
kubectl auth can-i --list
```

6. MONITORING & LOGGING

- [ ] Resource usage: top nodes, pods
- [ ] Events: view and understand
- [ ] Logs: containers, pods, system
- [ ] Metrics Server
- [ ] Health checks and probes
- [ ] Basic observability workflow
- [ ] Log locations and access

HIGH-YIELD COMMANDS

```
kubectl top nodes
kubectl top pods
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl logs <pod-name> -c <container> --tail=100
kubectl describe pod <pod-name>
```

7. SECURITY

- [ ] ServiceAccounts and tokens
- [ ] RBAC: least privilege
- [ ] NetworkPolicies
- [ ] Secrets management
- [ ] Pod Security Standards (basics)
- [ ] Image security (private registries)
- [ ] TLS for Ingress (basics)

HIGH-YIELD COMMANDS

```
kubectl get sa -A
kubectl get roles,rolebindings -A
kubectl get networkpolicies -A
kubectl get secrets -A
kubectl describe networkpolicy <name>
```

8. TROUBLESHOOTING

- [ ] Pods not starting (CrashLoopBackOff, ImagePullBackOff)
- [ ] Pod not running
- [ ] Service not working
- [ ] Ingress not accessible
- [ ] DNS issues
- [ ] Node not ready
- [ ] Use events, logs, describe effectively

HIGH-YIELD COMMANDS

```
kubectl get pods -A
kubectl describe pod <pod-name>
kubectl logs <pod-name> -c <container> --previous
kubectl get nodes
kubectl describe node <node-name>
```

LAST-HOUR GAME PLAN

1. Review this checklist top to bottom.
2. Read high-yield commands out loud.
3. Visualize common scenarios.
4. Focus on understanding, not syntax.
5. Stay calm. Breathe. You're ready.

YOU'VE PREPARED. NOW GO WIN!

MOST IMPORTANT FLAGS

-A	All namespaces
-n <ns>	Specific namespace
-o wide	Wide output
-o yaml	YAML output
-o json	JSON output
--kubeconfig	Use specific config
--context	Use specific context

EXAM SUCCESS FORMULA

Understand + Practice + Focus + Stay Calm = SUCCESS

"Preparation is the difference between anxiety and confidence."

SENIOR TIP

- ✓ Eliminate wrong answers first.
- ✓ Look for keywords in questions.
- ✓ Use kubectl describe wisely.
- ✓ Don't guess. Think logically.
- ✓ Time management is important.

VERIQTA

60. FINAL CKA READINESS ASSESSMENT

Your complete final review to confirm you are ready for the Kubernetes Certified Administrator (CKA) exam.

QUICK NOTES

- This is your final readiness checkpoint.
- Be honest with yourself.
- Fix weak areas now.
- Trust your preparation.
- Success is the result of consistent effort.

VERIQTA

1. COMPLETE EXAM-DAY CHECKLIST

- [ ] System working and updated
- [ ] Stable internet connection
- [ ] kubectl working and configured
- [ ] Practice environment ready
- [ ] No distractions during exam
- [ ] Exam ID and email confirmed
- [ ] Time zone and schedule confirmed
- [ ] Water, snack, and comfort ready
- [ ] Calm mind and positive mindset
- [ ] Confidence and focus locked in
- [ ] You're ready. Let's do this!

2. COMMON FAILURE PATTERNS

1. Not reading the question carefully
   → Leads to wrong approaches.
2. Wrong namespace
   → Work done in the wrong place.
3. Syntax errors
   → Small mistakes cost big time.
4. Not verifying results
   → Always check what you built.
5. Over-complicating solutions
   → Keep it simple and effective.
6. Forgetting labels/selectors
   → Services and Deployments break.
7. Incomplete solutions
   → Don't stop before requirement is met.
8. Ignoring resource limits
   → Causes scheduling and stability issues.
9. Not using correct resources
   → Use the best fit for the task.
10. Panicking during the exam
    → Breathe. Reset. Focus.

3. SENIOR ADMINISTRATOR PRINCIPLES

1. **Understand before you act**
   Analyze the problem. Plan the solution.
2. **Use the right tool**
   Know which resource fits the requirement.
3. **Keep it simple**
   Simplicity is the sign of mastery.
4. **Verify your work**
   Always validate your configuration.
5. **Think like the system**
   Understand how Kubernetes behaves.
6. **Automate when possible**
   Use declarative configs and reusability.
7. **Secure by default**
   Security is everyone's responsibility.
8. **Obsering and learn**
   Use logs, events, and metrics.
9. **Document in your mind**
   Remember key steps and patterns.
10. **Continuously improve**
    A good admin never stops learning.

4. CONFIDENCE CHECKLIST

Rate yourself: 1 (Weak) to 5 (Excellent)

Cluster fundamentals	1	2	3	4	5
Workloads & controllers	1	2	3	4	5
Services & networking	1	2	3	4	5
Storage & configuration	1	2	3	4	5
Troubleshooting skills	1	2	3	4	5
Security & access	1	2	3	4	5
Monitoring & logging	1	2	3	4	5
Speed & accuracy	1	2	3	4	5
Problem-solving mindset	1	2	3	4	5
Overall readiness	1	2	3	4	5

INTERPRETATION GUIDE

45 - 50	→	You are exam ready!
35 - 44	→	Review weak areas.
25 - 34	→	More practice needed.
Below 25	→	Focus and rebuild.

5. FINAL READINESS SELF-ASSESSMENT

I can confidently:

- [ ] Create, modify, and troubleshoot workloads.
- [ ] Expose applications using Services & Ingress.
- [ ] Manage storage, PVs, and ConfigMaps.
- [ ] Secure access using RBAC.
- [ ] Troubleshoot Pods, Deployments, and Nodes.
- [ ] Monitor and analyze cluster health.
- [ ] Work efficiently with kubectl.
- [ ] Solve problems under time pressure.
- [ ] Use logs, events, and describe for debugging.
- [ ] Build complete solutions from scratch.

IF YOU CHECKED ALL BOXES:

You are prepared.
Trust your practice.
Go and win!

6. EXAM-DAY SUCCESS STRATEGY

1. Read the task → Understand the goal.
2. Break it down → Identify key requirements.
3. Plan the solution → Choose the right resources.
4. Execute → Build step by step.
5. Verify → Validate your work.
6. Optimize → Improve if time allows.
7. Final check → Ensure everything works.
8. Submit with confidence!

SENIOR TIP

- ✓ Stay calm. Read. Think. Execute.
- ✓ Speed comes from experience and practice.
- ✓ You don't need to know everything.
- ✓ You need to solve the problem.
- ✓ You've got this. Believe in yourself.

VERIQTA

7. FINAL VERIQTA EXAM SUCCESS FRAMEWORK

KNOWLEDGE	+	PRACTICE	+	ANALYSIS	+	EXECUTION	+	VERIFICATION	=	EXAM SUCCESS
Understand the concepts and how Kubernetes works.		Hands-on practice builds speed and confidence.		Break down problems and identify what matters.		Apply the right commands and build clean solutions.		Always check, test, and confirm your results.		Consistent effort + Right mindset = CKA Certified Administrator

PREPARE SMART. PRACTICE CONSISTENTLY. SOLVE CONFIDENTLY. SUCCEED ALWAYS.